CS-663
Digital Image Processing (DIP)

Image Enhancement: Image Denoising

Suyash P. Awate

Noise Model

- A simple statistical model for noise:
  i.i.d. zero-mean additive Gaussian perturbations
- $Y_i := X_i + Z_i$
  - X_i = true/noiseless value at pixel i
  - Y_i = observed/acquired datum at pixel i
  - Z_i is Gaussian random variable with **mean 0**
  - Z_i and X_i are **independent** for any i
  - Z_i and Z_j are **independent** for any i,j

$$\varphi(z) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

- Why ?
  1) Thermal noise in electronic measurements
  2) Poisson-noise distributional form becomes similar to Gaussian for large λ
  3) Central limit theorems for sum of independent/dependent random variables

Spatial Filtering

- Limitation of weighted-mean/Gaussian filters
  - If we desire a sufficient reduction in noise,
    then they can fail to preserve edges (can over-smooth)
- Limitation of median filter
  - Can introduce artifacts, when using small neighborhoods
  - Also causes some degradation of corners (as we saw before)

noisy

Gaussian filter

Median filter

Spatial Filtering

- What if we convolve image
  with Gauss(0,V), and then again with Gauss(0,U)
- (Image * Gauss(0,V)) * Gauss(0,U) = Image * (Gauss(0,V) * Gauss(0,U))
- What is Gauss(0,V) * Gauss(0,U) ?
- This is like convolving 2 probability density functions (PDFs) of
  independent zero-mean Gaussian random variables X and Y
- This happens when we perform X + Y to get Z
- We know that Z has Gaussian PDF Gauss(0,V+U)
- So, Gauss(0,V) * Gauss(0,U) = Gauss(0,V+U)
- So, repeated Gaussian convolutions
  equivalent to single Gaussian convolution with larger variance

Spatial Filtering

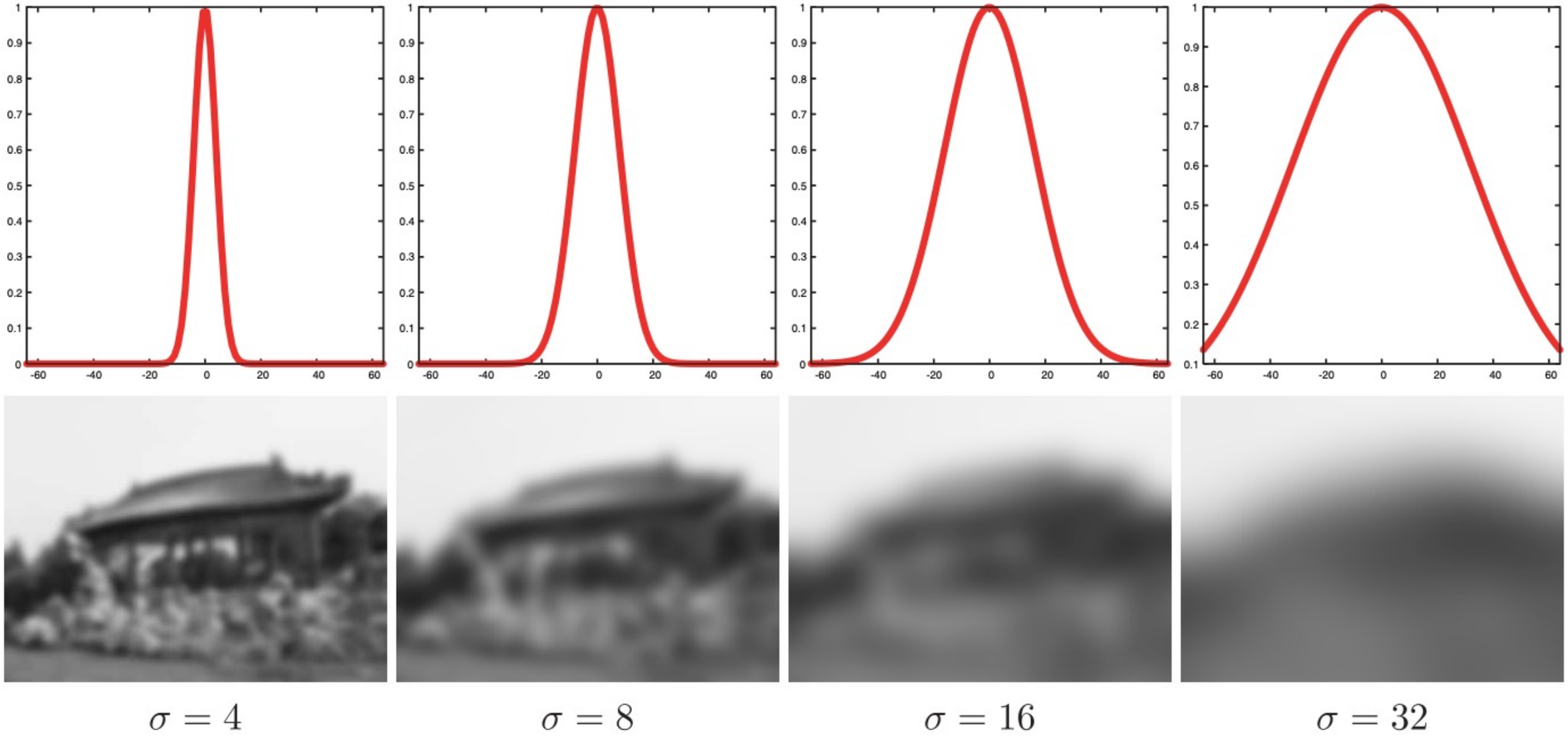


Fig. 2.1 Example of Gaussian linear filtering with different σ. Top row shows the profile of a 1D Gaussian kernel and bottom row the result obtained by the corresponding 2D Gaussian convolution filtering. Edges are lost with high values of σ because averaging is performed over a much larger area.

Spatial Filtering for Edge-Preserving Smoothing

- We desire a filter that:

1) Reduces noise
2) Preserves object edges within image

- Price we are willing to pay
  - Filtered value is a nonlinear function of pixel intensities
    (well, even median filter was nonlinear)
  - Filter cannot be represented as convolution with original image
  - May increase computational load
    (even more than that of median filter)

Edge-Preserving Smoothing

- General idea underlying any kind of smoothing
- At any chosen pixel,
  smoothed intensity := weighted average of all pixel intensities in image
- Key questions
  1) What are the weights ?
  2) Are some weights zero ? What neighborhood to choose ?
- Examples
  - Mean filter: weights $\propto$ 1 within neighborhood, 0 elsewhere
  - Weighted-means filter: weights $\propto$ Gaussian within neighborhood
  - Median filter: weight = 1 for only that neighboring pixel whose value is median

Kuwahara Filter

PROCESSING OF RI-ANGIOCARDIOGRAPHIC IMAGES

M. Kuwahara, K. Hachimura, S. Eiho & M. Kinoshita*

Automation Research Laboratory, Kyoto University, Kyoto

*Shiga Medical School, Otsu

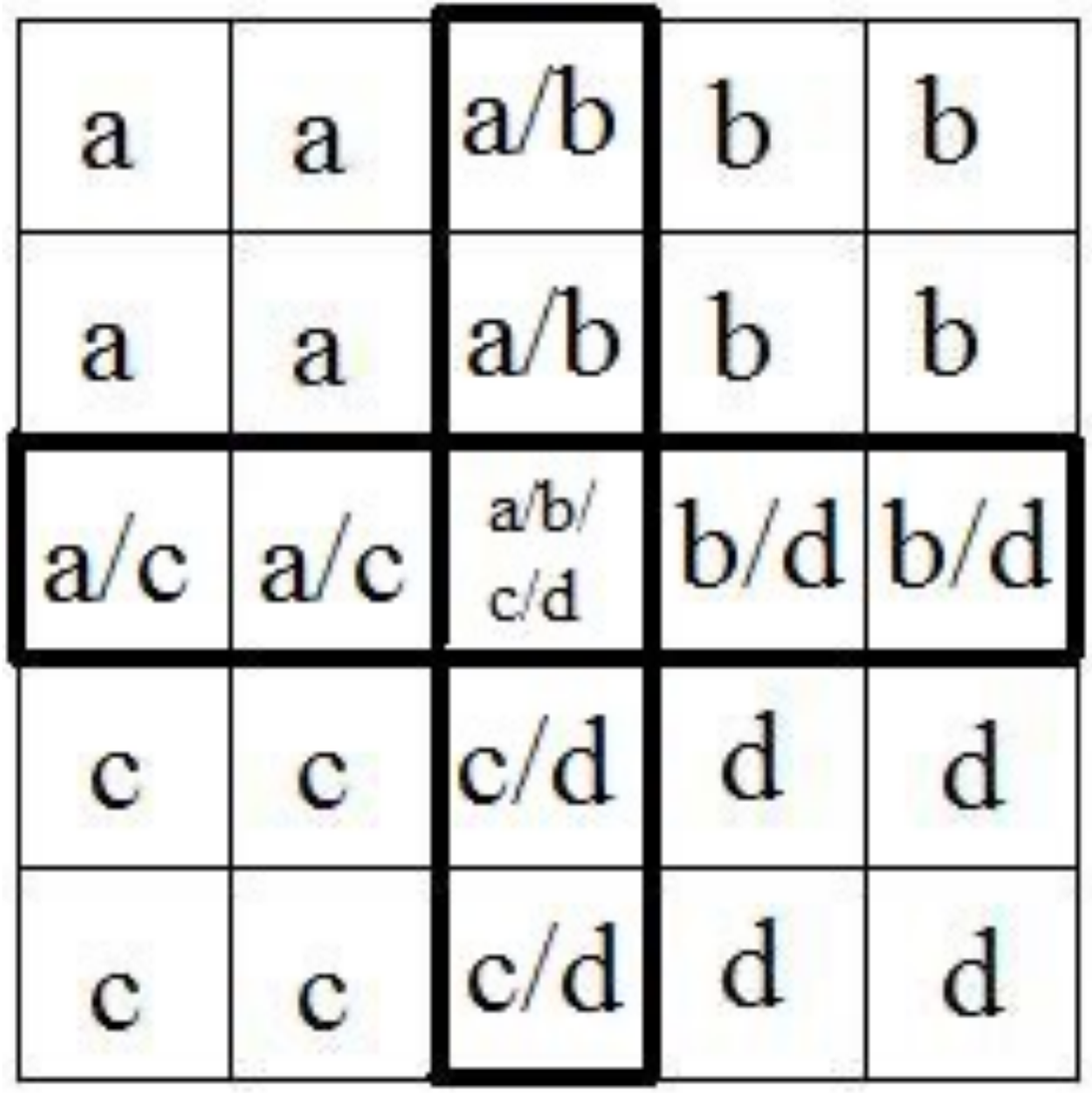
K. Preston Jr. et al. (eds.), *Digital Processing of Biomedical Images*
© Plenum Press, New York 1976

Kuwahara Filter

- Algorithm:

1) Neighborhood around pixel (x,y) divided into 4 square regions, {a, b, c, d} or {1, 2, 3, 4}, where:

- center pixel belongs to all 4 regions
- other pixels in central row & column belong to 2 regions



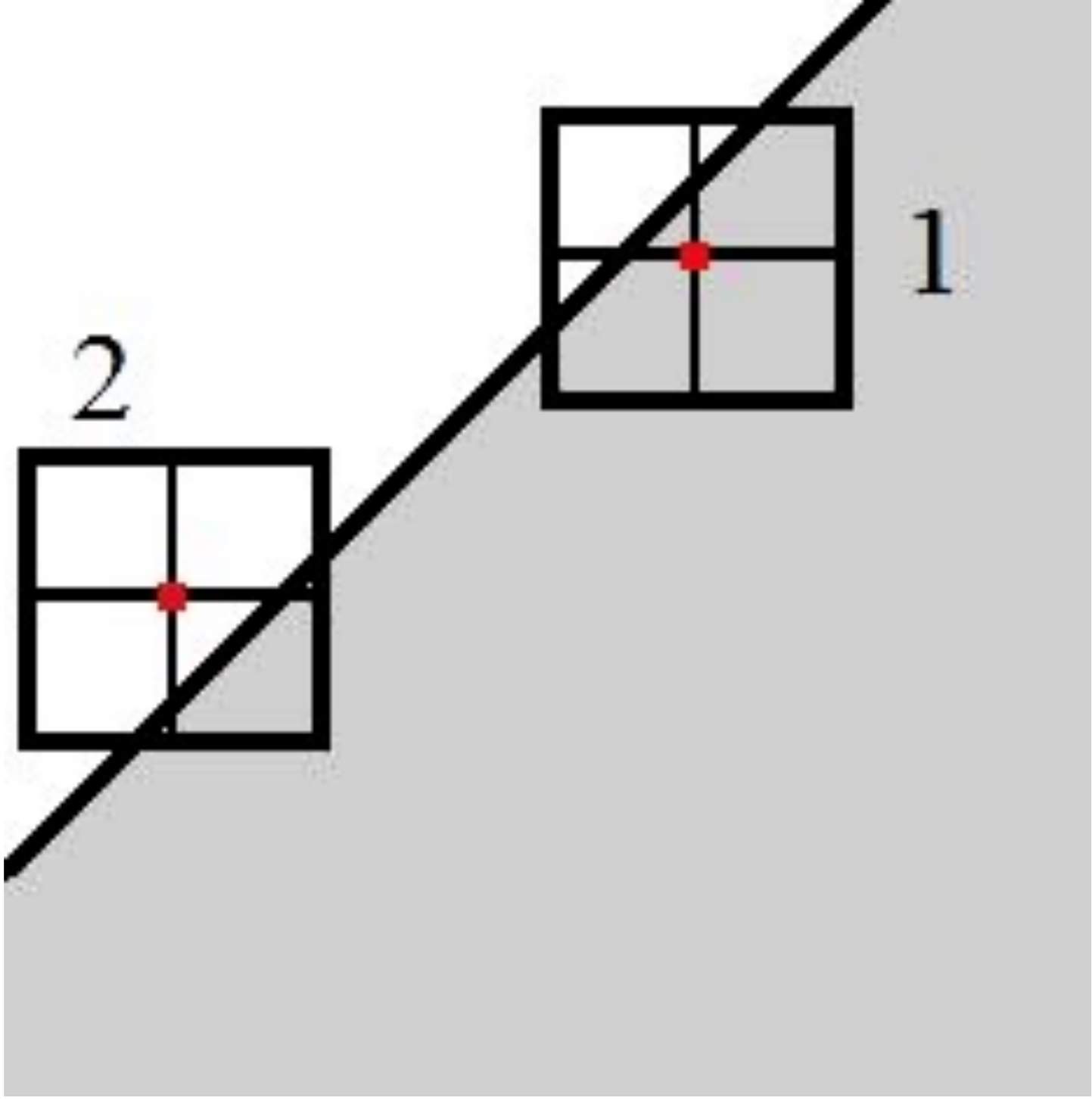
2) Find mean $m_i(x,y)$ & standard deviation $\sigma_i(x,y)$ of regions around (x,y)

3) Output intensity at (x,y) := $m_k(x,y)$ where $k := \arg\min_i \sigma_i(x,y)$

- So, center pixel takes mean value of region that is most homogenous

Kuwahara Filter

- Averaging leads to noise reduction
- Region selection leads to edge preservation
- Case 1: red location will take value gray
- Case 2: red location will take value white



Kuwahara Filter

- Top left: original
- Top right: 5x5 neighborhood
- Bottom left: 9x9 neighborhood
- Bottom right: 15x15 neighborhood

Kuwahara Filter

- For RGB color images, cannot apply independently to each channel
  - Red-channel min-variance region may differ from region for blue channel
  - May lead to color distortion
- For color images, apply filter to luminance channel only (in suitable color space)

Kuwahara

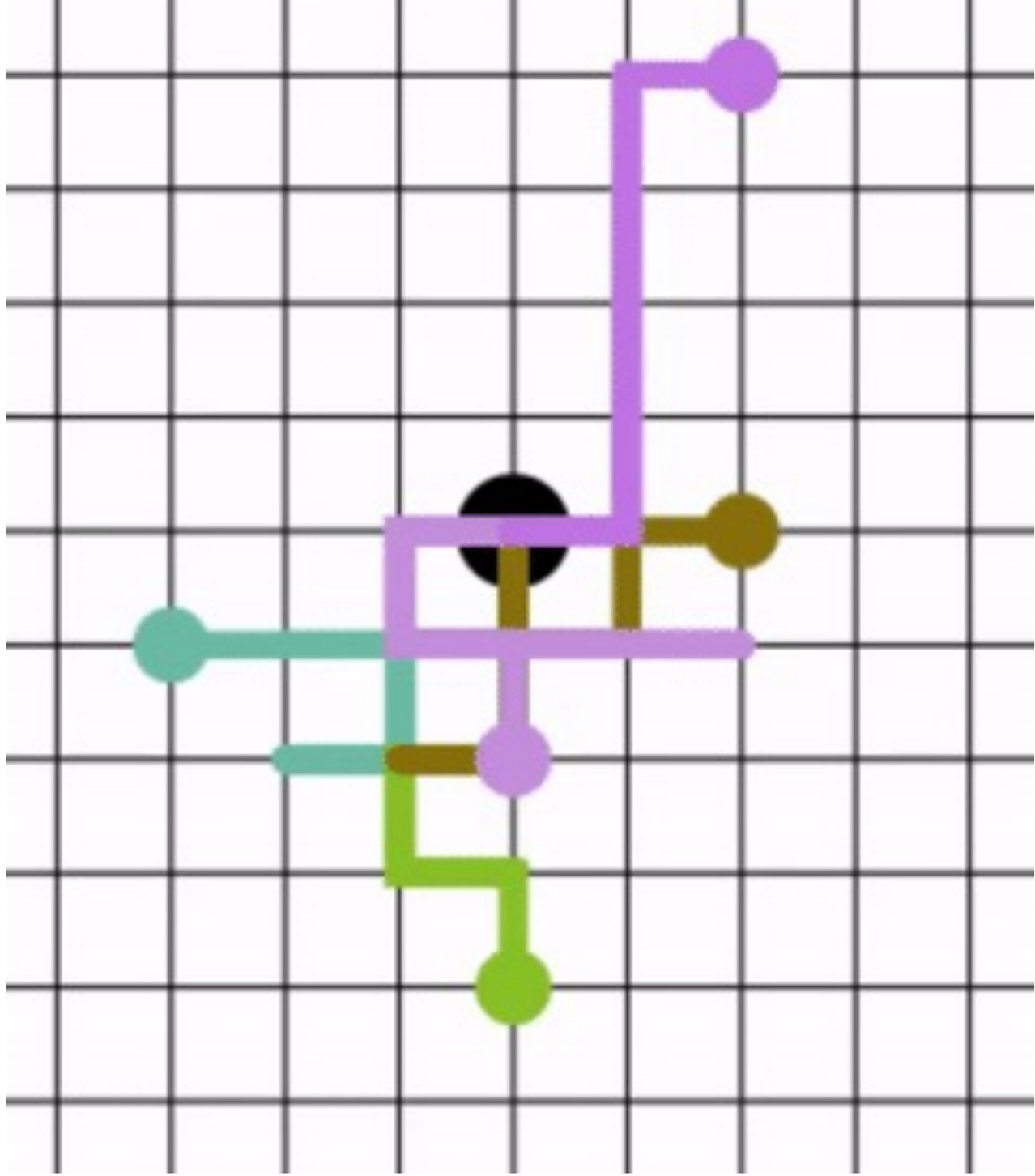
- Creates block artifacts for large neighbor-hoods

Kuwahara Filter

- Block artifacts clearly visible
- Has been actually used more towards segmentation than denoising

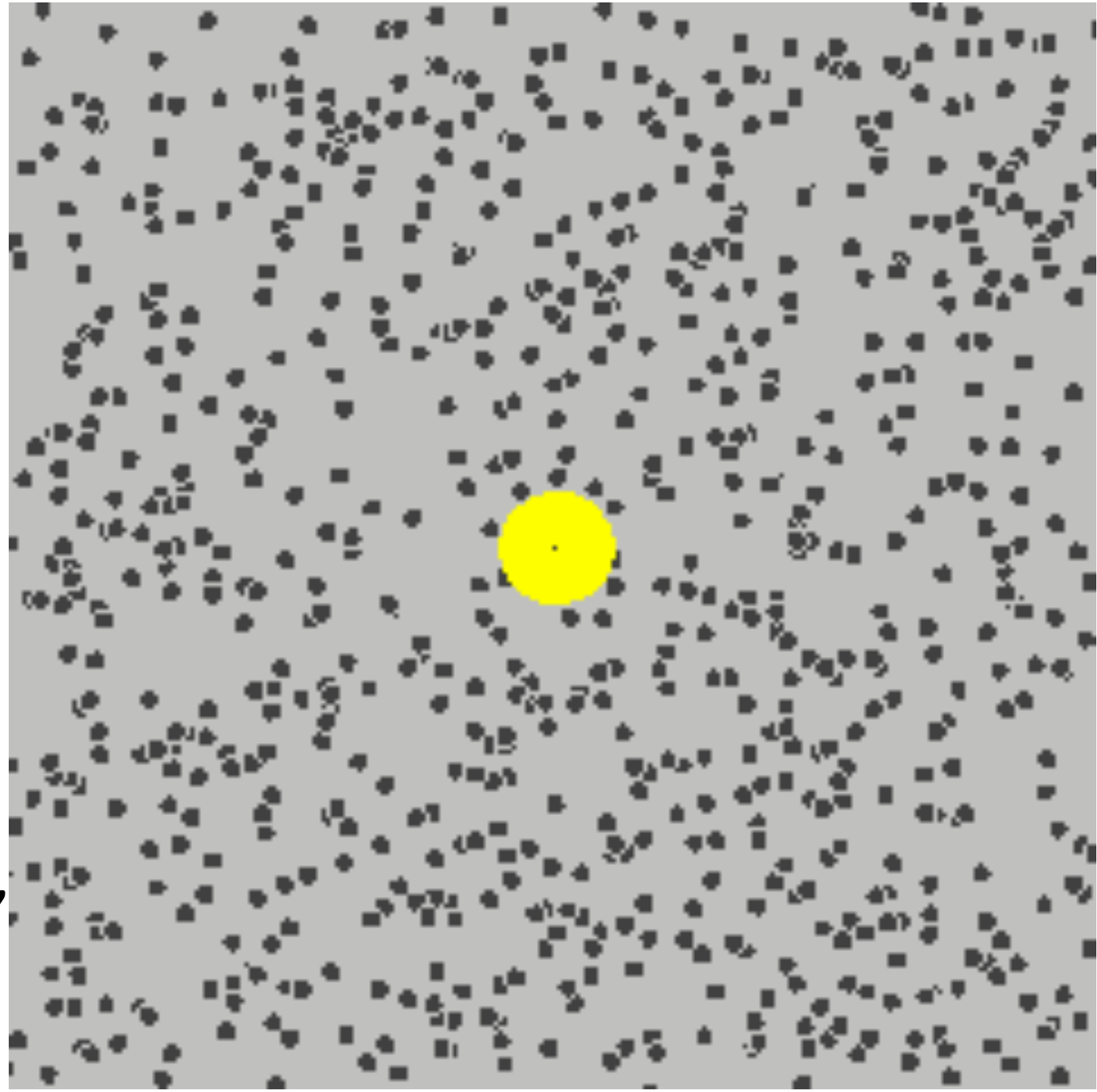
Isotropic Diffusion

- Random walk, drunkard's walk
- In 1D:
  walk starts at 0;
  at each time step Δt,
  walker moves +Δx or −Δx
  with equal probability
  - Similarly in higher dimensions
- On right, in 2D:
  5 different instances of
  8-step random walks
  starting at origin (black dot)



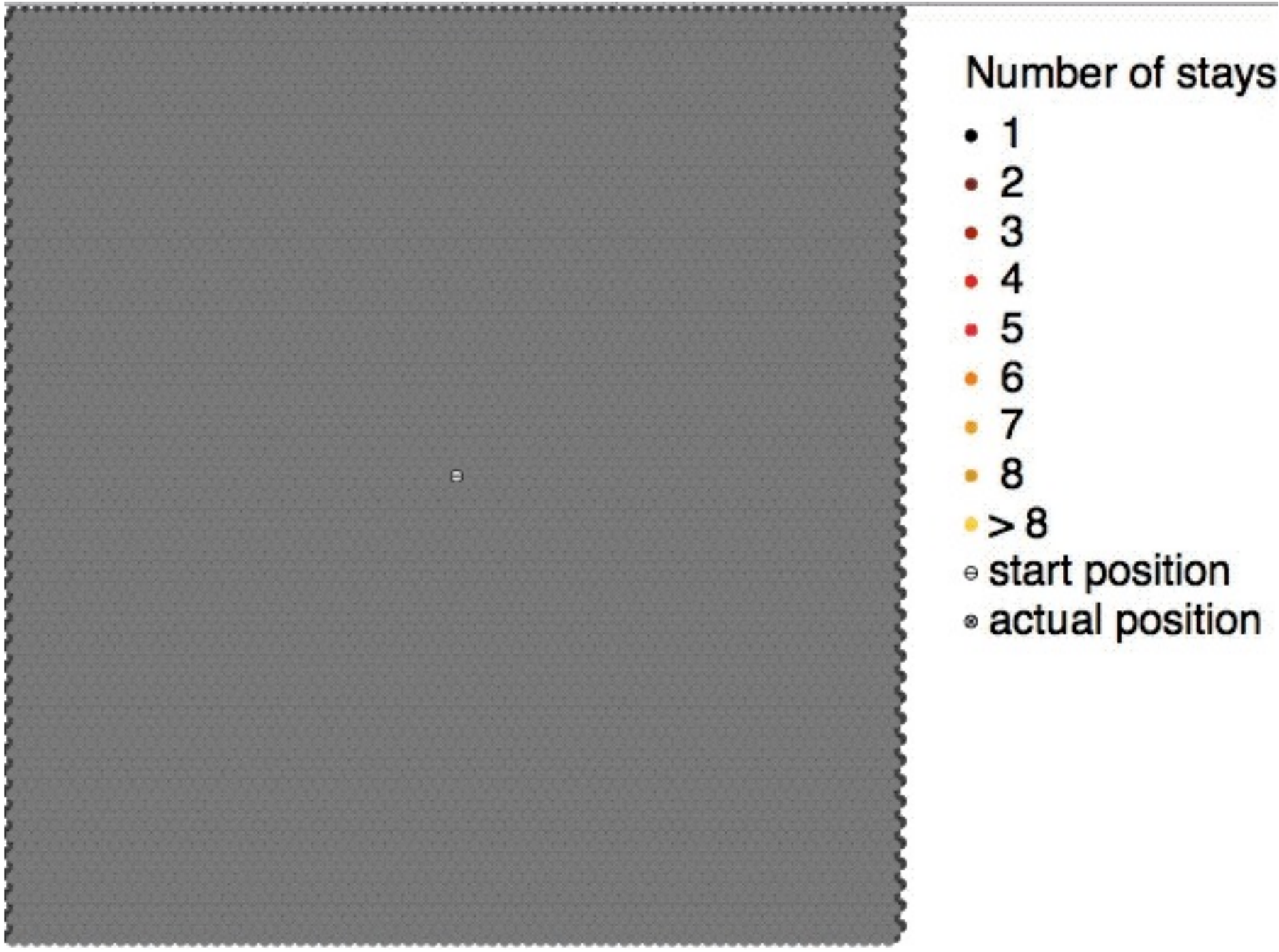
Isotropic Diffusion

- Brownian motion, Random walk
- Simulation of a dust particle, colliding with large set of **smaller gas molecules** that move with different velocities in different random directions
- In continuous space and time ($\Delta x \to 0$, $\Delta t \to 0$): P(locationOfDustParticle) is a Gaussian with:
  mean = start location of particle,
  variance $\propto$ time



Isotropic Diffusion

- Brownian motion, Random walk
- 2-D random walk of silver adatom on Ag(111) surface

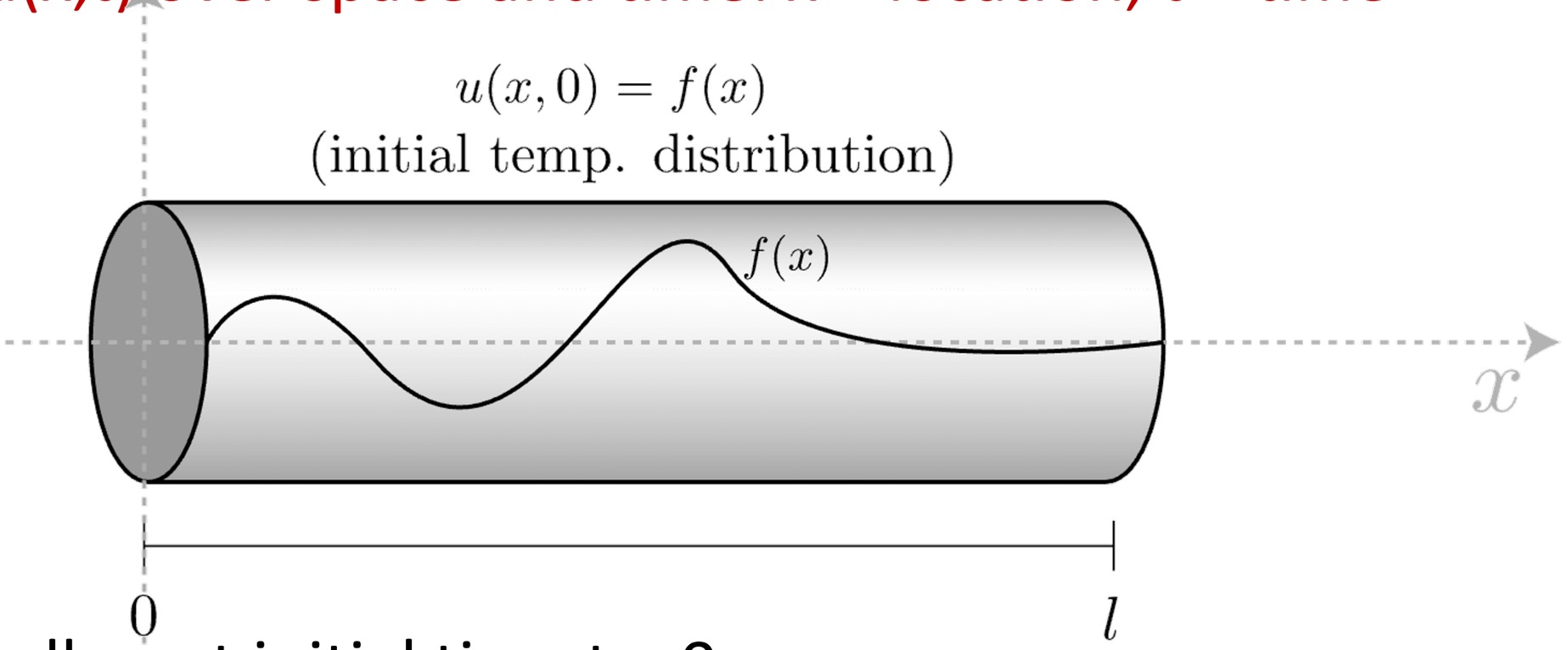


Isotropic Diffusion

- Intuition
- Consider image with non-zero intensity at single pixel, e.g., blue ink spot in water (or on paper)
- Consider that intensity (ink spot) spreading/diffusing across spatial domain, over time
- This diffusion can be modeled by Gaussian convolution
- Imagine multiple ink spots, of various colors, diffusing simultaneously and independently,
- This leads to convolution of image with Gaussian

Isotropic Diffusion: Heat Equation

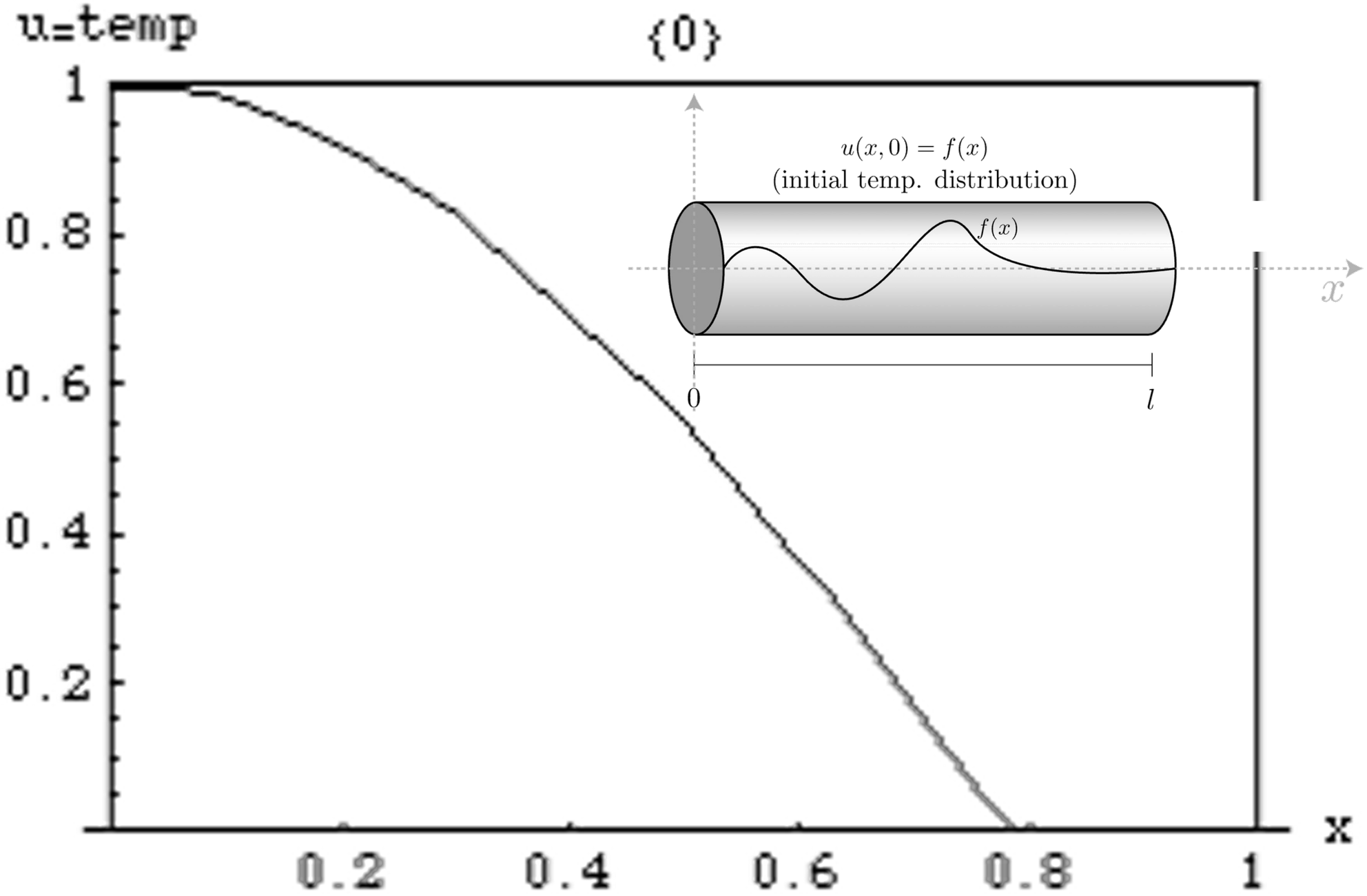
- Consider function u(x,t) over space and time: x = location, t = time
  - e.g., temperature distribution within object over time



- Given: u(x, t=0) for all x, at initial time t = 0
- Given: rate of change (in time) of u(.) ∝ Laplacian (over space) of u(.); i.e., $u_t(x,t) := k\, u_{xx}(x,t)$ where k > 0
- Find: u(x, t=T) for all x, for any specified time T > 0

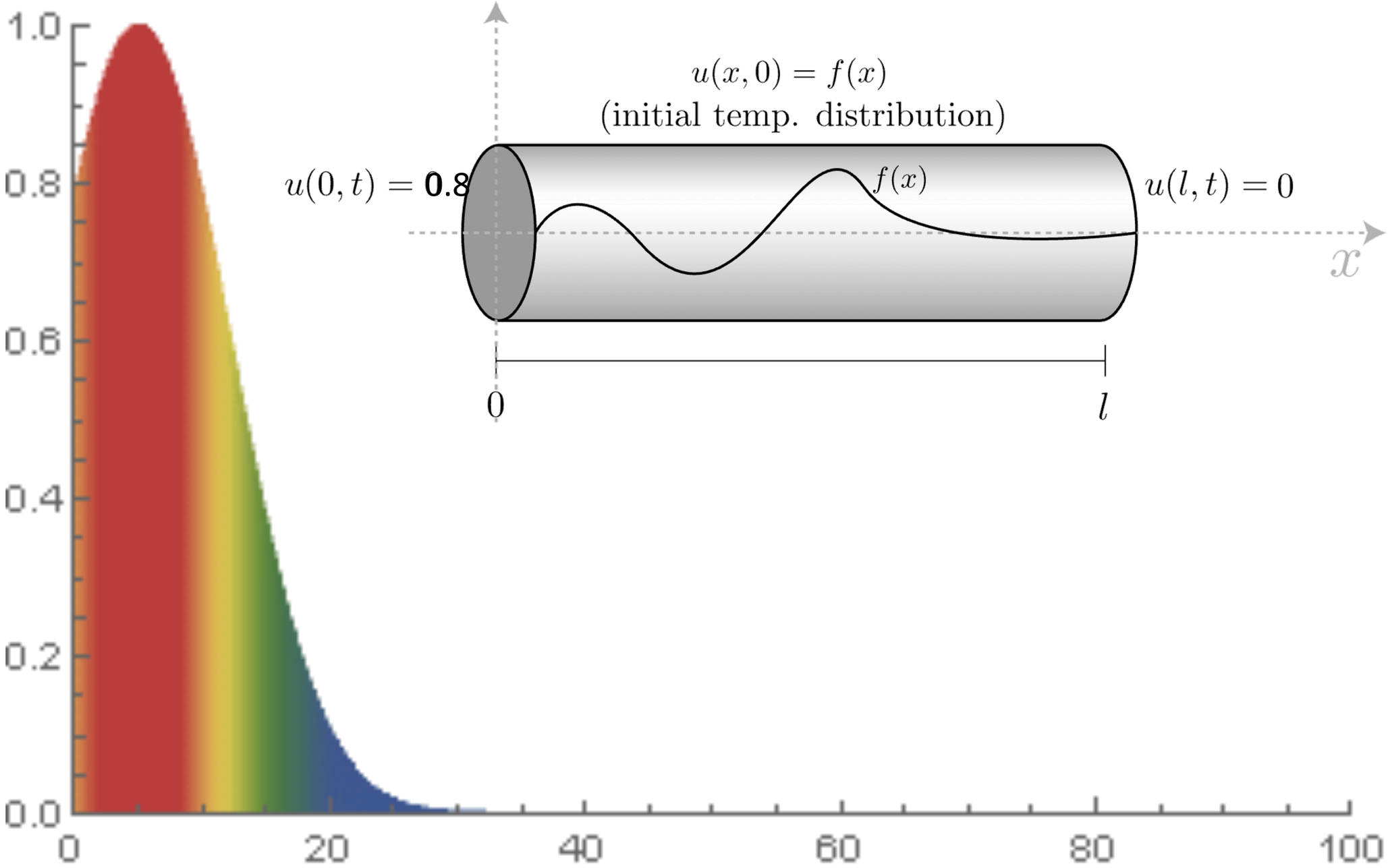
Isotropic Diffusion: Heat Equation

- Initial temperature distribution as shown
- Insulated endpoints



Isotropic Diffusion: Heat Equation

- Rod ends held/fixed at temperature 0.8 and 0



Isotropic Diffusion: Heat Equation

- Isotropic diffusion relates to “heat equation”

1) Gaussian function, of spacetime, with mean 0 (over x) & variance 2kt satisfies heat equation (ignore initial condition, for now)

$$S(x,t) = \frac{1}{\sqrt{4k\pi t}} e^{-x^2/(4kt)}$$

- Verify: $\frac{\partial S}{\partial t}(x,t) = k\frac{\partial^2 S}{\partial x^2}(x,t)$
- Partial derivative of S(x,t) w.r.t. t is $-\frac{(2kt - x^2)\, e^{-\frac{x^2}{4kt}}}{8\sqrt{\pi}\, t (kt)^{\frac{3}{2}}}$
- Partial 2nd-derivative of S(x,t) w.r.t. x is $\frac{(x^2 - 2kt)\, e^{-\frac{x^2}{4kt}}}{8\sqrt{\pi}\, k^2 t^2 \sqrt{kt}}$
- Gauss(x;0,2kt) is an eigenfunction of this generalized-eigenvalue problem with generalized eigenvalue k

Characteristic Polynomial, Eigenvalues, Eigenvectors

- "Characteristic" polynomial
  - … of a square matrix is a polynomial which is invariant under matrix similarity, and has eigenvalues as roots
  - For square matrix A, find non-zero vectors v such that $Av = \lambda v$
    → $(\lambda I - A)v = 0$ → $\det(\lambda I - A) = 0$ → "characteristic" equation
  - en.wikipedia.org/wiki/Characteristic_polynomial
- "Eigen"
  - In Dutch/German, means:
    "characteristic, typical, specific"
    "own, peculiar or private to someone, idiosyncratic, proper or proprietary"
  - en.wiktionary.org/wiki/eigen#German

Generalized Eigenvalue Problem

- Standard eigenvalue problem (matrix A as operator on vectors): $Av = \lambda v$
- When linear operator (e.g., derivative operator) is used with functions, i.e., $Df = \lambda f$, then we have notion of an eigenfunction
  - en.wikipedia.org/wiki/Eigenfunction e.g., $d/dx\ (\exp(x)) = \exp(x)$
- Generalized eigenvalue problem (matrix operators on vector): $Av = \lambda Bv$
  - A and B are matrices, i.e., linear operators
  - en.wikipedia.org/wiki/Eigendecomposition_of_a_matrix#Generalized_eigenvalue_problem
- Generalized eigenvalue problem with functions: $Df = \lambda Ef$
- "Generalized eigenvalue", "generalized eigenvector": overused terms
  - Also used in different context, for defective matrices: en.wikipedia.org/wiki/Generalized_eigenvector

Isotropic Diffusion: Heat Equation

- Isotropic diffusion relates to “heat equation”

2) Gaussian function, of spacetime, with shifted mean y & variance 2kt satisfies heat equation (ignore initial condition, for now)

$$S(x-y,t)=\frac{1}{\sqrt{4k\pi t}}e^{-(x-y)^2/(4kt)}$$

- Verify: $\frac{\partial S}{\partial t}(x,t)=k\frac{\partial^2 S}{\partial x^2}(x,t)$
- Partial derivative of S(x,t) w.r.t. t is $-\frac{\left(2kt-y^2+2xy-x^2\right)\mathrm{e}^{-\frac{(x-y)^2}{4kt}}}{8\sqrt{\pi}\,t(kt)^{\frac{3}{2}}}$
- Partial 2nd-derivative of S(x,t) w.r.t. x is $\frac{\left(x^2-2yx+y^2-2kt\right)\mathrm{e}^{-\frac{(x-y)^2}{4kt}}}{8\sqrt{\pi}\,k^2t^2\sqrt{kt}}$
- All Gauss(x;y,2kt) functions are eigenfunctions of this generalized-eigenvalue problem with same generalized eigenvalue k

Isotropic Diffusion: Heat Equation

- Isotropic diffusion relates to "heat equation"

$$u_t(x,t) = k u_{xx}(x,t), \quad -\infty < x < \infty, \quad t > 0$$

$$u(x,0) = f(x), \quad -\infty < x < \infty,$$

$$|u(x,t)| < M < \infty \quad \text{for some} \quad M \quad \forall\, x \in \mathbb{R}, \quad t \geq 0$$

3) Linear combinations of shifted (same-variance) Gaussians satisfy eqn.

- So, following Gaussian-convolved function also satisfies heat equation:

$$u(x,t) = \frac{1}{\sqrt{4k\pi t}} \int_{-\infty}^{\infty} e^{-(x-y)^2/(4kt)}\, f(y)\, dy$$

- Also, this solution matches initial-condition constraint:

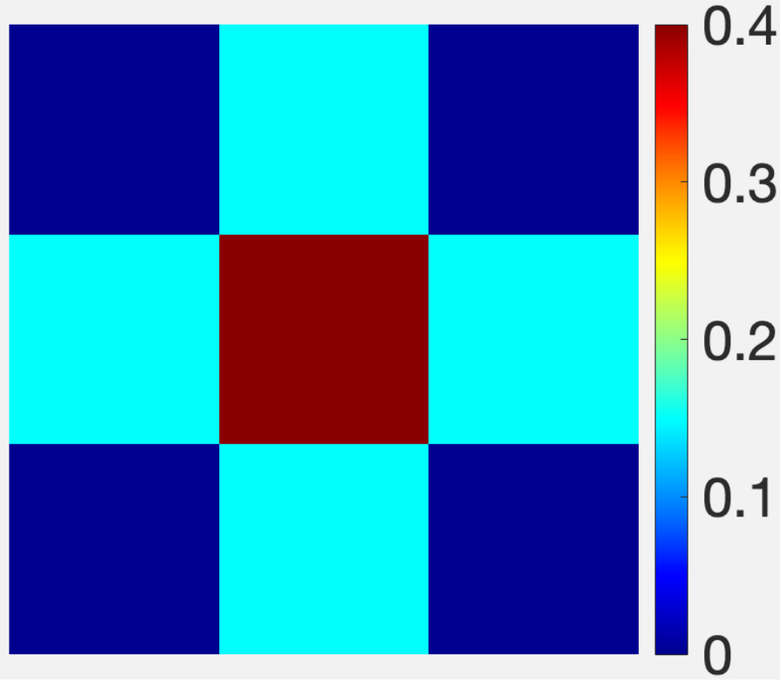
$$f(x) = u(x,0) = \lim_{t \downarrow 0} \frac{1}{\sqrt{4k\pi t}} \int_{-\infty}^{\infty} e^{-(x-y)^2/(4kt)}\, f(y)\, dy$$

Isotropic Diffusion: Heat Equation

- PDE-Solver View: discretization scheme in 2D: $\nabla_t f(x,y;t) = k \nabla^2_{x,y} f(x,y;t)$
  - Discretize time-derivative using forward difference
  - Discretize Laplacian using filter given earlier
- Let time-step := $\lambda > 0$; without loss of generality, set k := 1
- $f(x,y; t+1) := f(x,y; t) + \lambda [f(x+1,y; t) + f(x-1,y; t) + f(x,y+1; t) + f(x,y-1; t) - 4 f(x,y; t)]$
- LHS = convolution of f(x,y; t) with filter:

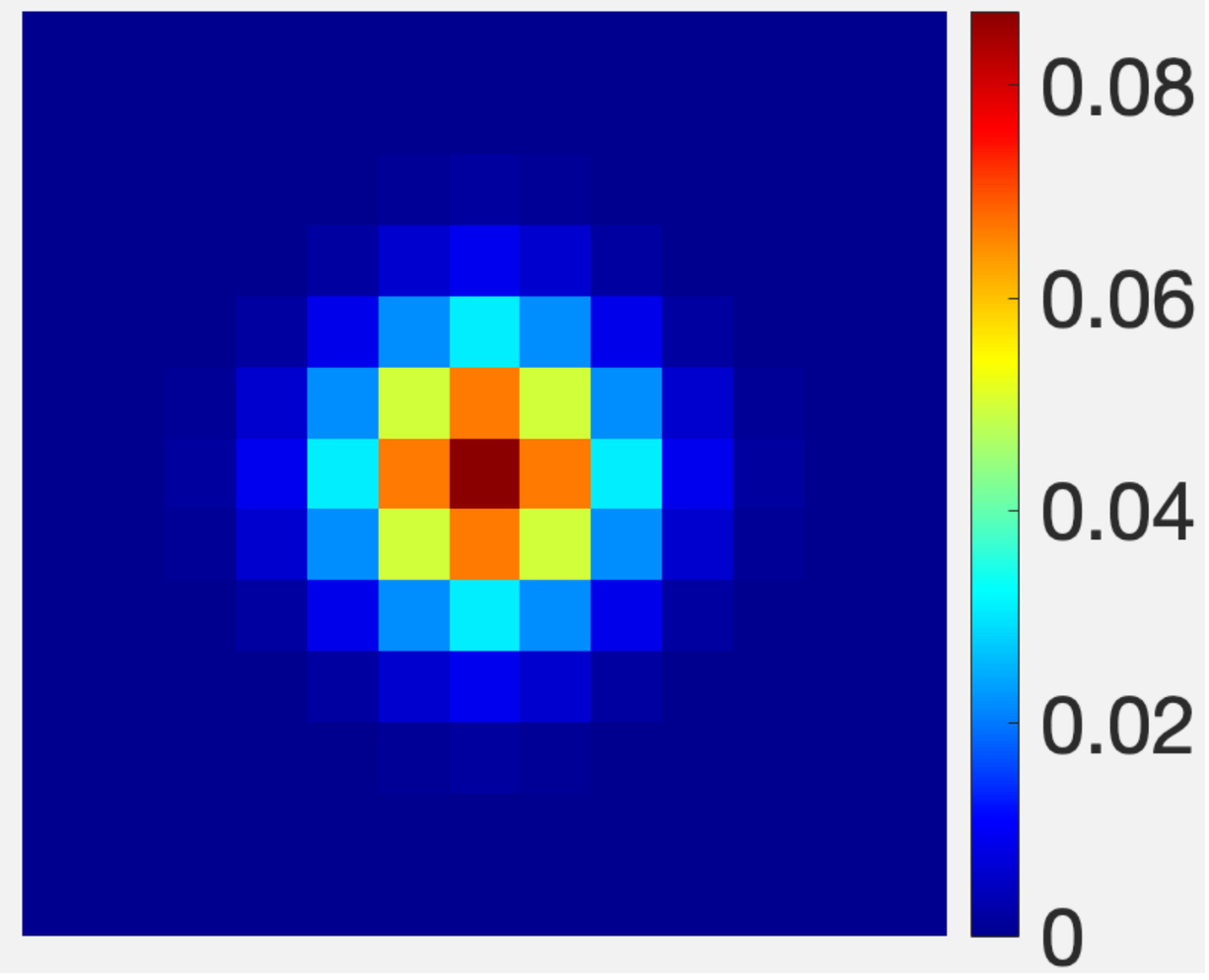
$$\begin{bmatrix} 0 & \lambda & 0 \\ \lambda & 1-4\lambda & \lambda \\ 0 & \lambda & 0 \end{bmatrix}$$

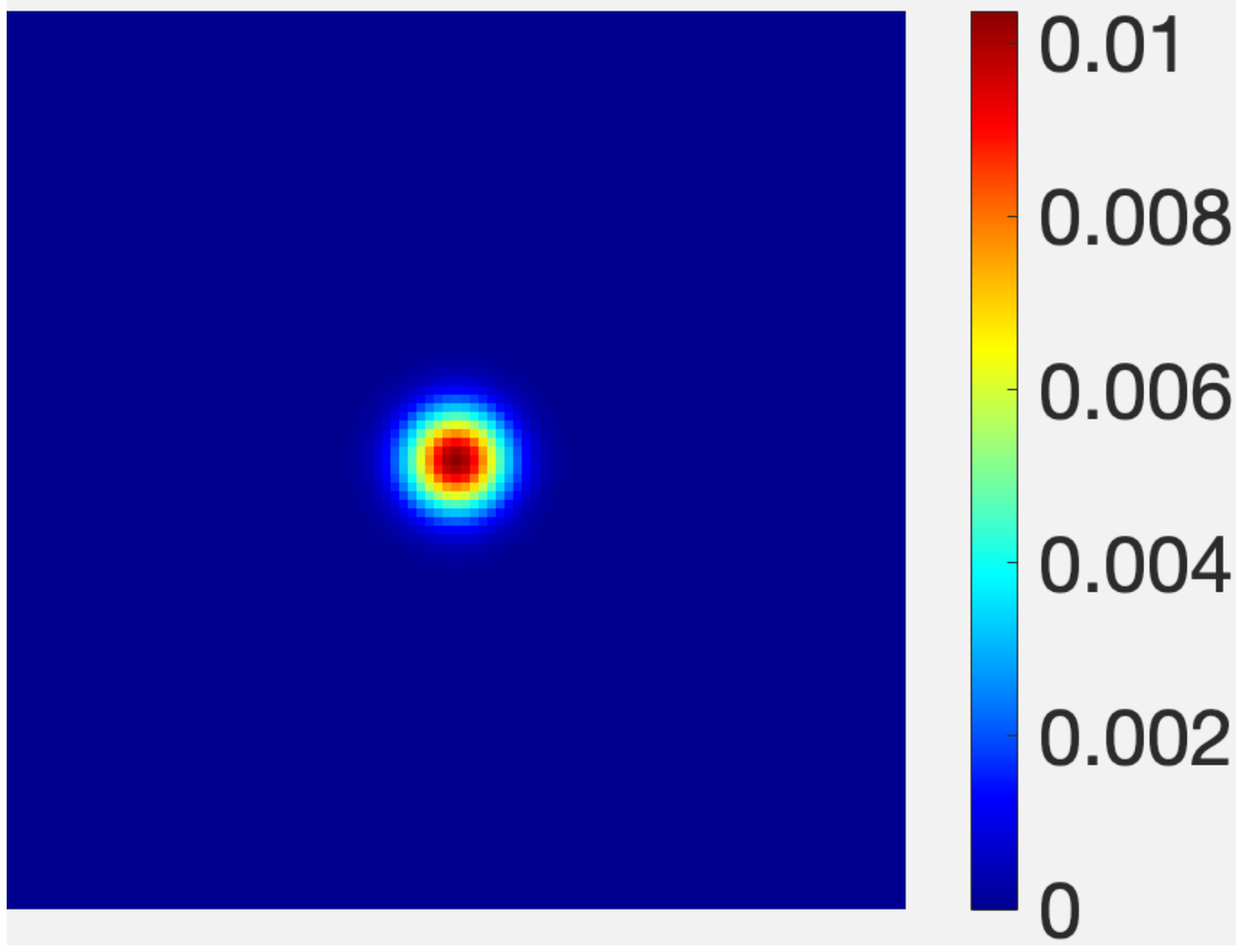
- For $0 < \lambda < 0.25$, all filter-matrix elements ≥ 0
- λ=0.2 or λ=0.25 give "mean" filters; $\lambda = 0.15$ →



Isotropic Diffusion: Heat Equation

- Repeated convolution makes effective kernel tend towards Gaussian
  - Central limit theorem (as long as initial kernel is non-negative; sums to 1)
- λ = 0.15. Left: conv repeated 5 times. Right: conv repeated 50 times.



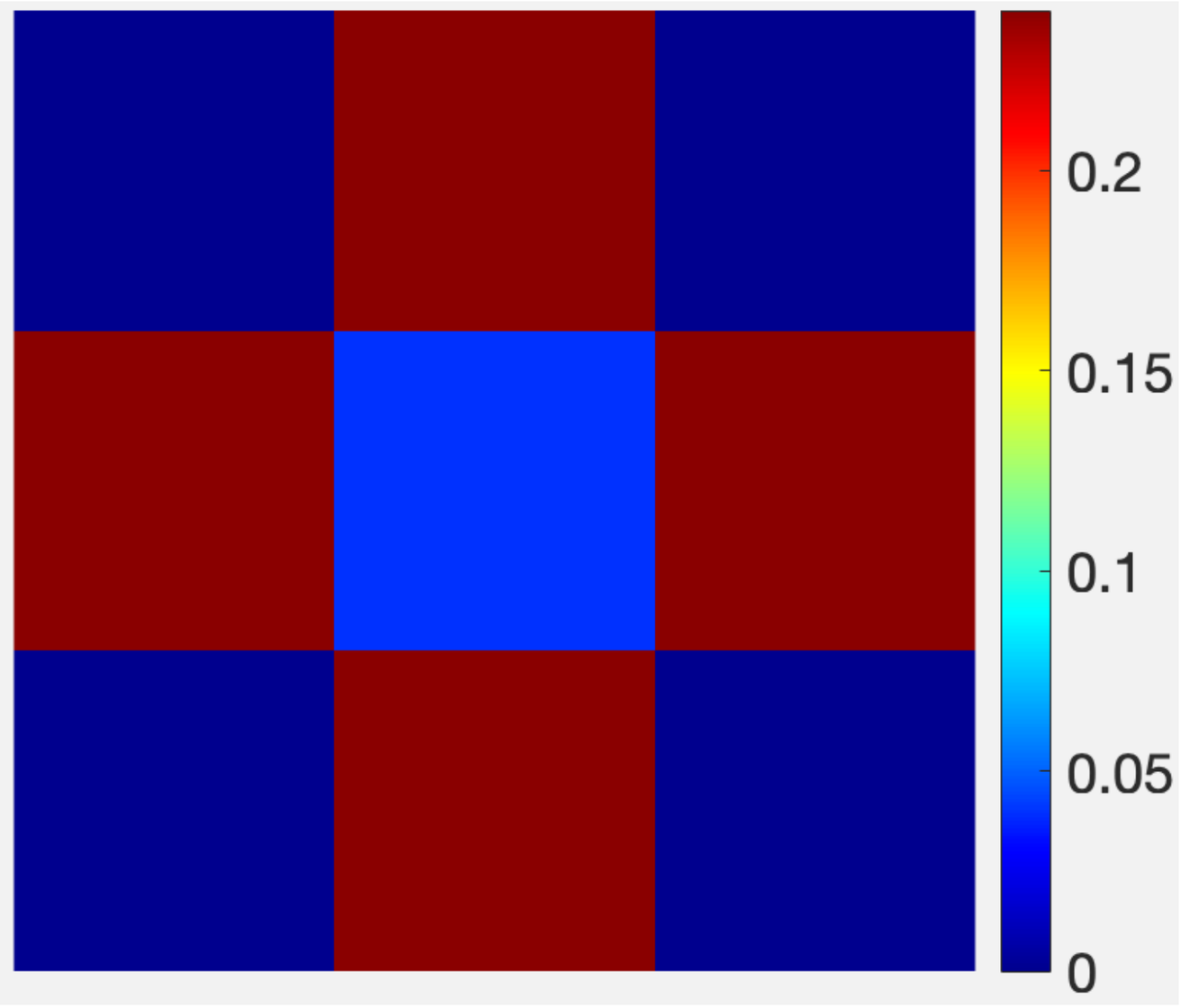


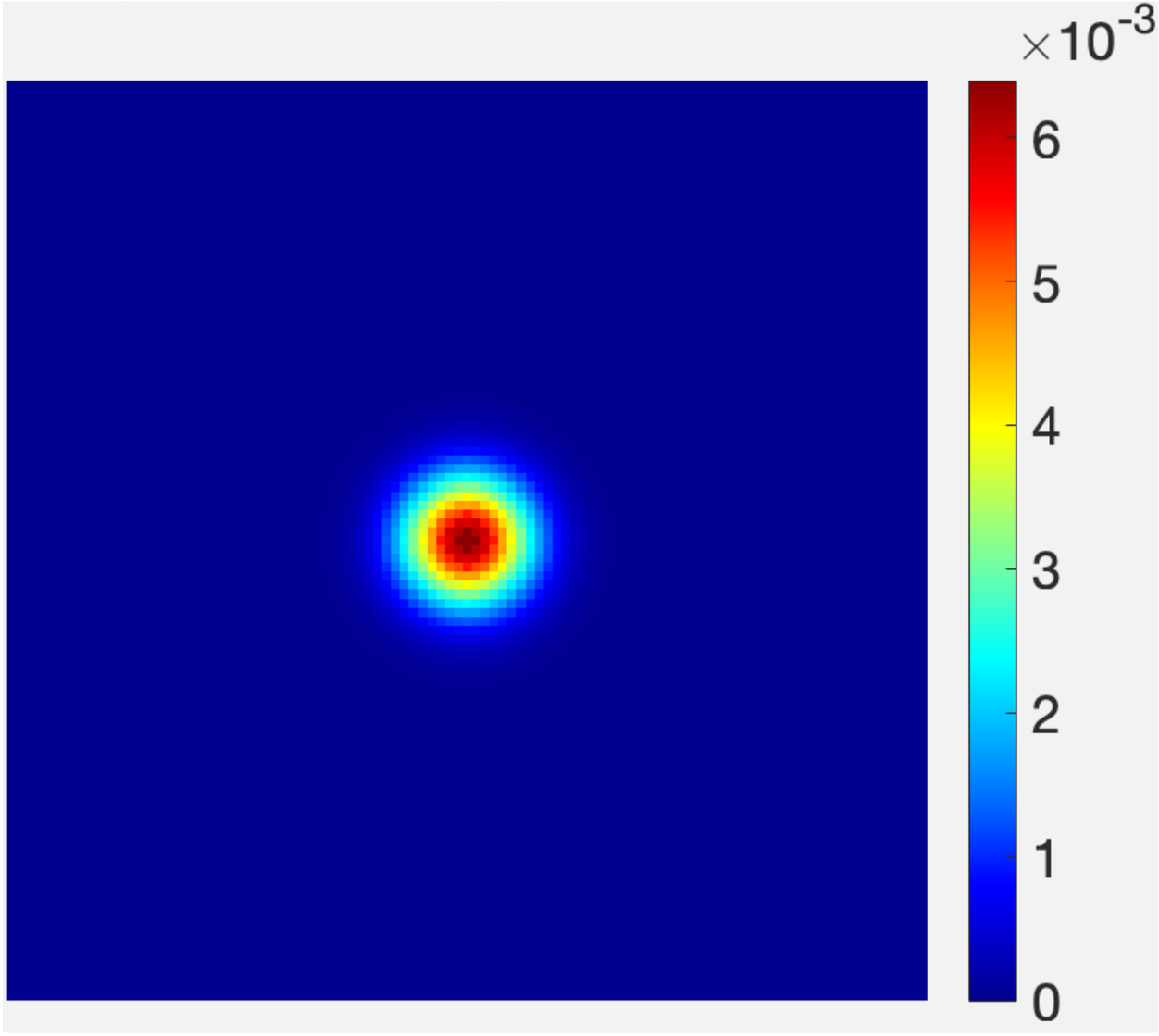
Isotropic Diffusion: Heat Equation

- $\lambda = 0.24$

  Left: Kernel.

Right: Conv repeated 50 times.

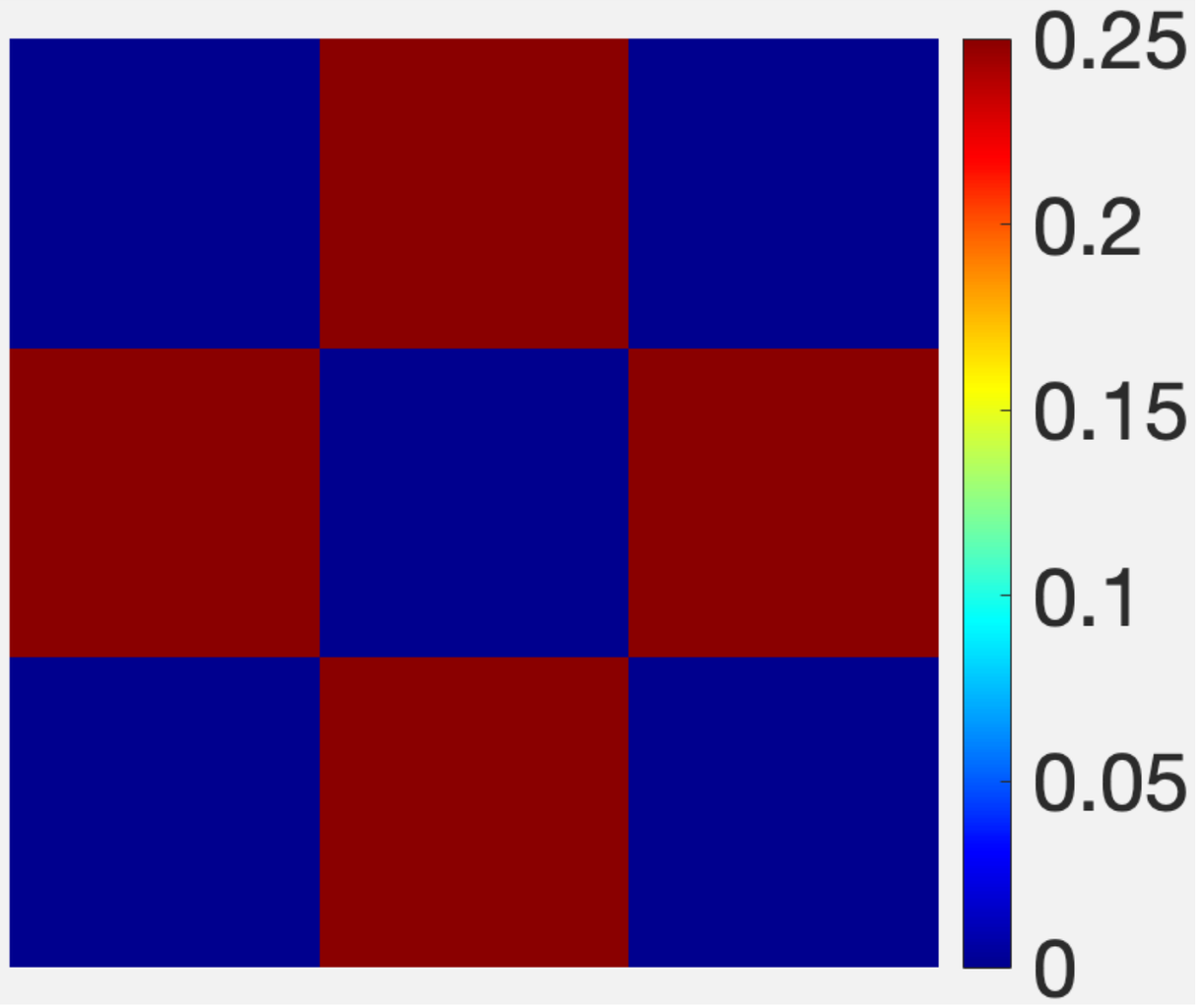


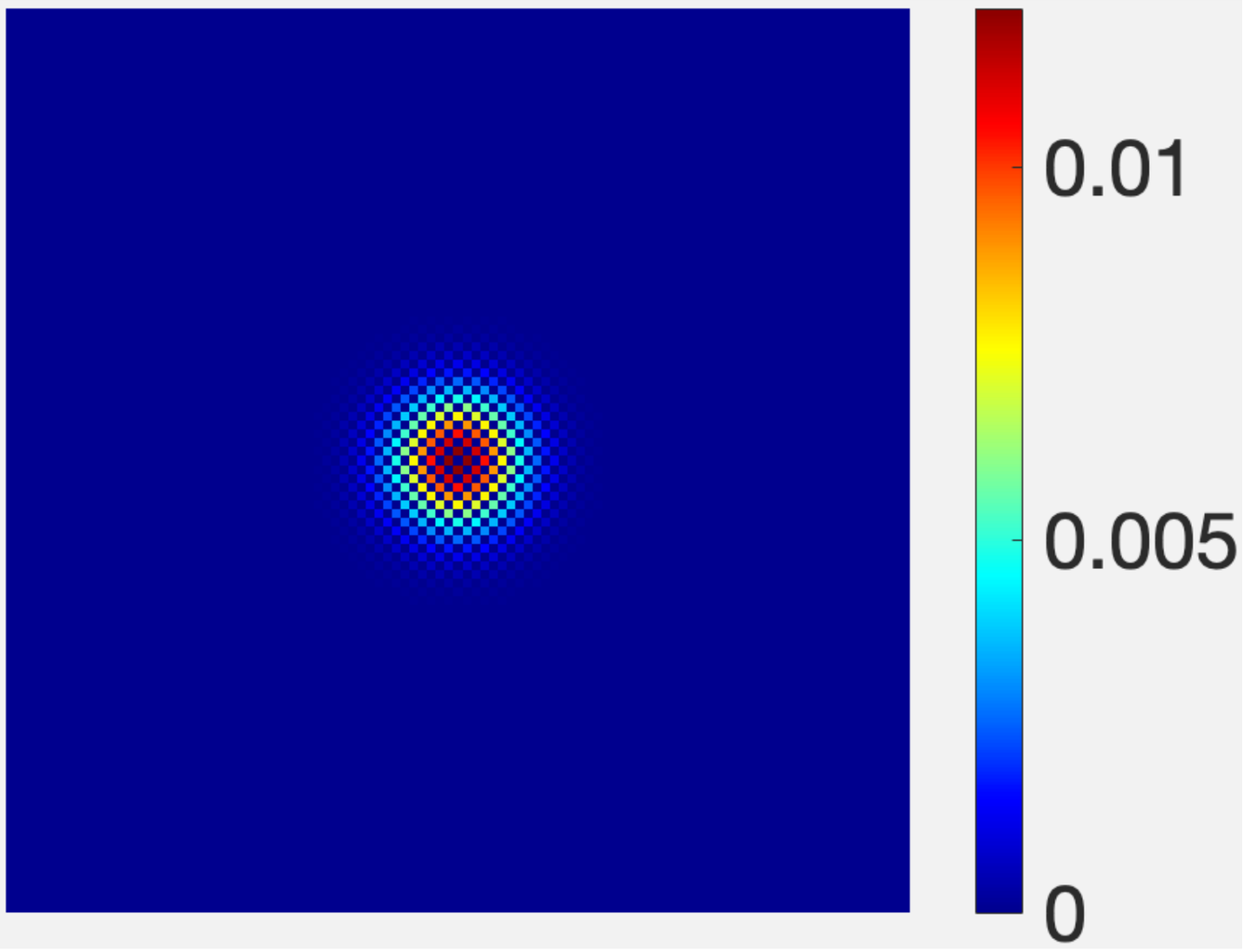


Isotropic Diffusion: Heat Equation

- $\lambda = 0.25$
- Left: Kernel.

Right: Conv repeated 50 times.

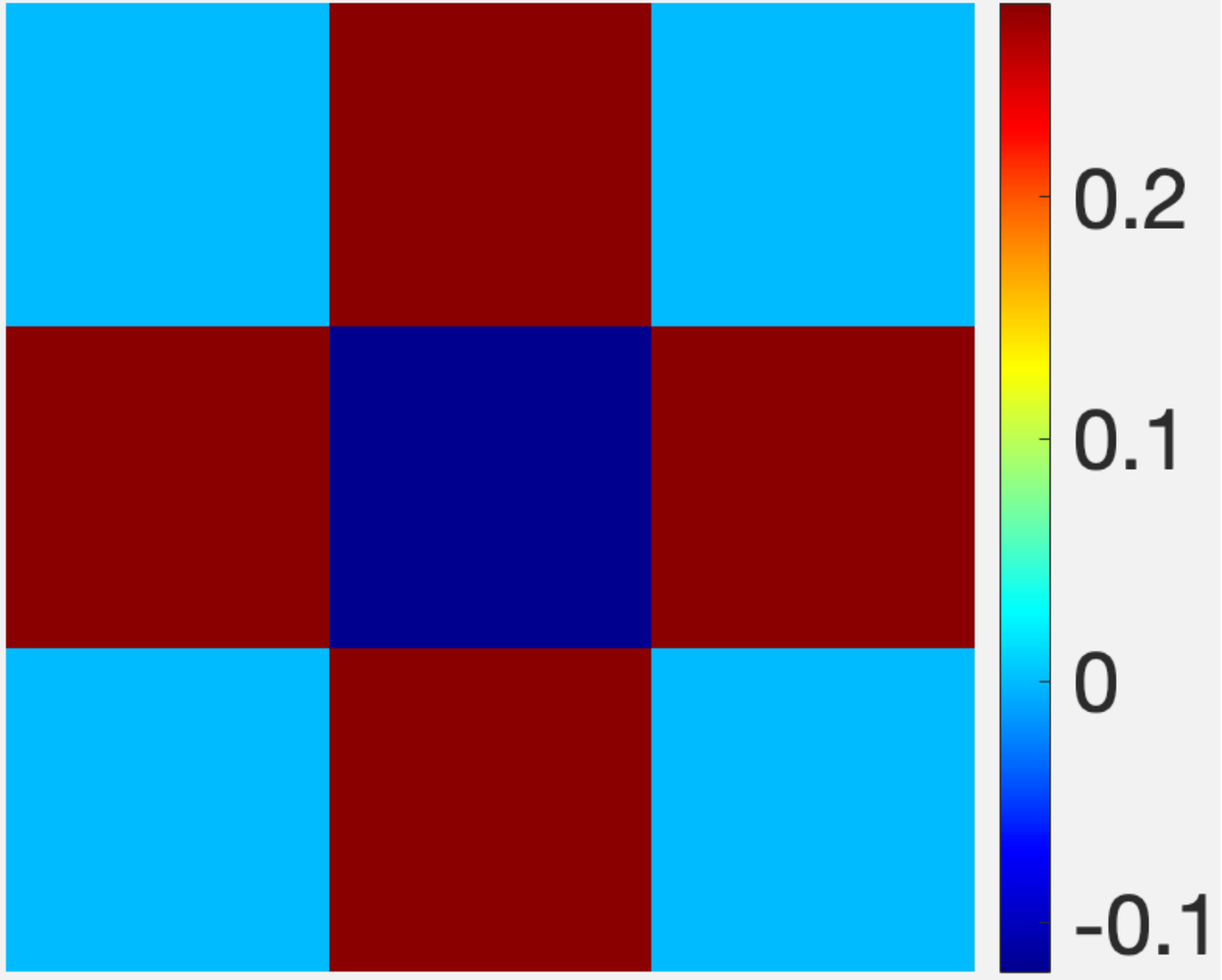


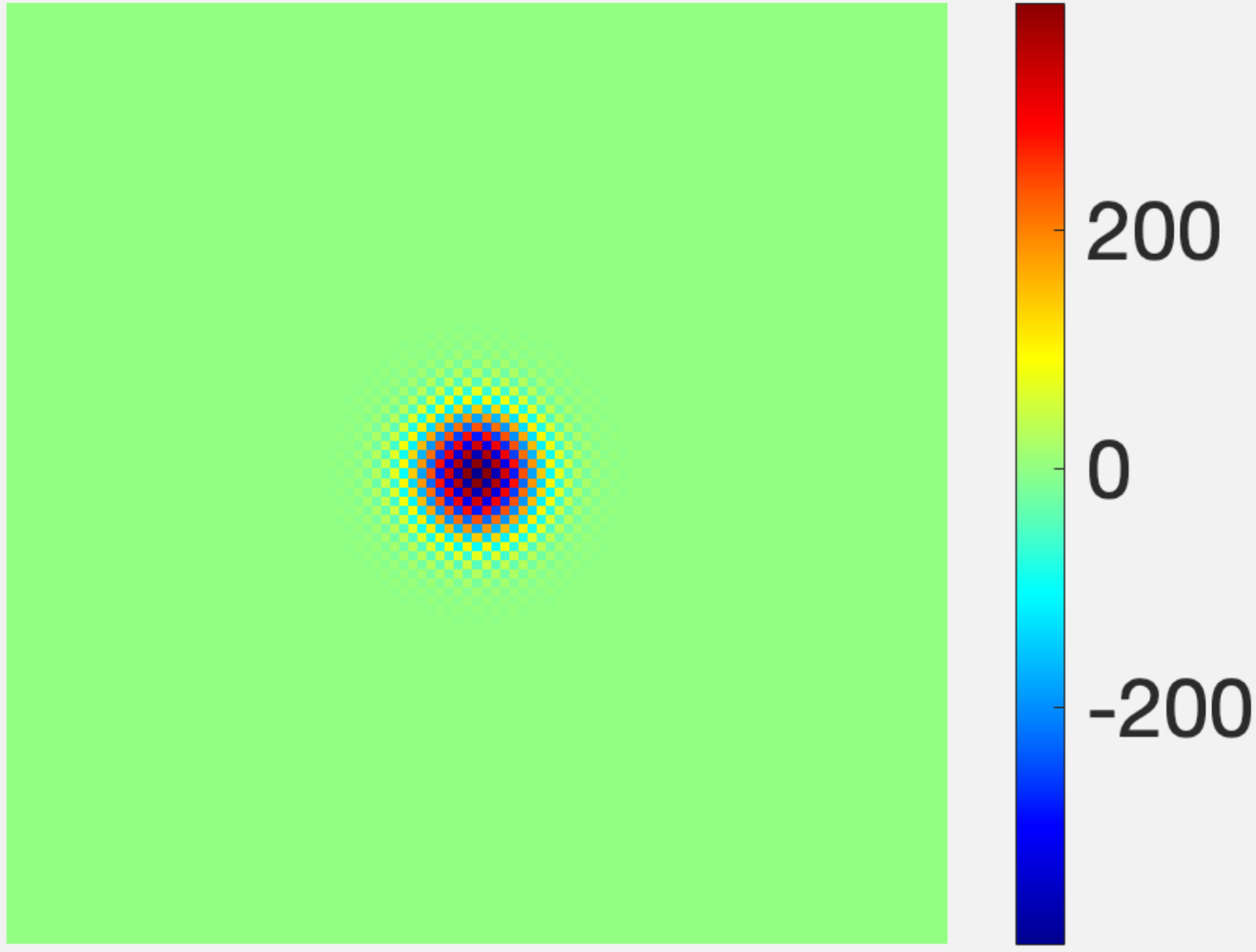


Isotropic Diffusion: Heat Equation

- $\lambda = 0.28$ (instability when $\lambda > 0.25$)
- Left: Kernel.

Right: Conv repeated 50 times.



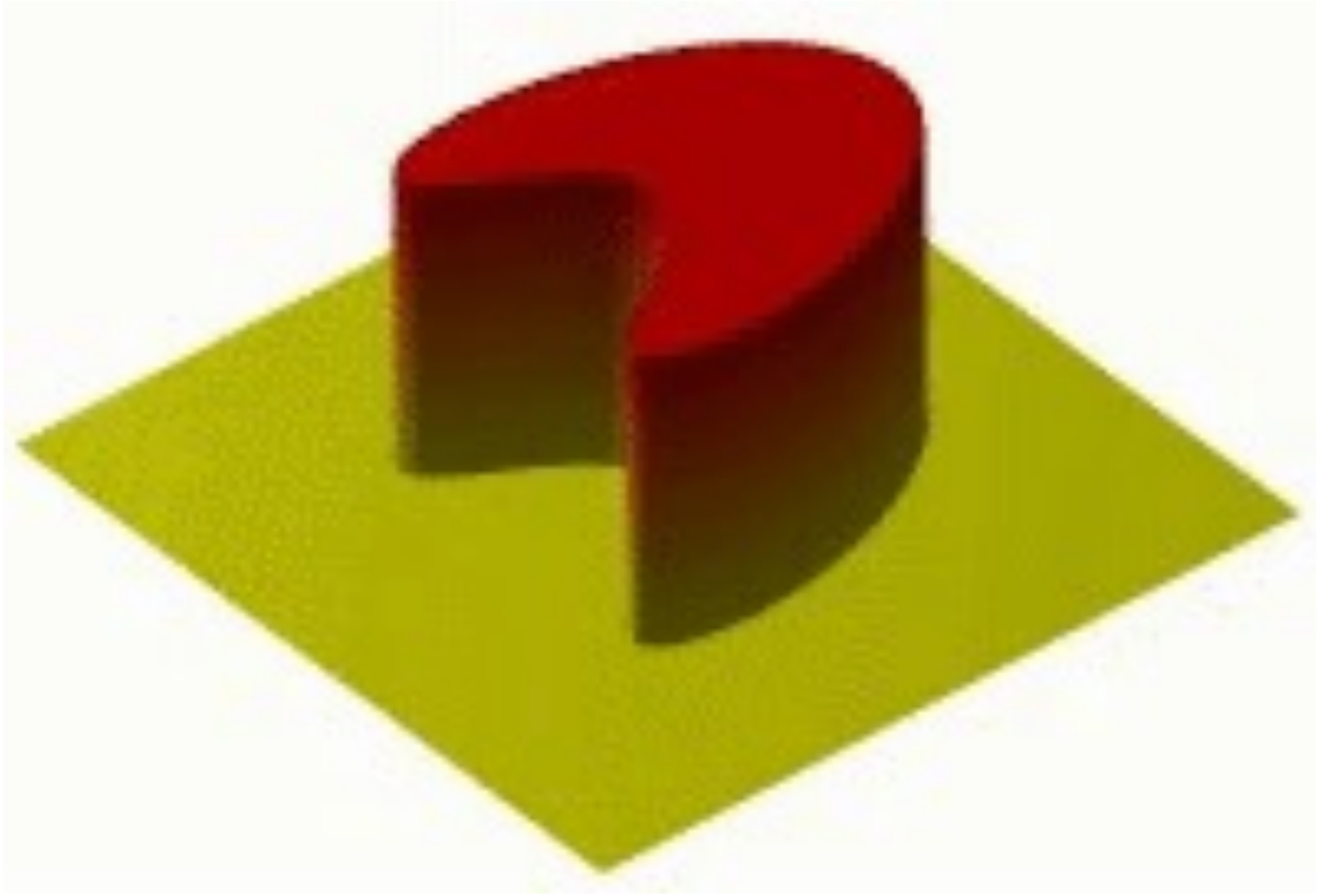


Numerical Stability/Instability

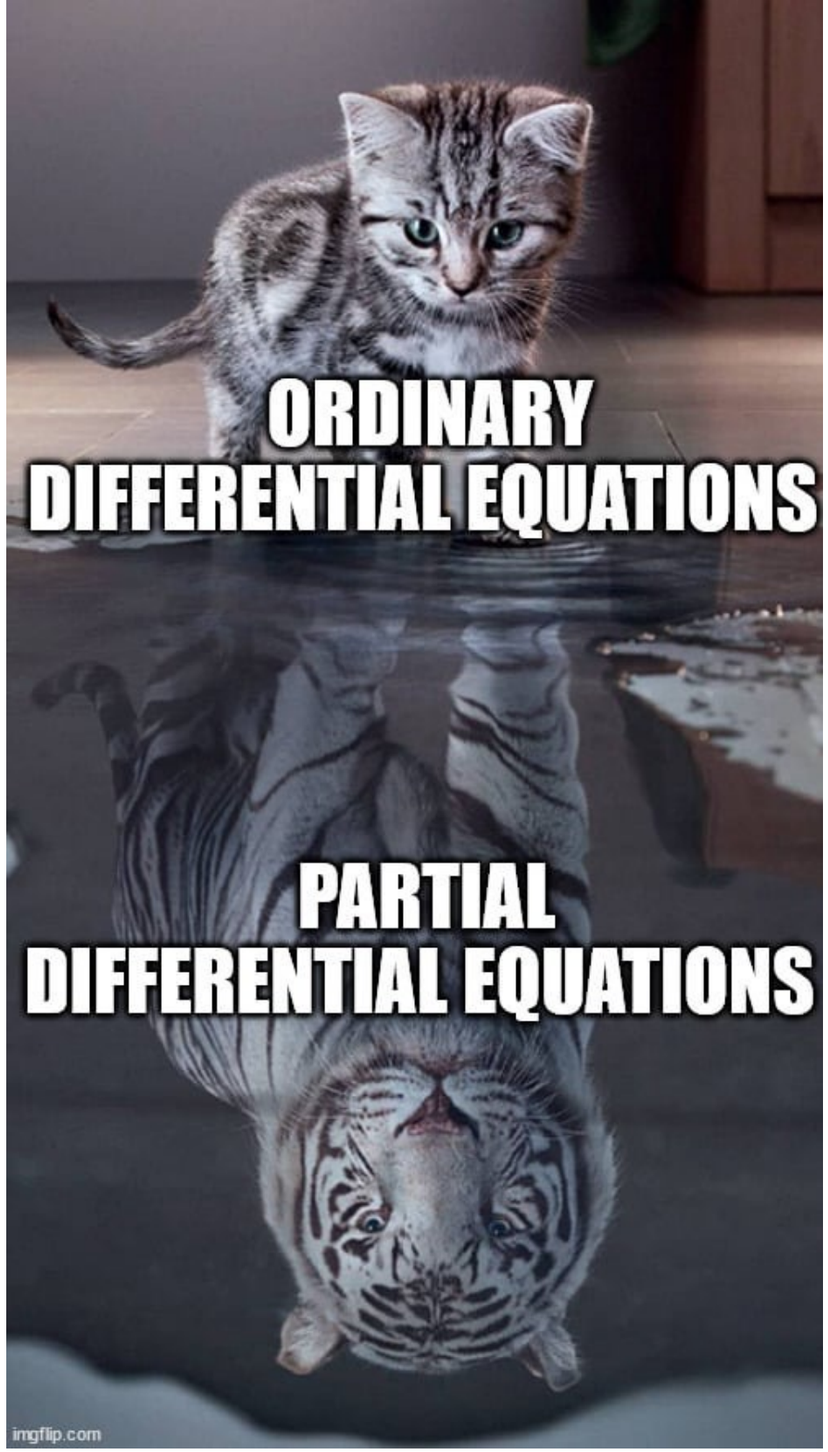
- Root cause of numerical instability:
  approximations because of finite-precision arithmetic
- Context = numerical algorithms for differential equations:
  e.g., instability caused by growing round-off errors,
  e.g., instability caused by small fluctuations in initial data which might cause a large deviation of final answer from exact solution
- Context = numerical linear algebra:
  Instabilities caused by proximity to singularities of various kinds,
  e.g., computing matrix inverse for a nearly singular matrix
- Numerically stable algorithms don't magnify such errors
- en.wikipedia.org/wiki/Numerical_stability

Isotropic Diffusion: Heat Equation

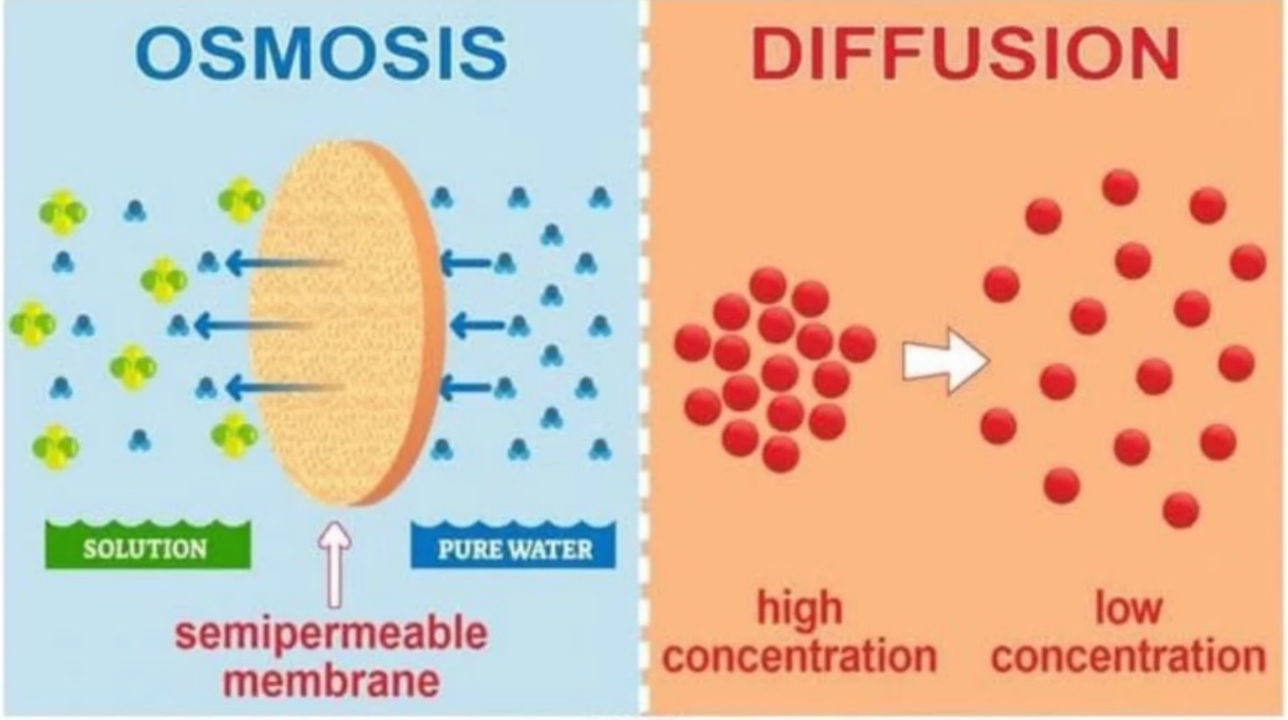
- Discontinuities in initial distribution instantly smoothed out

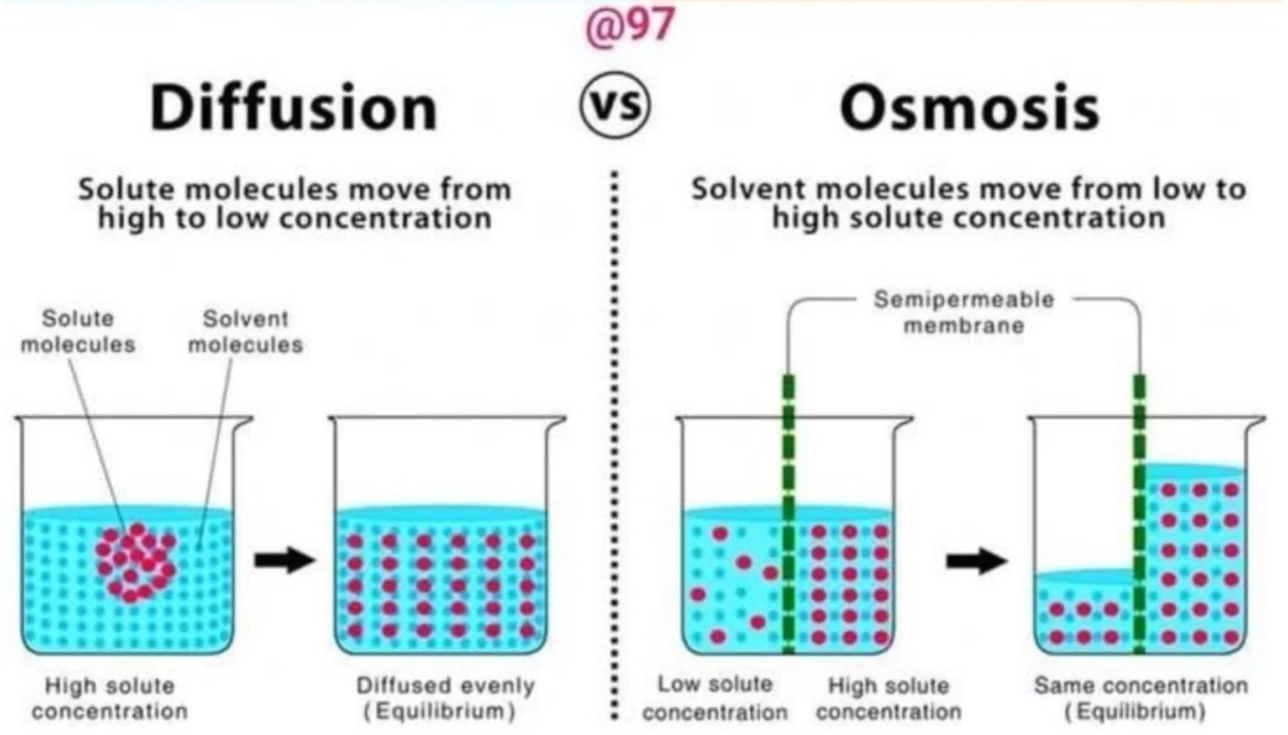


Differential Equations

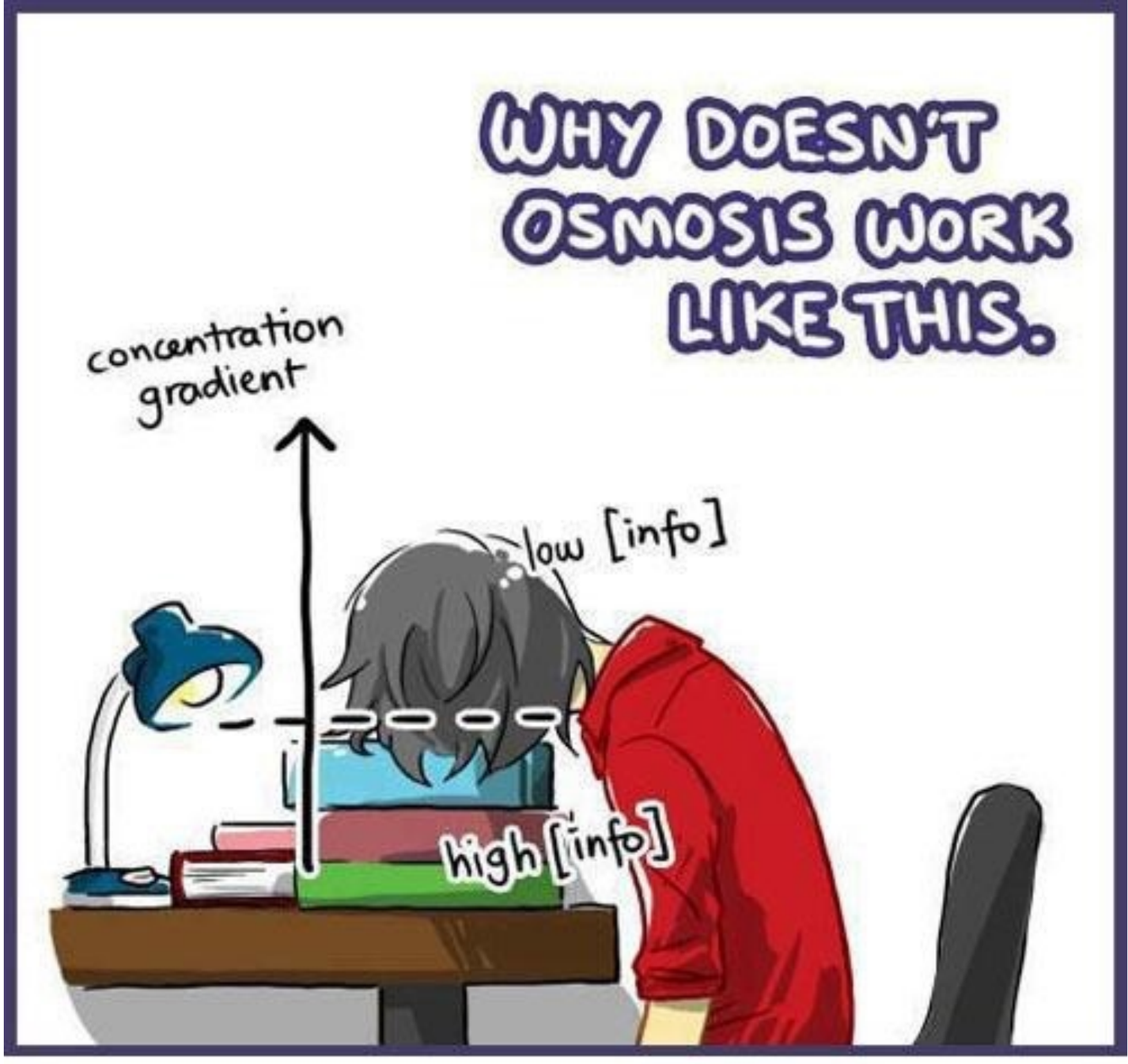


Diffusion vs. Osmosis





Knowledge Osmosis



Anisotropic Diffusion

- 1990

IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 12, NO. 7, JULY 1990

Scale-Space and Edge Detection Using Anisotropic Diffusion

PIETRO PERONA AND JITENDRA MALIK

Scale-space and edge detection using anisotropic diffusion 17733

P Perona, J Malik

IEEE Transactions on pattern analysis and machine intelligence 12 (7), 629-639, 2002

Multivariate Calculus

- Course details of MA 105 – Calculus
  - Taylor series
  - Jacobian
  - Directional derivative
  - Gradient (∇)
  - Divergence ($\nabla\bullet$)
  - Curl ($\nabla\times$)
  - Vector fields

Text Reference	1. Hughes-Hallett et al., Calculus - Single and Multivariable (3rd Edition), John-Wiley and Sons (2003).2. James Stewart, Calculus (5th Edition), Thomson (2003).3. T. M. Apostol, Calculus, Volumes 1 and 2 (2nd Edition), Wiley Eastern 1980.4. G. B. Thomas and R. L. Finney, Calculus and Analytic Geometry (9th Edition), ISE Reprint, Addison-Wesley, 1998.
Description	Review of limits, continuity, differentiability. Mean value theorem, Taylors Theorem, Maxima and Minima. Riemann integrals, Fundamental theorem of Calculus, Improper integrals, applications to area, volume. Convergence of sequences and series, power series. Partial Derivatives, gradient and directional derivatives, chain rule, maxima and minima, Lagrange multipliers. Double and Triple integration, Jacobians and change of variables formula. Parametrization of curves and surfaces, vector Fields, line and surface integrals. Divergence and curl, Theorems of Green, Gauss, and Stokes.
Last Update	2010-03-19 15:26:39.299444

Multivariate Calculus

The *directional derivative* of a scalar function

$$f(\mathbf{x}) = f(x_1, x_2, \ldots, x_n)$$

along a vector

en.wikipedia.org/wiki/Directional_derivative

$$\mathbf{v} = (v_1, \ldots, v_n)$$

is the function $\nabla_{\mathbf{v}} f$ defined by the limit[1]

$$\nabla_{\mathbf{v}} f(\mathbf{x}) = \lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{v}) - f(\mathbf{x})}{h} = \frac{\mathrm{d}}{\mathrm{d}t} f(\mathbf{x} + t\mathbf{v})\Big|_{t=0}$$

Multivariate Calc

- Directional Derivative is a scalar
- www.geogebra.org/m/Bx8nFMNc

Function

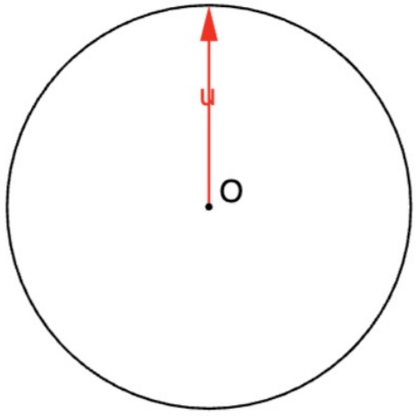
$f(x, y) =$ $x^2 + y^2$

Point $P = (1, 1)$

X = 1

Y = 1

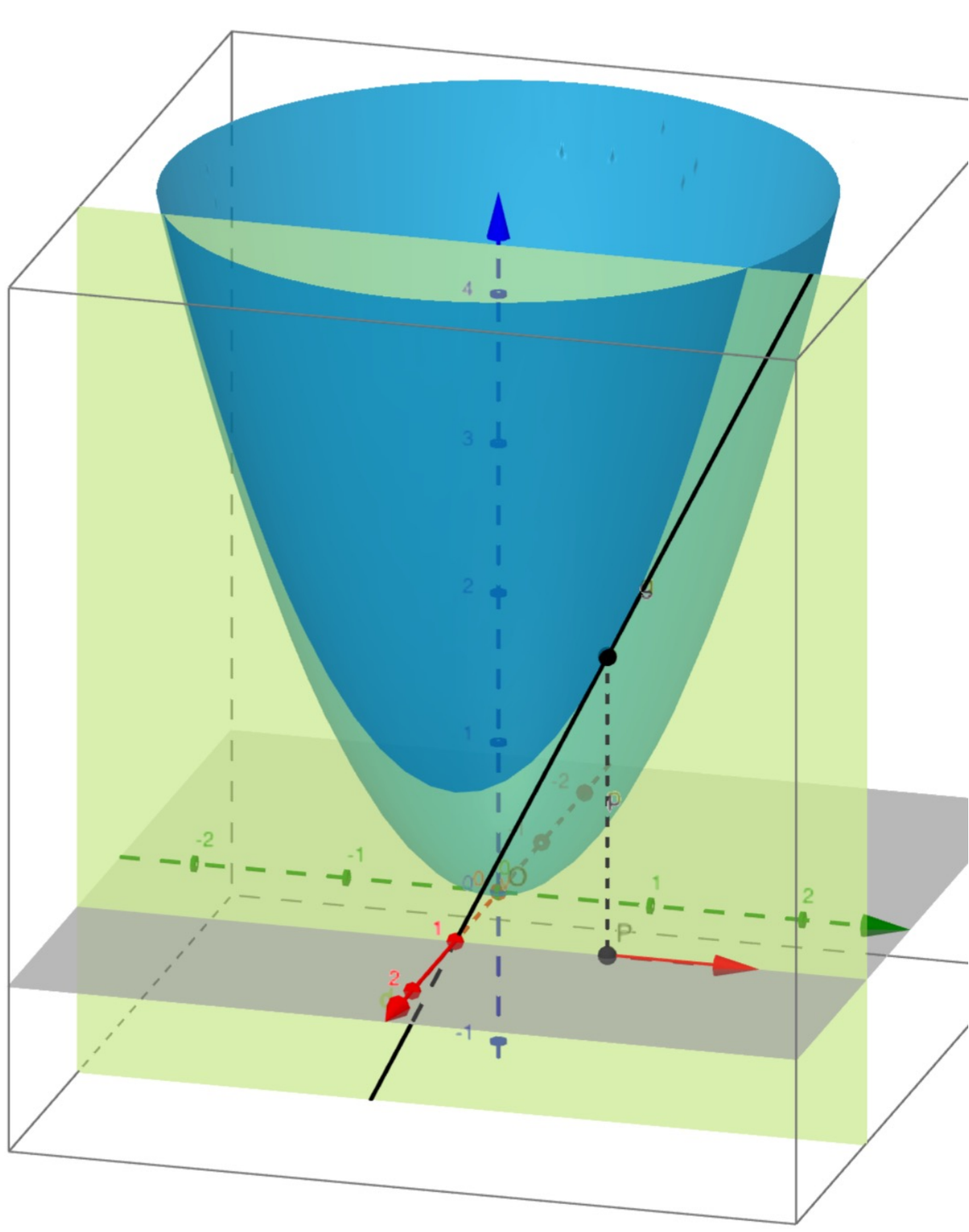
Vector $u = (0, 1)$



Angle = 90

Directional derivative

$f'_u(1, 1) = 2$



Multivariate Calc

- Directional Derivative is a scalar
- www.geogebra.org/m/Bx8nFMNc

Function

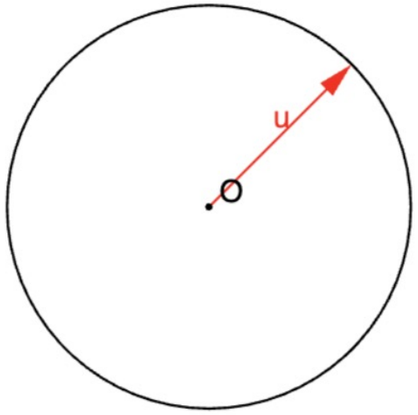
$f(x, y) =$ $x^2 + y^2$

Point $P = (1, 1)$

X = 1

Y = 1

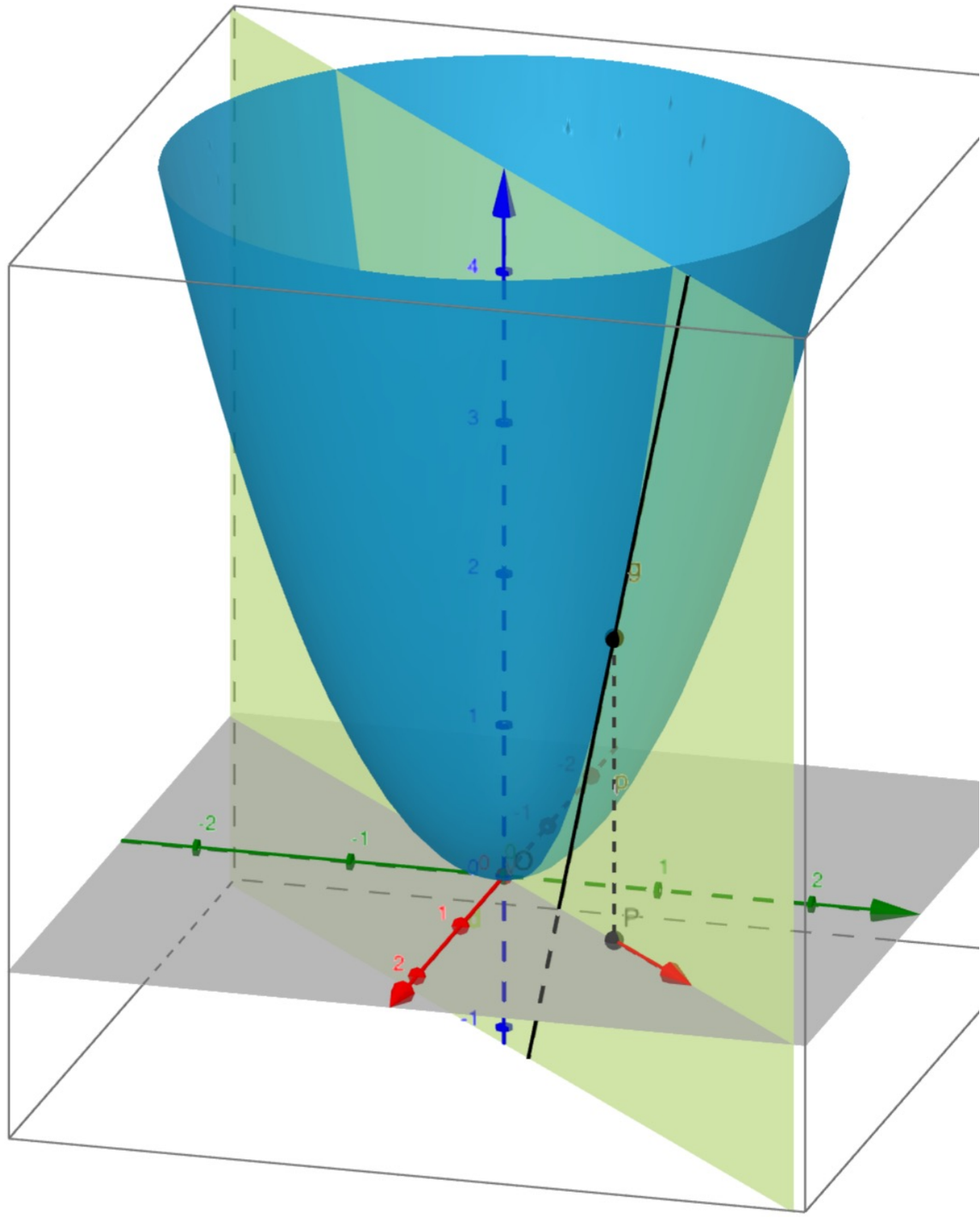
Vector $u = (0.71, 0.71)$



Angle = 45

Directional derivative

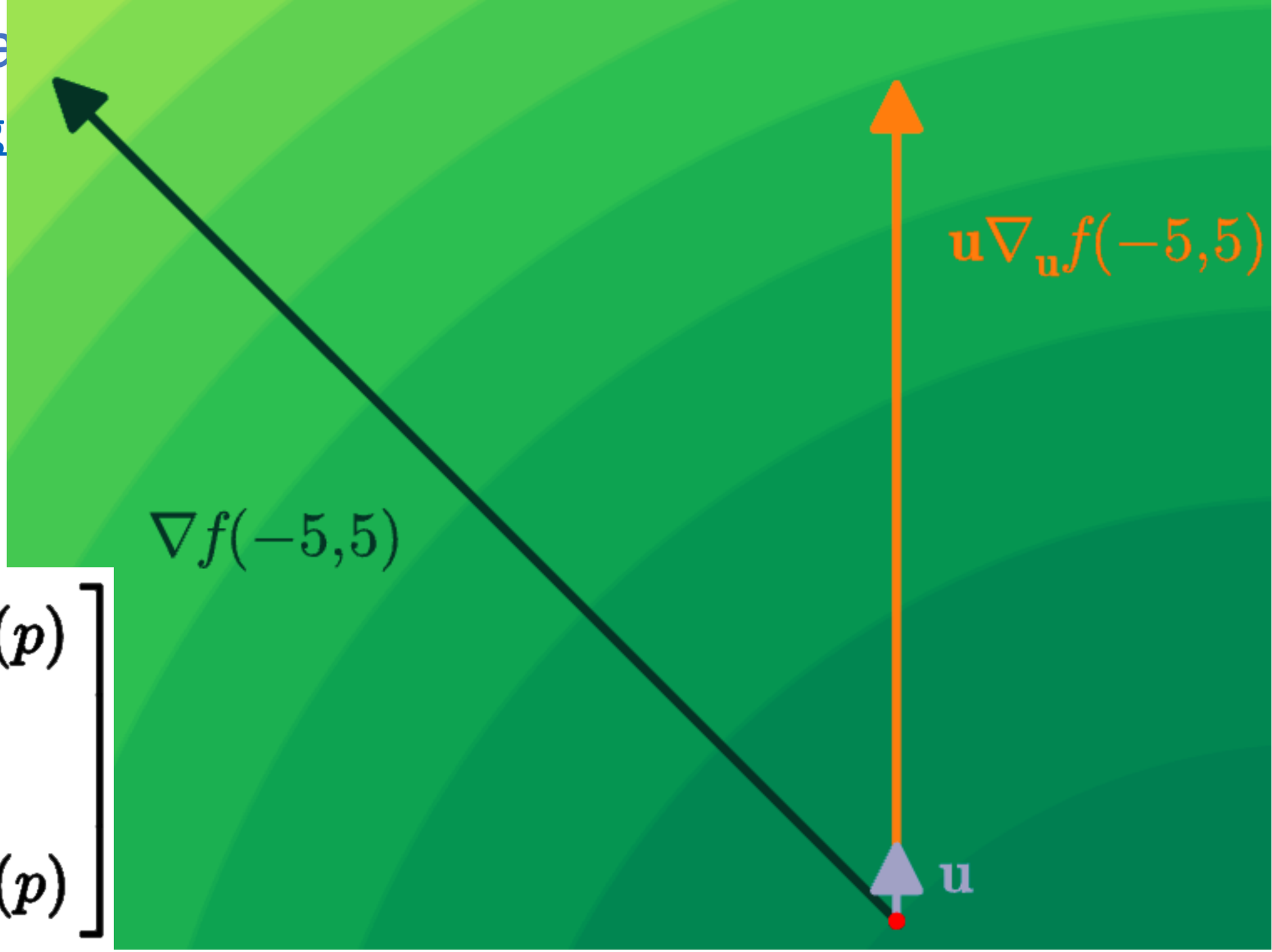
$f'_u(1, 1) = 2.83$



Multivariate Ca

- en.wikipedia.org/wiki/Gradient
- Function f(.) of n variables: $R^n \rightarrow R$
- Point p = $(x_1, \ldots, x_n)$

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$



Multivariate Calculus

- Relationship between gradient and directional derivative

- If gradient of a function is non-zero at a point p, then:

1) At point p, gradient direction
   = direction in which directional derivative has largest value
2) At point p, gradient magnitude
   = magnitude of directional derivative in aforementioned direction

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^{\mathsf{T}} \Delta\mathbf{x} + \frac{1}{2} \Delta\mathbf{x}^{\mathsf{T}} \mathbf{H}(\mathbf{x}) \Delta\mathbf{x}$$

Multivariate Calculus

- Relationship between gradient and directional derivative

$$\nabla_{\mathbf{v}} f(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \mathbf{v}$$

- Because:

$$\begin{aligned}
\nabla_{\mathbf{v}} f(\mathbf{x}) - \nabla f(\mathbf{x}) \cdot \mathbf{v} &= \lim_{t \to 0} \frac{f(\mathbf{x} + t\mathbf{v}) - f(\mathbf{x})}{t} - \nabla f(\mathbf{x}) \cdot \mathbf{v} \\
&= \lim_{t \to 0} \frac{f(\mathbf{x} + t\mathbf{v}) - f(\mathbf{x}) - t\nabla f(\mathbf{x}) \cdot \mathbf{v}}{t} \\
&= 0,
\end{aligned}$$

Vector Calculus

- en.wikipedia.org/wiki/Vector_calculus_identities

- Gradient operator $\nabla = \left(\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n} \right)$

- Gradient operator applied to scalar-valued function f(.) of 3 variables gives vector

$$\operatorname{grad}(f) = \nabla f = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right) f = \frac{\partial f}{\partial x}\mathbf{i} + \frac{\partial f}{\partial y}\mathbf{j} + \frac{\partial f}{\partial z}\mathbf{k}$$

- Divergence operator $\nabla \cdot$

- Divergence operator applied to vector-valued function $\mathbf{F} := (F_x, F_y, F_z)$ gives scalar

$$\operatorname{div} \mathbf{F} = \nabla \cdot \mathbf{F} = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right) \cdot (F_x, F_y, F_z)$$

$$= \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} + \frac{\partial F_z}{\partial z}.$$

Vector Calculus

- Divergence $\nabla \cdot$ of gradient ∇ of scalar-valued function f(.)
- = Divergence $\nabla \cdot$ of vector-valued function ∇f
- = Laplacian of f(.)

$$\nabla = \left(\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n} \right)$$

$$\Delta f = \sum_{i=1}^{n} \frac{\partial^2 f}{\partial x_i^2}$$

- Another way to look at it:
  What is composition operator defined by divergence-of-gradient $\nabla \cdot \nabla$?
  - Input = scalar-valued function
  - Output = scalar-valued function

$$\Delta = \nabla \cdot \nabla = \sum_{j=1}^{n} \frac{\partial^2}{\partial x_j^2}$$

- en.wikipedia.org/wiki/Laplace_operator

$$\Delta f = \nabla^2 f = \nabla \cdot \nabla f$$

Anisotropic Diffusion

- Let $I(x,t)$ be a spatiotemporal image (e.g., video)
- Let $I(x,t=0)$ be initial 2D image
- Let $I(x,t'>0)$ be 2D image formed by diffusion of initial image

- Consider differential equation: $\frac{\partial I}{\partial t} = \mathrm{div}(c(x,y,t)\nabla I)$

- Motivation: Limit diffusion/smoothing at pixels (x,y), where image gradient is large, by reducing coefficient c(x,y)
- Design c(x,y) to be inversely related to image-gradient magnitude
  - Proposed 2 options

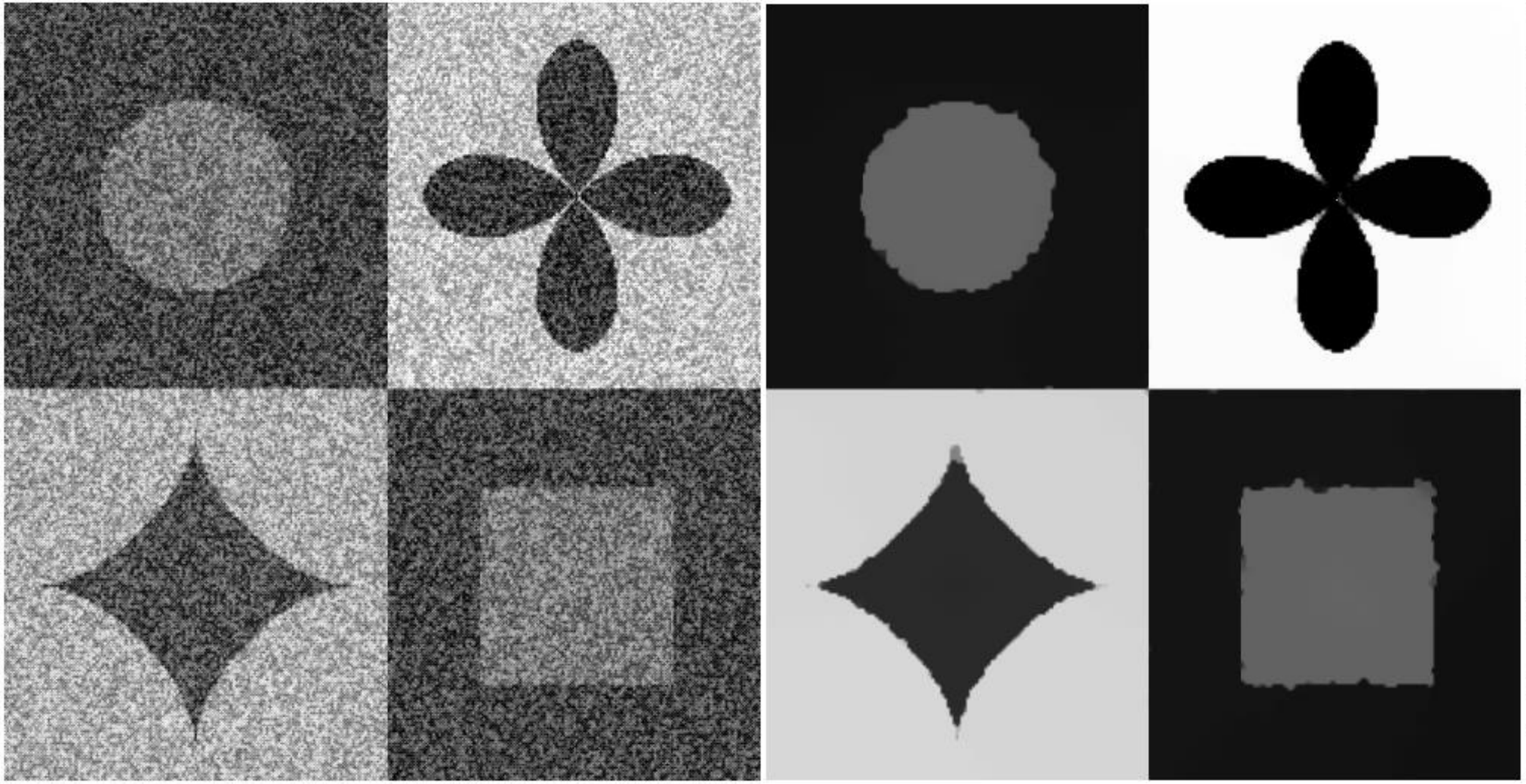
$$c(\|\nabla I\|) = e^{-(\|\nabla I\|/K)^2}$$

$$c(\|\nabla I\|) = \frac{1}{1+\left(\frac{\|\nabla I\|}{K}\right)^2}$$

Anisotropic Diffusion

- $\nabla_t I(x,y) = \nabla \cdot \big(c(x,y)\, \nabla I(x,y)\big)$ where c(x,y) > 0
- PDE-Solver View: Discretize differential equation using finite differences
- At each (x,y), P-M interpret c(x,y) as a (different) modulating constant c
- $I^{\text{new}}(x,y) = I(x,y) + \lambda\big[\nabla_x, \nabla_y\big] \cdot \big[c\, \nabla_{\text{x}}(x,y), c\, \nabla_{\text{y}}(x,y)\big]$
- $= I(x,y) + \lambda\, c\, \nabla_{\text{x}}^2 I(x,y) + \lambda\, c\, \nabla_{\text{y}}^2 I(x,y)$
- $= I(x,y) + \lambda\, c\, [I(x+1,y) - 2I(x,y) + I(x-1,y)]$
  $+ \lambda\, c\, [I(x,y+1) - 2I(x,y) + I(x,y-1)]$
- $= I(x,y) + \lambda\, c_N\, \nabla I_N(x,y) + \lambda\, c_S\, \nabla I_S(x,y)$
  $+ \lambda\, c_E\, \nabla I_E(x,y) + \lambda\, c_W\, \nabla I_W(x,y)$
- s.t. $\nabla I_N(x,y) \coloneqq I(x+1,y) - I(x,y)$, $\nabla I_S(x,y) \coloneqq I(x-1,y) - I(x,y)$, …
- s.t. $c_N(x,y) \coloneqq g(\nabla I_N(x,y))$, $c_S(x,y) \coloneqq g(\nabla I_S(x,y))$, …

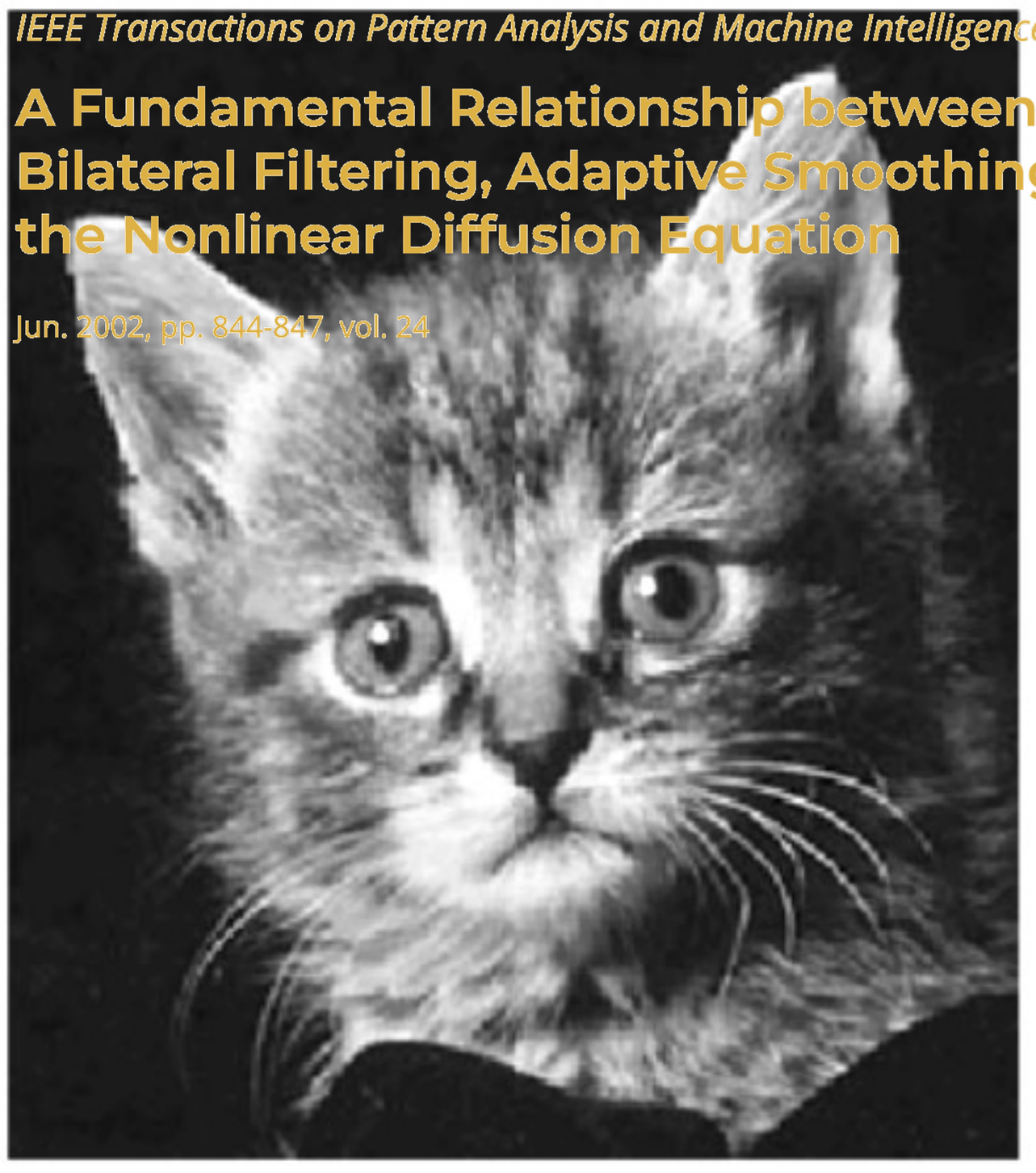
Anisotropic Diffusion



IEEE Transactions on Pattern Analysis and Machine Intelligence

A Fundamental Relationship between Bilateral Filtering, Adaptive Smoothing, and the Nonlinear Diffusion Equation

Jun. 2002, pp. 844-847, vol. 24

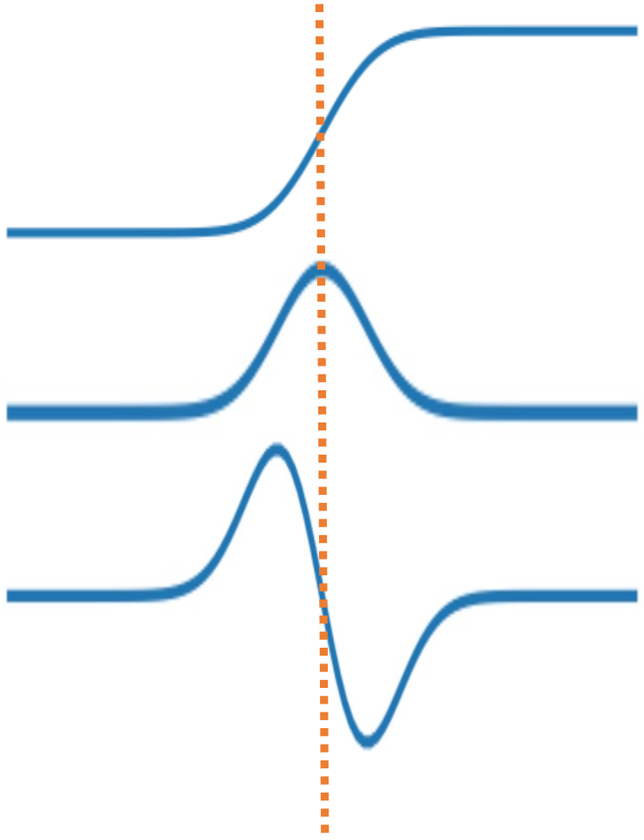


Anisotropic Diffusion

$$\frac{\partial I}{\partial t} = \operatorname{div}(c(x,y,t)\nabla I) = \nabla c \cdot \nabla I + c(x,y,t)\,\Delta I$$

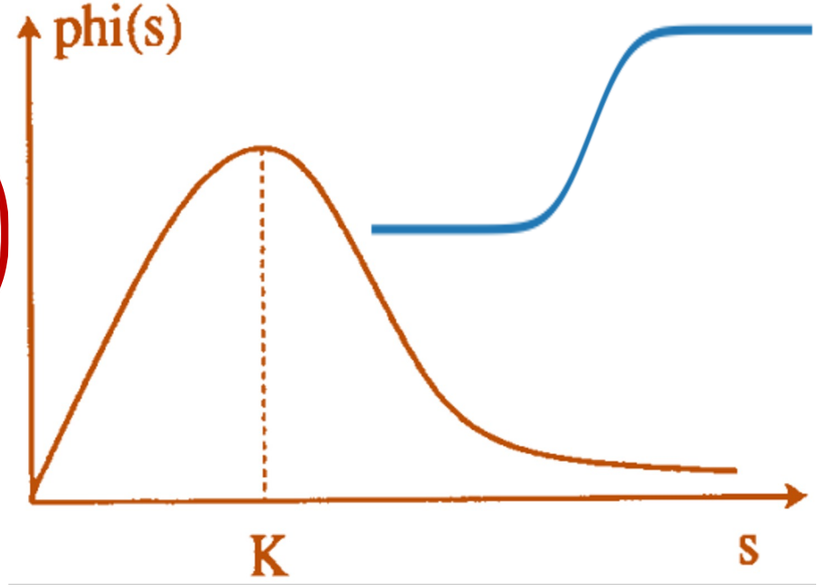
- **1D Analysis:** during diffusion process, image I(x,t) is a function of x & t
- We designed diffusion coefficient c(x) as function of image gradient $\frac{\partial I}{\partial x}$
- Then, anisotropic diffusion becomes $\frac{\partial I}{\partial t} = \frac{\partial}{\partial x}\left(c\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial I}{\partial x}\right)$
- Defining $\phi\left(\frac{\partial I}{\partial x}\right) := c\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial I}{\partial x}$ gives $\frac{\partial I}{\partial t} = \frac{\partial}{\partial x}\left(\phi\left(\frac{\partial I}{\partial x}\right)\right) = \phi'\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial^2 I}{\partial x^2}$
- Now, we analyze how an edge's slope changes over time, i.e., $\frac{\partial}{\partial t}\left(\frac{\partial I}{\partial x}\right)$
- $= \frac{\partial}{\partial x}\left(\frac{\partial I}{\partial t}\right) = \frac{\partial}{\partial x}\left(\phi'\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial^2 I}{\partial x^2}\right)$
- $= \phi'\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial^3 I}{\partial x^3} + \phi''\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial^2 I}{\partial x^2} \cdot \frac{\partial^2 I}{\partial x^2}$

Anisotropic Diffusion



- We have $\frac{\partial}{\partial t}\left(\frac{\partial I}{\partial x}\right) = \phi'\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial^3 I}{\partial x^3} + \phi''\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial^2 I}{\partial x^2} \cdot \frac{\partial^2 I}{\partial x^2}$
- Consider an example edge with slope $\frac{\partial I}{\partial x} > 0$
- At **point of inflection**, $\frac{\partial^2 I}{\partial x^2} = 0$ and $\frac{\partial^3 I}{\partial x^3} < 0$
- For points **x** in **neighborhood** of **point of inflection**,
  LHS has sign opposite to that of $\boldsymbol{\phi'\left(\frac{\partial I}{\partial x}\right)}$
- So, if $\phi'\left(\frac{\partial I}{\partial x}\right) > 0$, then slope $\frac{\partial I}{\partial x}$ (now > 0) will decrease with time ∂t
- And, if $\phi'\left(\frac{\partial I}{\partial x}\right) < 0$, then slope $\frac{\partial I}{\partial x}$ (now > 0) will **increase!** with time ∂t
- Recall $\phi\left(\frac{\partial I}{\partial x}\right) \coloneqq c\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial I}{\partial x}$

Anisotropic Diffusion



- Our choices of $c\left(\frac{\partial I}{\partial x}\right)$, e.g., $c(\cdot) = 1/\left(1 + \left(\frac{\cdot}{K}\right)^2\right)$ which decrease with magnitude of slope $\frac{\partial I}{\partial x}$, lead to $\phi\left(\frac{\partial I}{\partial x}\right) \coloneqq c\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial I}{\partial x}$ that increases until some slope magnitude (e.g., K in case of above c(.)) & then decreases
- So, for slope **magnitudes ≤ K** (e.g., resulting from noise), $\phi'\left(\frac{\partial I}{\partial x}\right) > 0$ leading to $\frac{\partial}{\partial t}\left(\frac{\partial I}{\partial x}\right) < 0$ → **smoothing**
- And, for slope **magnitudes ≥ K** (e.g., resulting from signal), $\phi'\left(\frac{\partial I}{\partial x}\right) < 0$ leading to $\frac{\partial}{\partial t}\left(\frac{\partial I}{\partial x}\right) > 0$ → **sharpening!**
- Also, recall $\frac{\partial I}{\partial t} = \phi'\left(\frac{\partial I}{\partial x}\right) \cdot \frac{\partial^2 I}{\partial x^2}$ → $\phi'\left(\frac{\partial I}{\partial x}\right) < 0$ leads to **reverse** heat eq.

Anisotropic Diffusion

- Can “reverse diffusion” (i.e., concentration) lead to instability ?
- In general, yes, but not in this particular differential equation
- Special case of a more general class of elliptic equations that satisfy a “maximum-minimum principle” that states:
  all maxima & minima of solution of equation in space & time belong to initial condition (= original image) & boundaries of domain of interest
  provided that conduction coefficient c(.) is positive
- Using “adiabatic boundary conditions” (i.e., derivative 0 across boundary), we guarantee that
  maxima only belong to original image

Article

A strong maximum principle for parabolic equations

Louis Nirenberg

First published: May 1953 |
https://doi.org/10.1002/cpa.3160060202 | Citations: 176

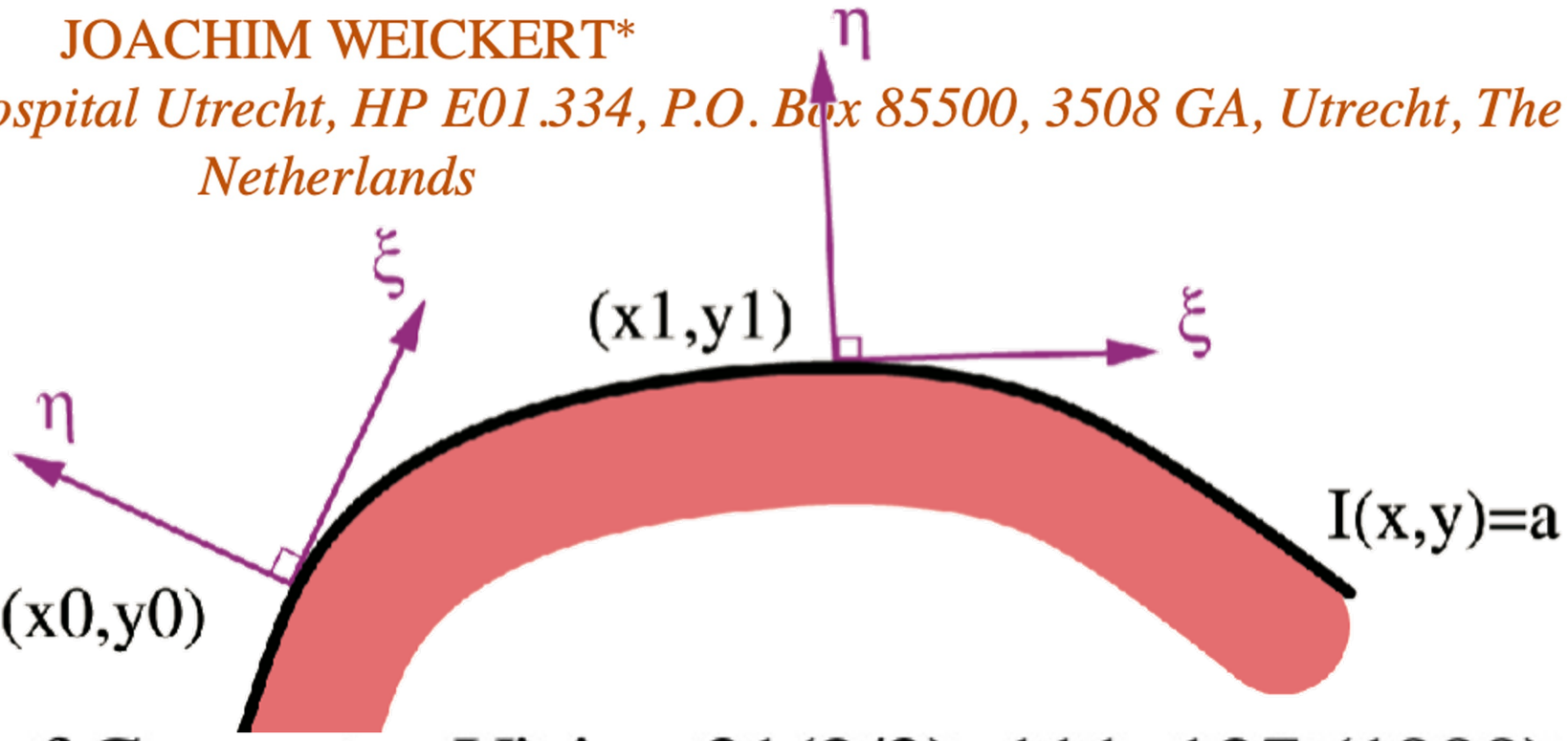
Anisotropic Diffusion with Diffusion Tensors

- 1999

Coherence-Enhancing Diffusion Filtering

JOACHIM WEICKERT*

Image Sciences Institute, University Hospital Utrecht, HP E01.334, P.O. Box 85500, 3508 GA, Utrecht, The Netherlands



International Journal of Computer Vision 31(2/3), 111–127 (1999)

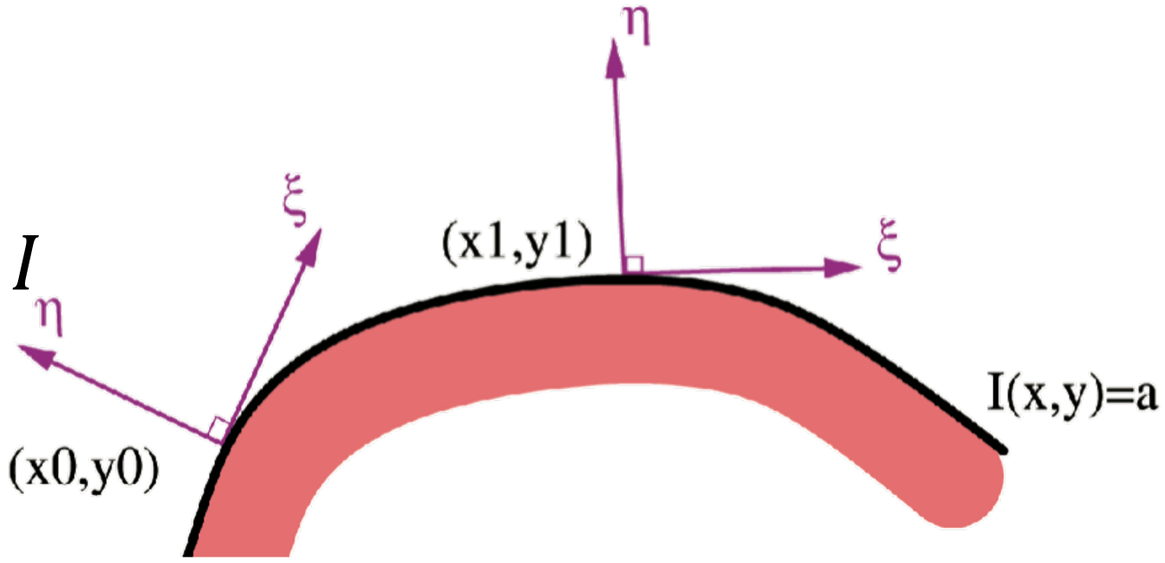
1999 Kluwer Academic Publishers. Manufactured in The Netherlands.

Anisotropic Diffusion with Diffusion Tensors

- We had: $I^{\text{new}}(x,y) = I(x,y) + \lambda\, \mathrm{c}\, \nabla^2_{\mathrm{x}} I(x,y) + \lambda\, c\, \nabla^2_{\mathrm{y}} I(x,y)$
- Equivalent to: $\dfrac{\partial I}{\partial t} = \operatorname{div}\left[\underbrace{(c \cdot \mathbf{I}_2)}_{D} \cdot \nabla I\right] = \operatorname{div}\left[\begin{pmatrix} c & 0 \\ 0 & c \end{pmatrix} \cdot \nabla I\right]$
- Rewrite in **rotated** coordinate **frame**:
  axes along image **iso-contour (tangent)**
  along image **gradient (normal)**
- We want: $I^{\text{new}}(x,y) = I(x,y) + \lambda\, \mathrm{c}\, \nabla^2_{\mathrm{t}} I(x,y) + \lambda\, c\, \nabla^2_{\mathrm{n}} I(x,y)$
- Replace c by 2 coefficients (more control) to modulate smoothing based on local edge direction (bit more data-driven than Perona-Malik)
- We want: $I^{\text{new}}(x,y) = I(x,y) + \lambda\, \mathrm{c}_t\, \nabla^2_t I(x,y) + \lambda\, c_n\, \nabla^2_{\mathrm{n}} I(x,y)$
  where $\mathrm{c}_t \gg \mathrm{c}_n \to 0$

Anisotropic Diffusion with Diffusion Tensors

- **Design D** with eig-vectors of structure tensor & eig-values $c_t \gg c_n \to 0$
- Why does that work ?
- $\mathrm{div}(D\ \nabla I) = \nabla \cdot (\mathrm{U}\Lambda U^{\mathsf{T}}\ \nabla I)$ where U := [t_{2x1} ; n_{2x1}] and Λ := diag (c_t, c_n)
- $= \nabla^{\mathsf{T}}(t c_t t^{\mathsf{T}} + n c_n n^{\mathsf{T}})\ \nabla I$
- $= \nabla^{\mathsf{T}}\ t c_t t^{\mathsf{T}}\ \nabla I + \nabla^{\mathsf{T}}\ n c_n n^{\mathsf{T}}\ \nabla I$
- $= (t^{\mathsf{T}}\nabla)^{\mathsf{T}}\ c_t\ (t^{\mathsf{T}}\nabla)\ I + (n^{\mathsf{T}}\nabla)^{\mathsf{T}}\ c_n\ (n^{\mathsf{T}}\nabla)\ I$
- $= (\nabla_t)^{\mathsf{T}}\ c_t\ (\nabla_t)\ I + (\nabla_n)^{\mathsf{T}}\ c_n\ (\nabla_n)\ I$
- $= c_t\ \nabla_t^2 I(x,y) + c_n\ \nabla_n^2 I$



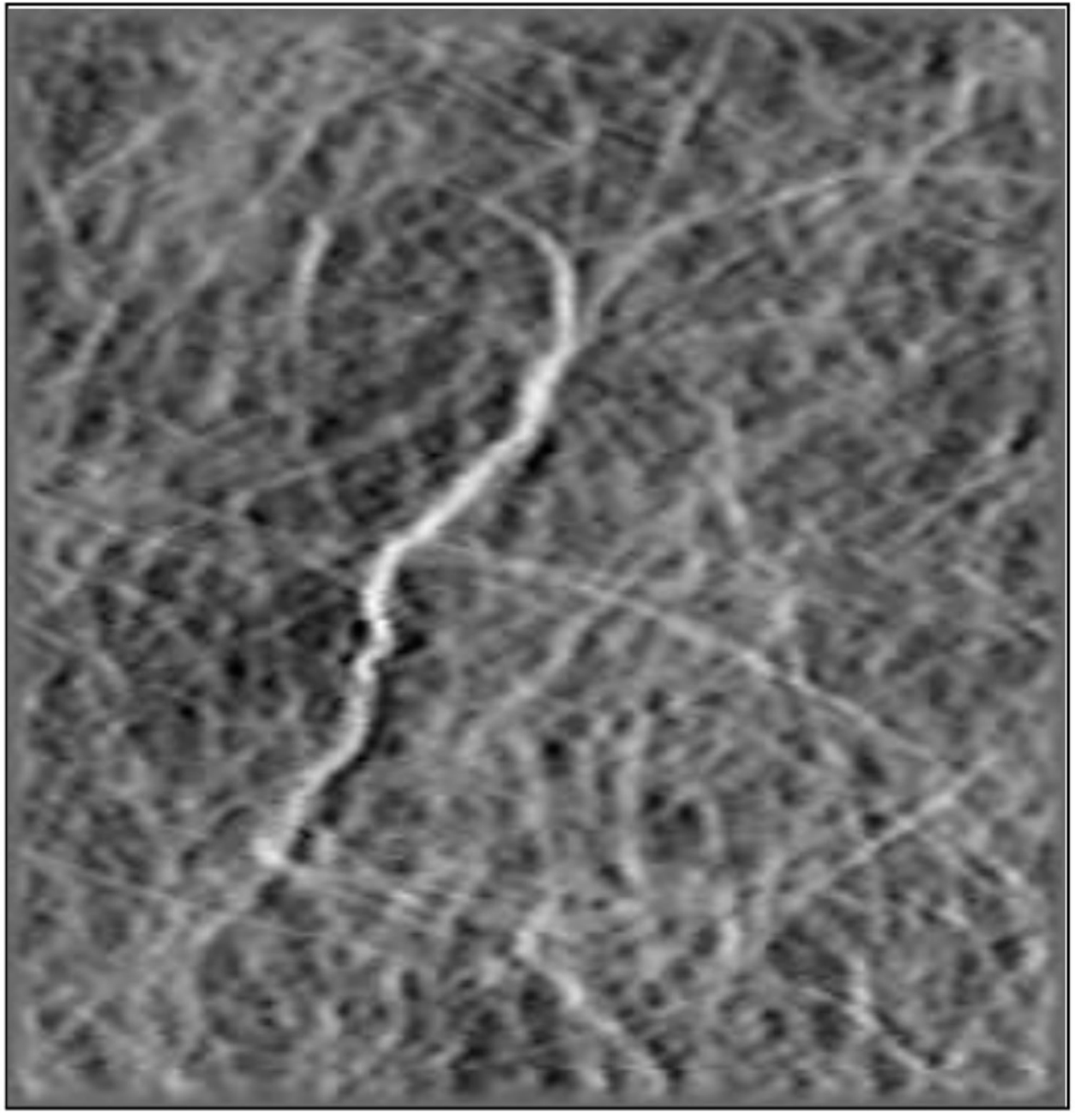
- We can interpret this as $\nabla_U \cdot (\Lambda\ \nabla_U I) = \mathrm{div}_U(\Lambda\ \nabla_U I)$ where $\nabla_U := \left(\frac{\partial}{\partial t}, \frac{\partial}{\partial n}\right)^{\mathsf{T}}$
- where $\nabla_t := t^{\mathsf{T}}\nabla$ & $\nabla_n := n^{\mathsf{T}}\nabla$ are directional-derivative operators along directions t & n, instead of cardinal directions in XY domain

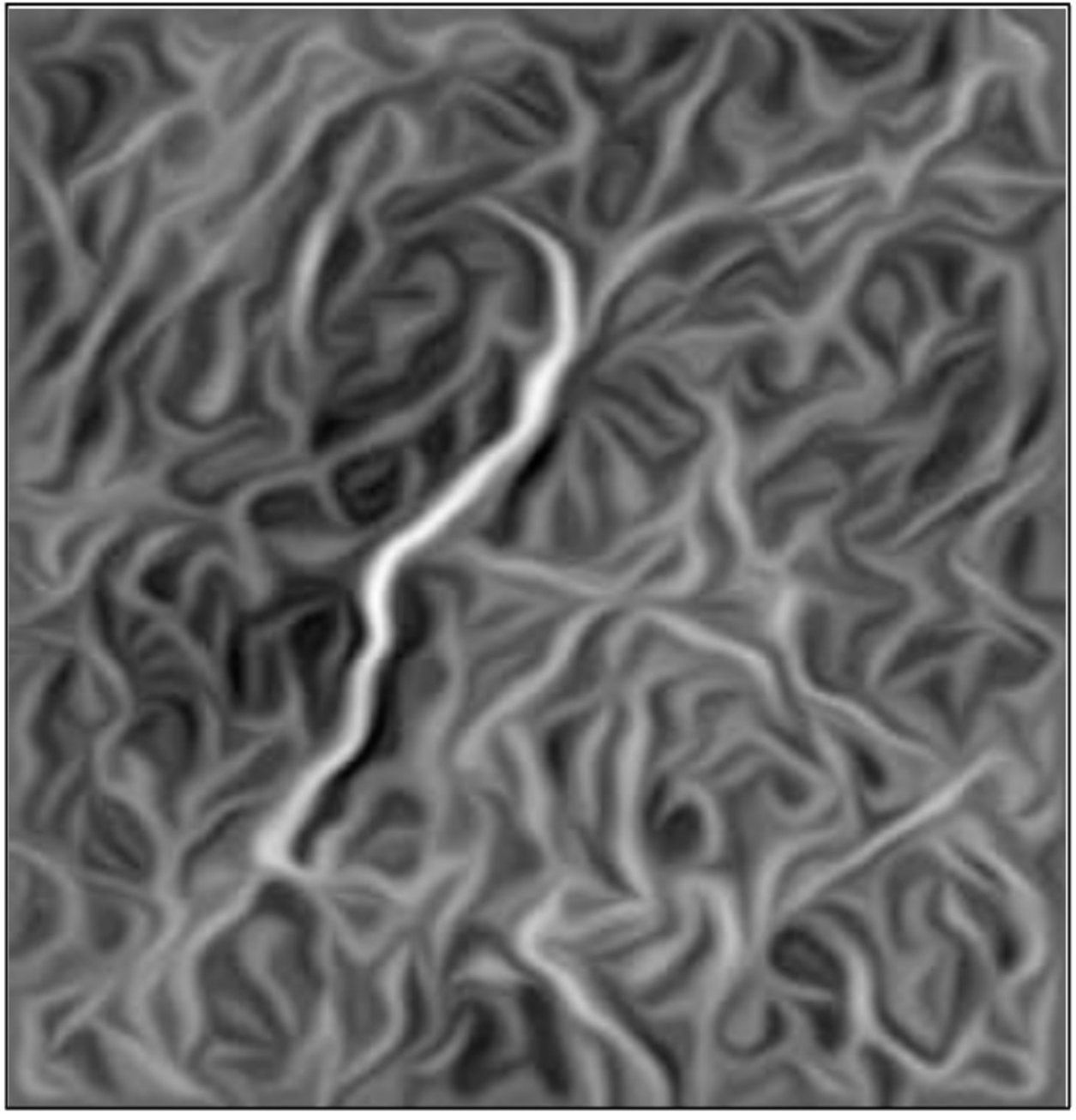
Anisotropic Diffusion with Diffusion Tensors

www.mia.uni-saarland.de/weickert/demos.html

Anisotropic Diffusion with Diffusion Tensors

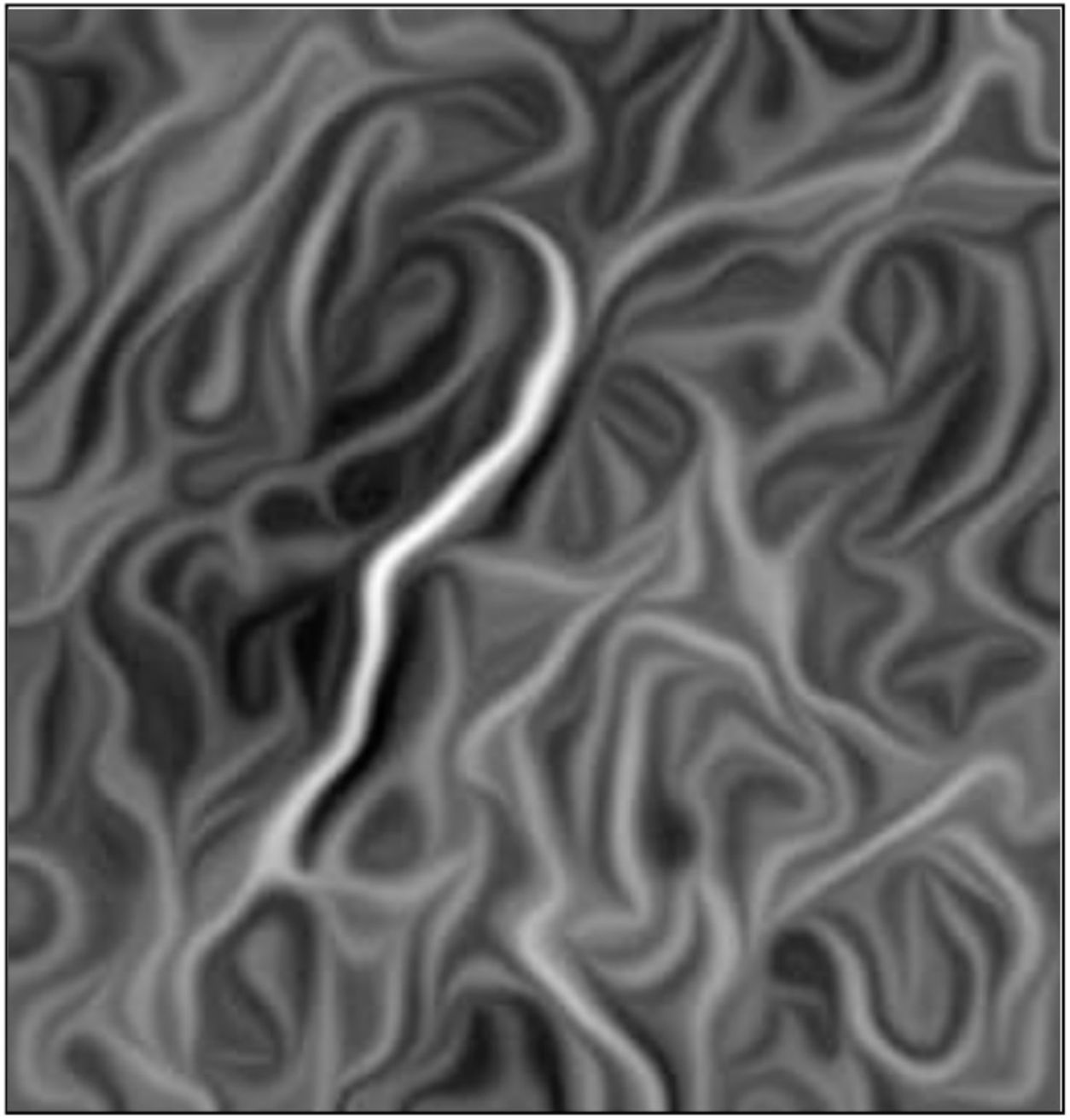
www.mia.uni-saarland.de/weickert/demos.html

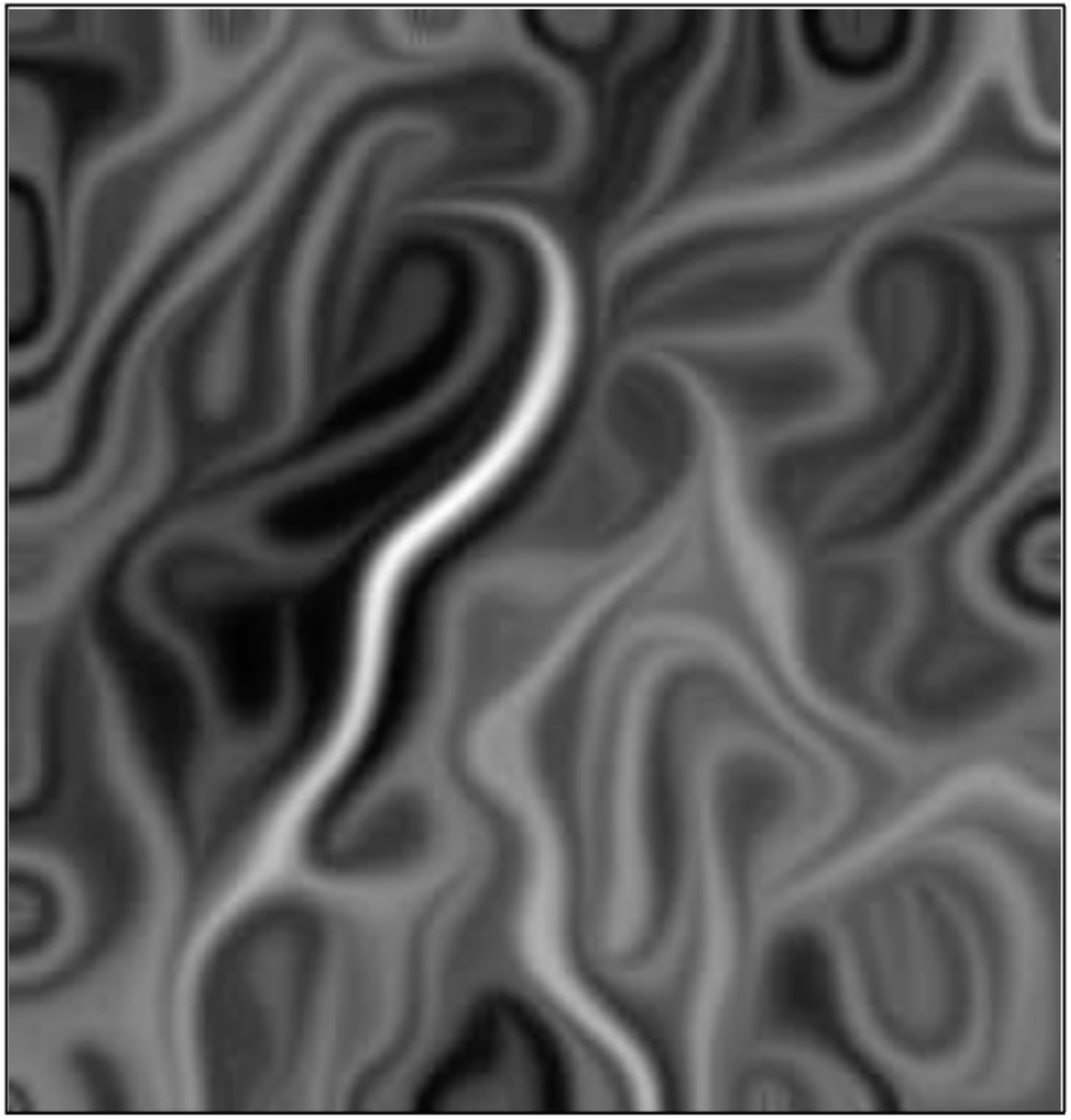




Anisotropic Diffusion with Diffusion Tensors

www.mia.uni-saarland.de/weickert/demos.html

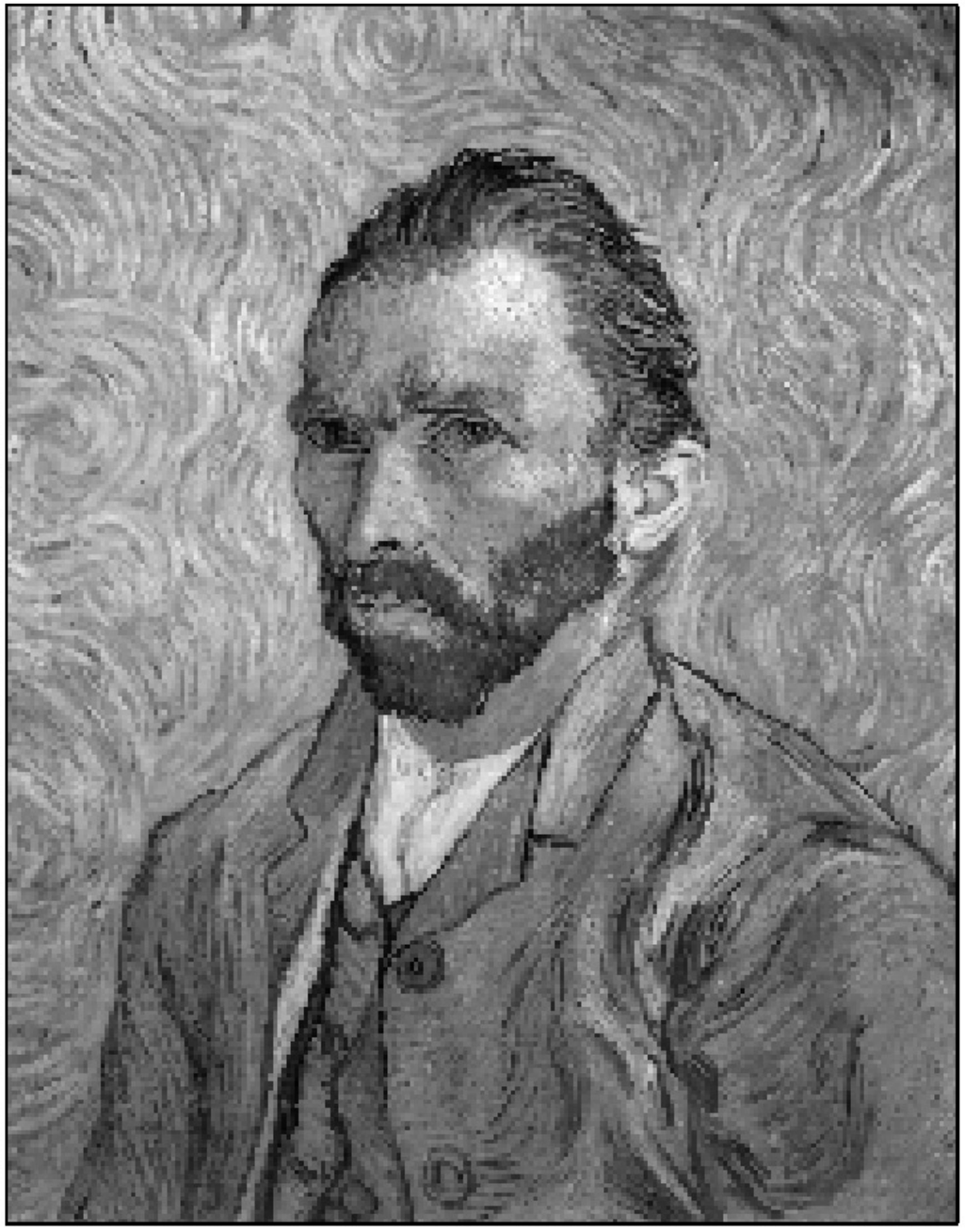




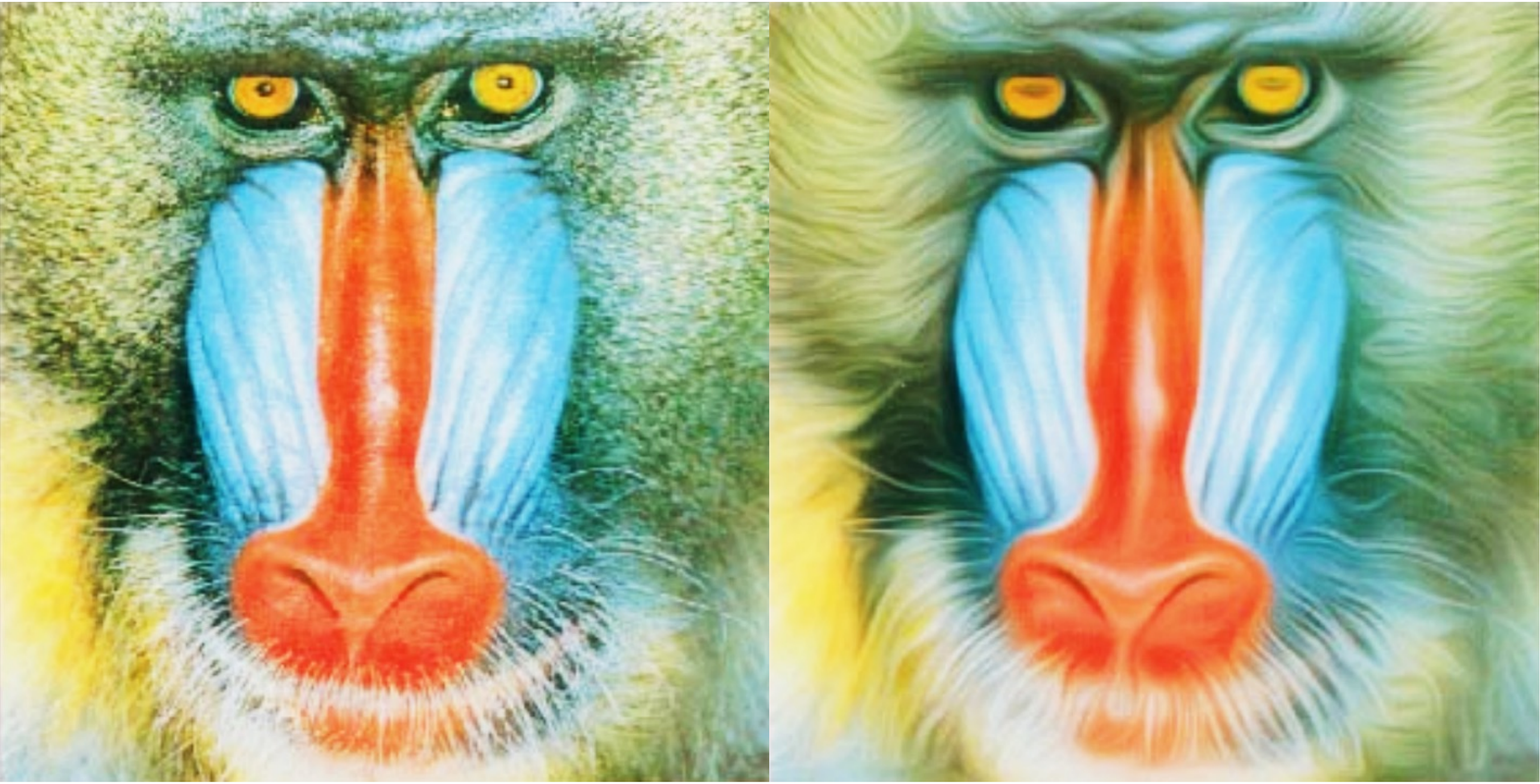
Anisotropic Diffusion with Diffusion Tensors

Anisotropic Diffusion with Diffusion Tensors

www.mia.uni-saarland.de/weickert/demos.html



Anisotropic Diffusion with Diffusion Tensors



Anisotropic Diffusion with Diffusion Tensors

- Robust to noise

Anisotropic Diffusion with Trace and Hessian

- 2005

IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 27, NO. 4, APRIL 2005

Vector-Valued Image Regularization with PDEs: A Common Framework for Different Applications

David Tschumperlé and Rachid Deriche

Multivariate Calculus

- Consider m-valued function of n variables:
  $\mathbf{f}(x_1, \ldots, x_n) := [\, f_1(x_1, \ldots, x_n) \ldots f_m(x_1, \ldots, x_n) \,]$
- **Jacobian**

$$\mathbf{J_f} = \begin{bmatrix} \dfrac{\partial \mathbf{f}}{\partial x_1} & \cdots & \dfrac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^\mathsf{T} f_1 \\ \vdots \\ \nabla^\mathsf{T} f_m \end{bmatrix} = \begin{bmatrix} \dfrac{\partial f_1}{\partial x_1} & \cdots & \dfrac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial f_m}{\partial x_1} & \cdots & \dfrac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Multivariate Calculus

- Consider scalar-valued function $f(x_1, \ldots, x_n)$ of n variables
- **Hessian** matrix

$$(\mathbf{H}_f)_{i,j} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}$$

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \, \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \, \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \, \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \, \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \, \partial x_1} & \frac{\partial^2 f}{\partial x_n \, \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

$$\mathbf{H}(f(\mathbf{x})) = \mathbf{J}(\nabla f(\mathbf{x}))^T$$

Anisotropic Diffusion with Trace and Hessian

- PDE formulation
- Heat equation can be written as:
- Instead, we want to use:

$$\frac{\partial \mathbf{I}}{\partial t} = c_1 \frac{\partial^2 \mathbf{I}}{\partial \xi^2} + c_2 \frac{\partial \mathbf{I}}{\partial \eta^2} = c_1 \mathbf{I}_{\xi\xi} + c_2 \mathbf{I}_{\eta\eta}$$

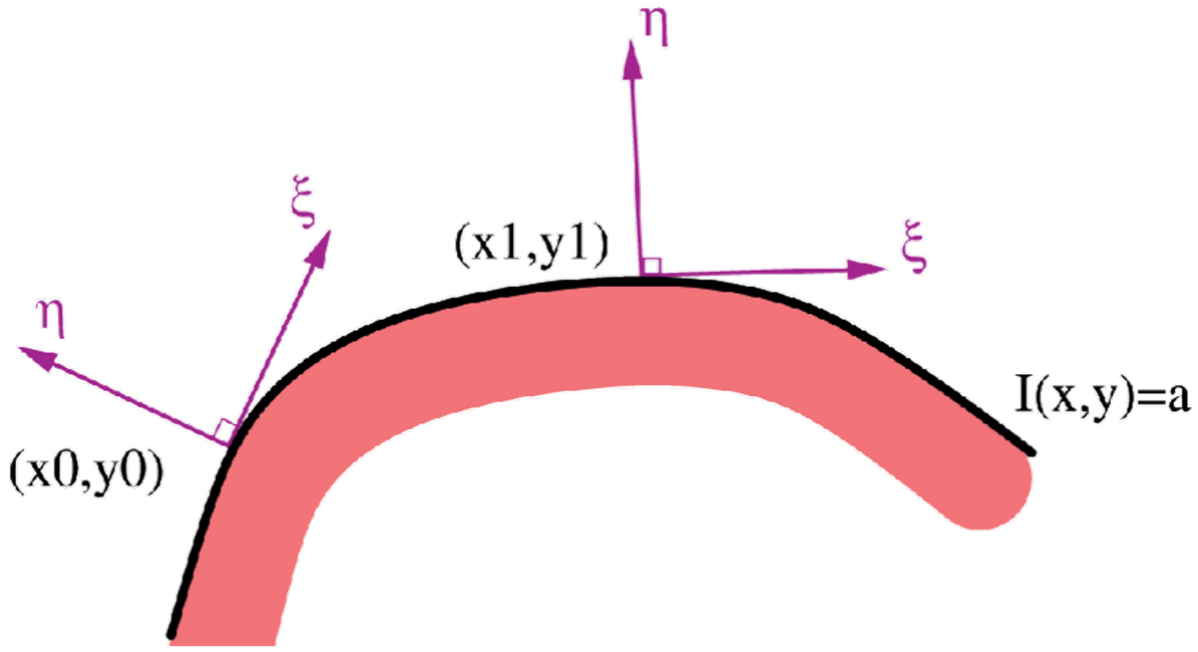
- So, do, for channel i in (RGB) image:

$$\frac{\partial I_i}{\partial t} = c_1 I_{i_{\xi\xi}} + c_2 I_{i_{\eta\eta}} = \mathbf{trace}\,(\mathbf{T}\mathbf{H}_i)$$

$$\mathbf{T} = c_1 \xi \xi^T + c_2 \eta \eta^T$$

$$\frac{\partial I}{\partial t} = c \cdot (\nabla^2 I) = \mathrm{div}(c \cdot \nabla I)$$

$$= \mathrm{trace}\,((c \cdot \mathbf{I}_2) \cdot \mathbf{H}_I) = \mathrm{trace}\,(c \cdot \mathbf{H}_I)$$



- T defines smoothing directions (ξ, η) & amounts (where c_1, $c_2 > 0$)
  T remains same for all color channels
  T accounts for information in all color channels
- H_i can change with each channel i

Anisotropic Diffusion with Trace and Hessian

- PDE formulation

$$\frac{\partial I_i}{\partial t} = c_1 \, I_{i\xi\xi} + c_2 \, I_{i\eta\eta} = \mathbf{trace}\,(\mathbf{T}\mathbf{H}_i)$$

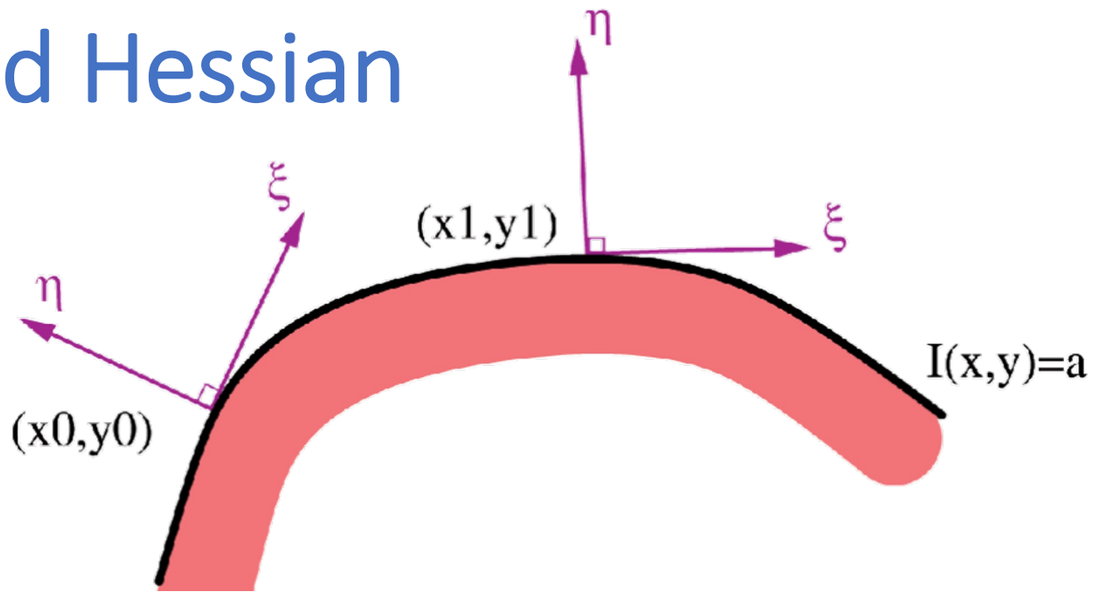
- Why does that work ?
- Let T := some SPD matrix $:= U\Lambda U^{\mathsf{T}}$ where U orthogonal, Λ diagonal
- $\mathrm{tr}(TH) = \mathrm{tr}(U\Lambda U^{\mathsf{T}} H) = \mathrm{tr}(U^{\mathsf{T}} U\Lambda U^{\mathsf{T}} HU)$ …
  - … because trace(AB)=trace(BA), and we let A := U^{T} and B := $U\Lambda U^{\mathsf{T}} HU$
  - … because trace is invariant to orthogonal similarity transformations
- $= \mathrm{tr}(\Lambda U^{\mathsf{T}} HU)$ where $U^{\mathsf{T}} HU$ is Hessian in coordinate basis U …
- … because, in Taylor series,

$$f(\mathbf{x}+\Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^{\mathsf{T}} \Delta\mathbf{x} + \frac{1}{2}\,\Delta\mathbf{x}^{\mathsf{T}} \mathbf{H}(\mathbf{x})\,\Delta\mathbf{x}$$

  term $\Delta x^{\mathsf{T}} H \Delta x = \Delta x^{\mathsf{T}}\, UU^{\mathsf{T}}\, H\, UU^{\mathsf{T}}\, \Delta x = \Delta y^{\mathsf{T}}\, U^{\mathsf{T}} HU\, \Delta y$
  where $\Delta y := U^{\mathsf{T}} \Delta x$ are coordinates of displacement in basis U
  - Similarly, term $\nabla f(x)^{\mathsf{T}} \Delta x = \nabla f(x)^{\mathsf{T}} UU^{\mathsf{T}} \Delta x = (\mathrm{U}^{\mathsf{T}} \nabla f(x))^{\mathsf{T}} \Delta y$

Anisotropic Diffusion with Trace and Hessian

- Trace-Hessian based formulation
- Tschumperle-Deriche method



proposes $\dfrac{\partial I_k}{\partial t} = \mathrm{trace}\left(\boldsymbol{A} \cdot \mathbf{H}_k\right)$

- Design $\boldsymbol{A} = \begin{pmatrix} A_0 & A_1 \\ A_1 & A_2 \end{pmatrix} = \underbrace{f_1(\lambda_1, \lambda_2)}_{c_1} \cdot (\hat{e}_2 \cdot \hat{e}_2^{\mathsf{T}}) + \underbrace{f_2(\lambda_1, \lambda_2)}_{c_2} \cdot (\hat{e}_1 \cdot \hat{e}_1^{\mathsf{T}})$

  - e_1, e_2 ← eigenvectors (unit-norm) of structure tensor
  - λ_1, λ_2 ← eigenvalues (non-negative) of structure tensor
  - If $\lambda_1 \gg \lambda_2$, i.e., direction e_1 is across edge, then smooth/diffuse mainly along e_2, i.e., coeff. of $e_2\, e_2^{\mathsf{T}}$ should be larger i.e., $c_1 \gg c_2$
  - i.e., in $f_1(.)$ and $f_2(.)$, $a_2 \gg a_1$

$$c_1 = f_1(\lambda_1, \lambda_2) = \frac{1}{(1 + \lambda_1 + \lambda_2)^{a_1}}$$

$$c_2 = f_2(\lambda_1, \lambda_2) = \frac{1}{(1 + \lambda_1 + \lambda_2)^{a_2}}$$

Anisotropic Diffusion with Trace and Hessian

- Tschumperle-Deriche propose different discretization for solving PDE
- For each channel k

$$I_k \leftarrow I_k + \alpha \cdot \mathrm{trace}\left(\boldsymbol{A} \cdot \mathbf{H}_k\right) = I_k + \alpha \cdot \beta_k$$

$$= I_k + \alpha \cdot \left(A_0 \cdot I_{k,\mathrm{xx}} + A_1 \cdot I_{k,\mathrm{xy}} + A_1 \cdot I_{k,yx} + A_2 \cdot I_{k,\mathrm{yy}}\right)$$

$$= I_k + \alpha \cdot \underbrace{\left(A_0 \cdot I_{k,\mathrm{xx}} + 2 \cdot A_1 \cdot I_{k,\mathrm{xy}} + A_2 \cdot I_{k,\mathrm{yy}}\right)}_{\beta_k}$$

- Choose time step for stability

$$\alpha = \frac{d_\mathrm{t}}{\max \beta_k} = \frac{d_\mathrm{t}}{\max\limits_{k,\boldsymbol{u}} \left|\mathrm{trace}(\boldsymbol{A} \cdot \mathbf{H}_k)\right|}$$

Anisotropic D

- Left half: noisy (iid Gaussian and salt-&-pepper)
- Right half: denoised

Anisotropic Diffusion with Trace and Hessian

Bilateral Filter

- 1997 International Journal of Computer Vision (IJCV)

SUSAN—A New Approach to Low Level Image Processing

STEPHEN M. SMITH*
Department of Clinical Neurology, Oxford University, Oxford, UK
steve@robots.ox.ac.uk

J. MICHAEL BRADY
Department of Engineering Science, Oxford University, Oxford, UK
jmb@robots.ox.ac.uk

Bilateral Filter

- 1998 International Conference on Computer Vision (ICCV)

Bilateral Filtering for Gray and Color Images

C. Tomasi *

Computer Science Department
Stanford University
Stanford, CA 94305
tomasi@cs.stanford.edu

R. Manduchi

Interactive Media Group
Apple Computer, Inc.
Cupertino, CA 95014
manduchi@apple.com

Bilateral Filter

Foundations and Trends® in
Computer Graphics and Vision
Vol. 4, No. 1 (2008) 1–73
© 2009 S. Paris, P. Kornprobst, J. Tumblin and
F. Durand
DOI: 10.1561/0600000020

now
the essence of knowledge

Bilateral Filtering: Theory and Applications

By Sylvain Paris, Pierre Kornprobst, Jack Tumblin
and Frédo Durand

Bilateral Filt

(a) Input image

(b) Output of the bilateral filter

Fig. 1.1 The bilateral filter converts any input image (a)to a smoothed version (b). It removes most texture, noise, and fine details, but preserves large sharp edges without blurring.

Bilateral Filter

- At pixel location p,
  weight for another pixel location q is based on, **BOTH**,:

1) Spatial distance between locations p and q
   - Larger distance → lower weight
   - e.g., Gaussian weighting over distance, as done in Gaussian convolution
2) Dissimilarity between intensities $I(p)$ and $I(q)$
   - Larger dissimilarity → lower weight
   - e.g., Gaussian weighting over intensity

Bilateral Filter

- Updated pixel intensity =

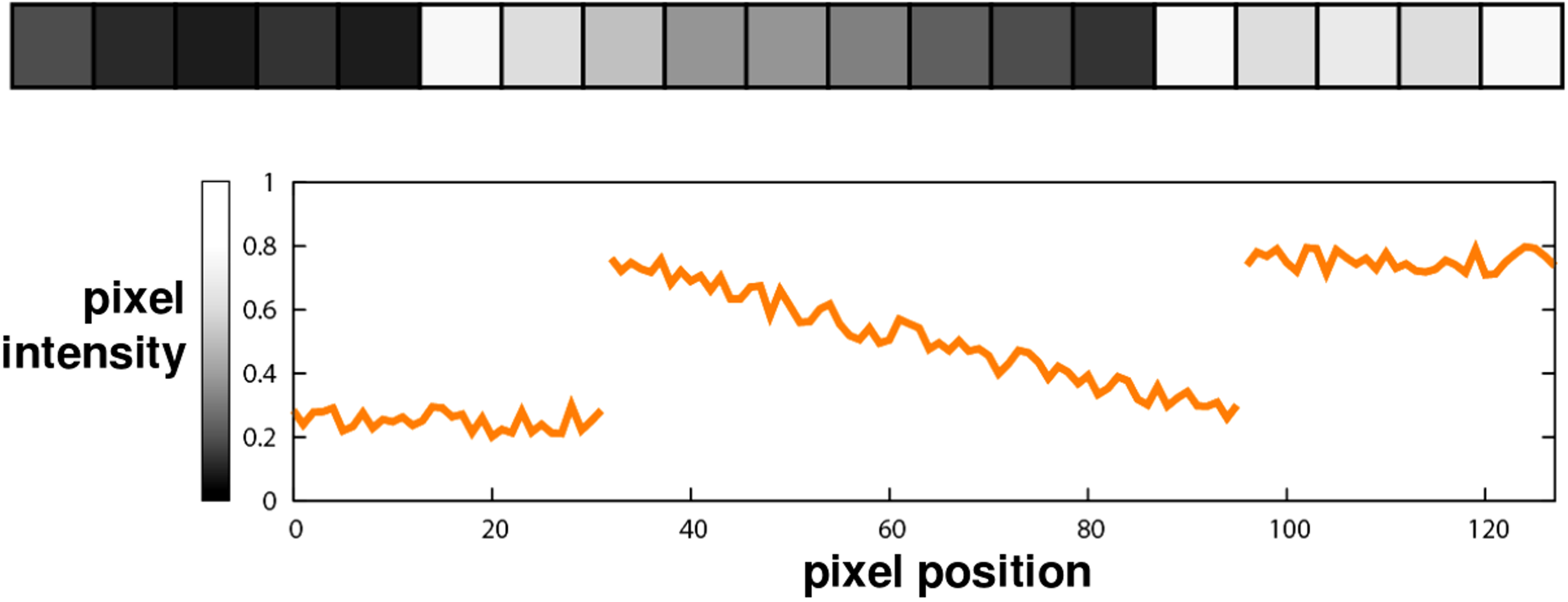
$$BF[I]_{\mathbf{p}} = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)\, I_{\mathbf{q}}$$

- where normalization factor W_p
  ensures weights sum to 1,
  i.e., leads to convex combination of intensities

$$W_{\mathbf{p}} = \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)$$

Bilateral Filter

- Example 1D image

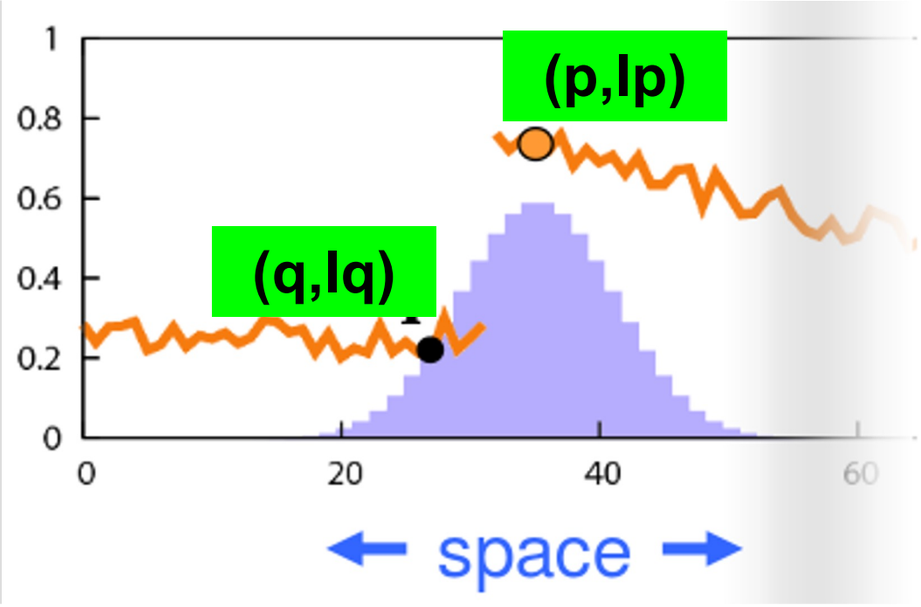


cs.brown.edu/courses/cs129/lectures/bf_course_Brown_Oct2012.pdf

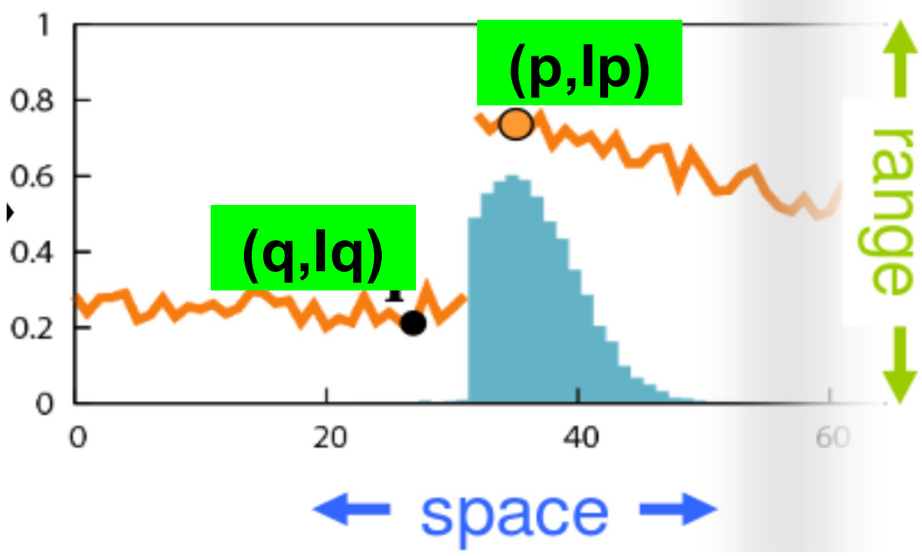
Bilateral Filter

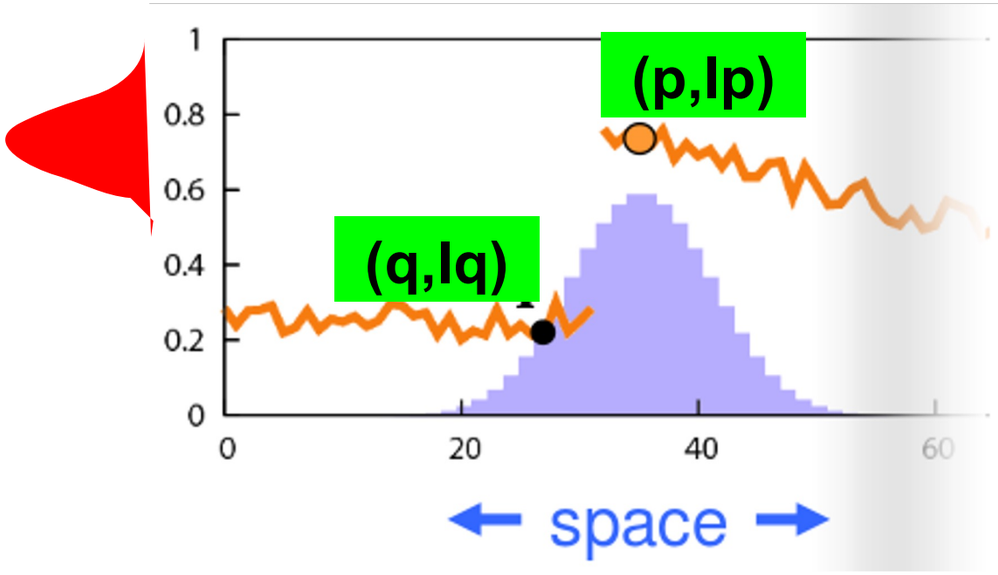
$$BF[I]_{\mathbf{p}} = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)\, I_{\mathbf{q}}$$

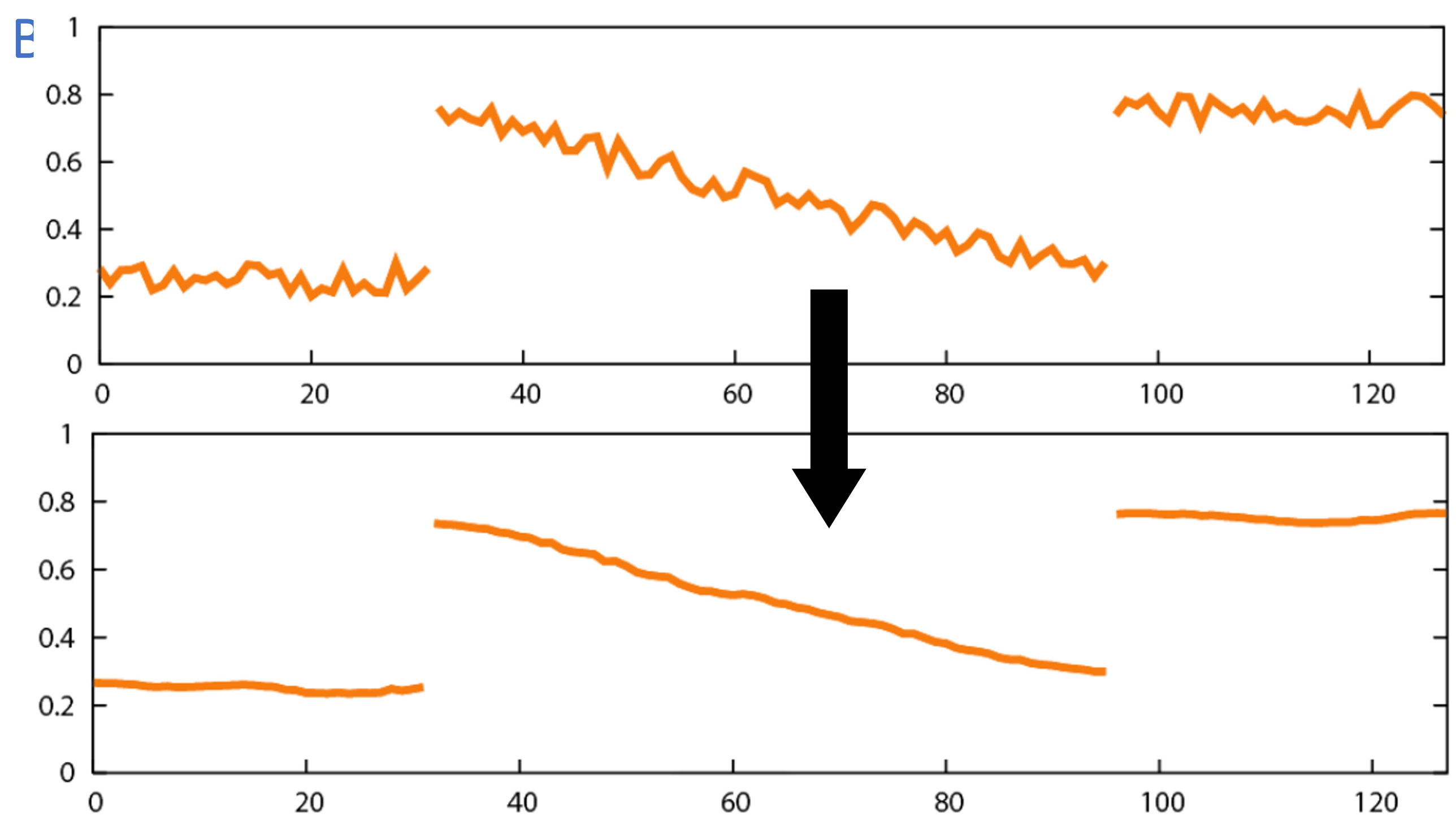
- Effect of **Gaussian** filter on weights at pixel location p



- Effect of **bilateral** filter on weights at pixel location p






B
1
0.8
0.6
0.4
0.2
0
0
20
40
60
80
100
120
1
0.8
0.6
0.4
0.2
0
0
20
40
60
80
100
120

Bilateral

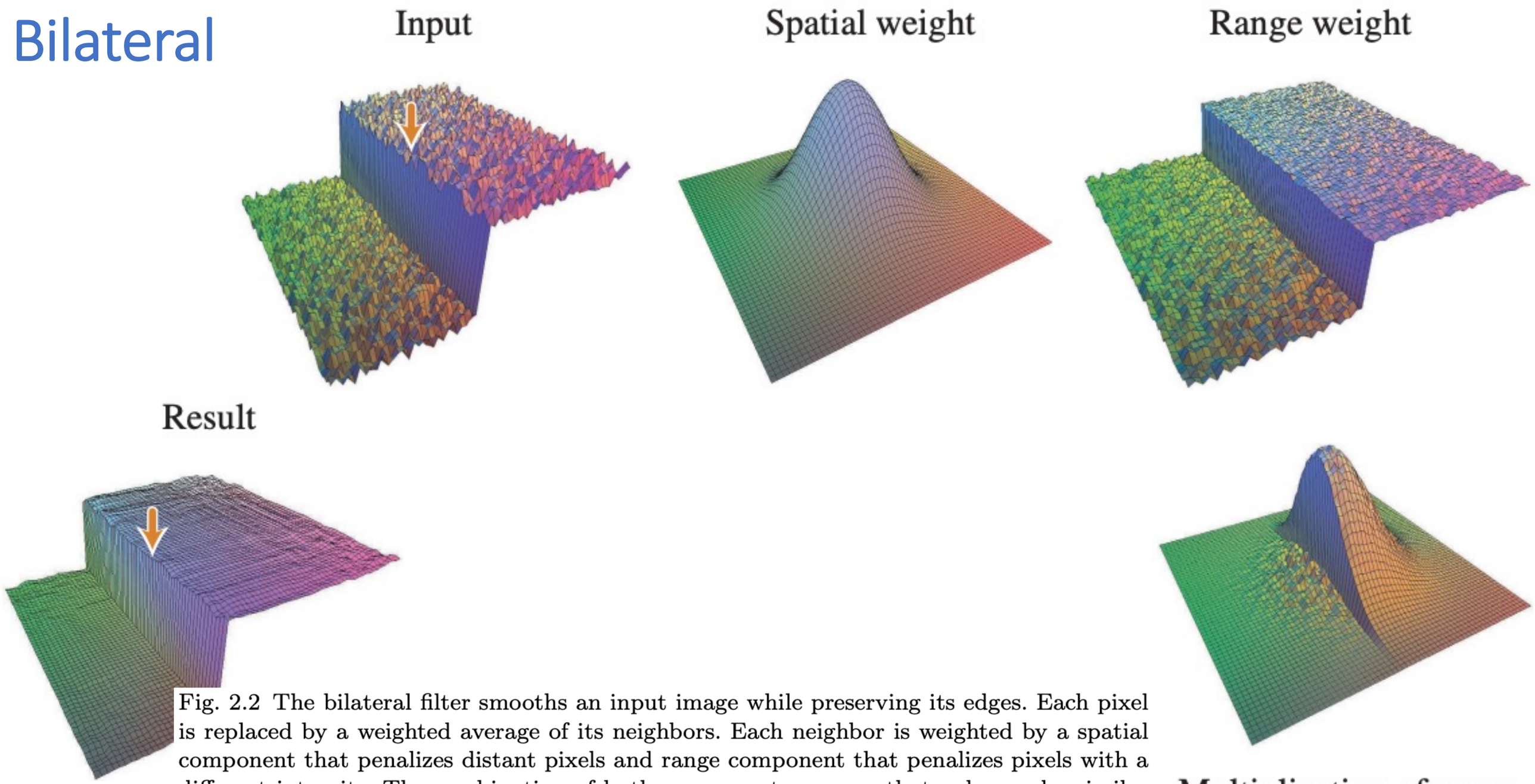
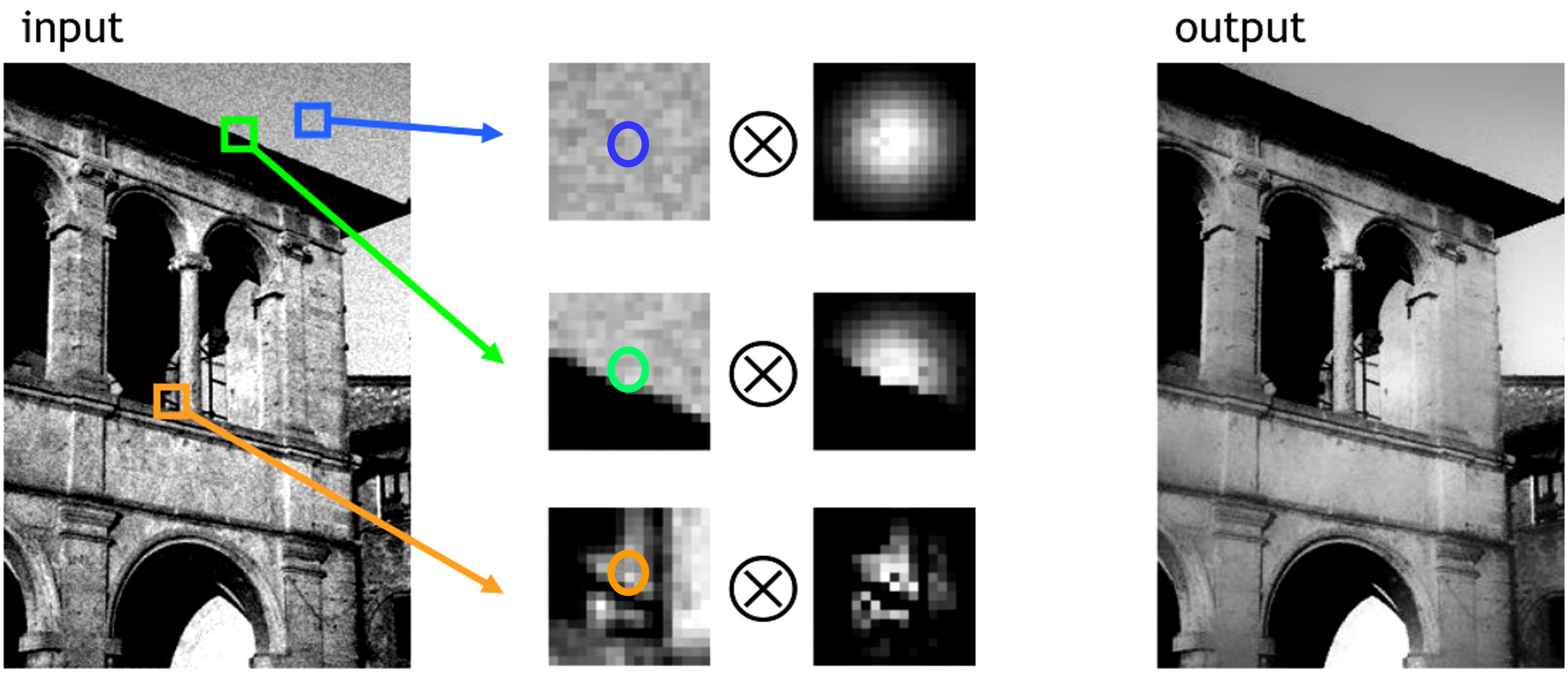


Fig. 2.2 The bilateral filter smooths an input image while preserving its edges. Each pixel is replaced by a weighted average of its neighbors. Each neighbor is weighted by a spatial component that penalizes distant pixels and range component that penalizes pixels with a different intensity. The combination of both components ensures that only nearby similar pixels contribute to the final result. The weights shown apply to the central pixel (under the arrow). The figure is reproduced from [21].

Bilateral Filter

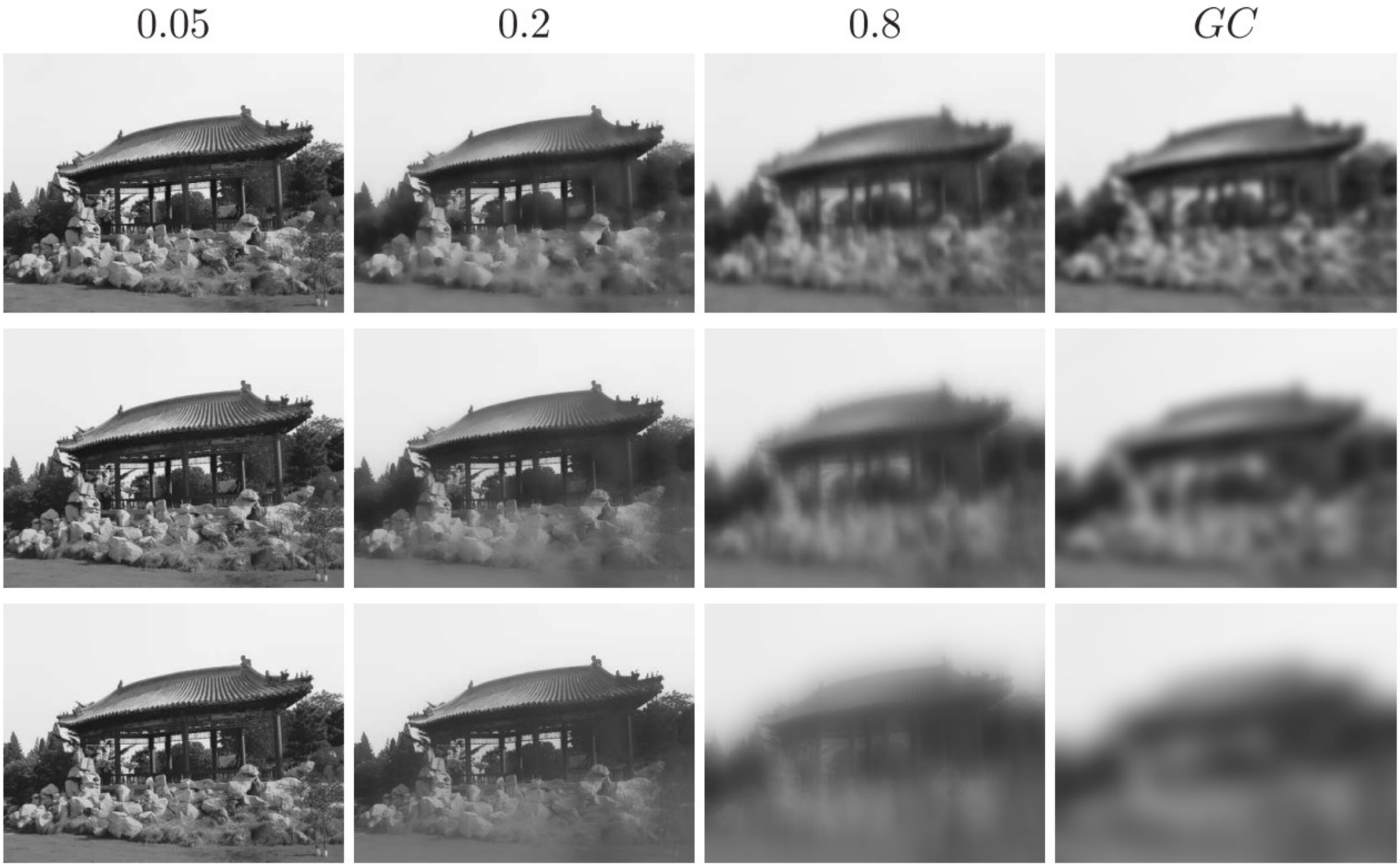
- On real images, weight distribution, for different pixels, can be complex



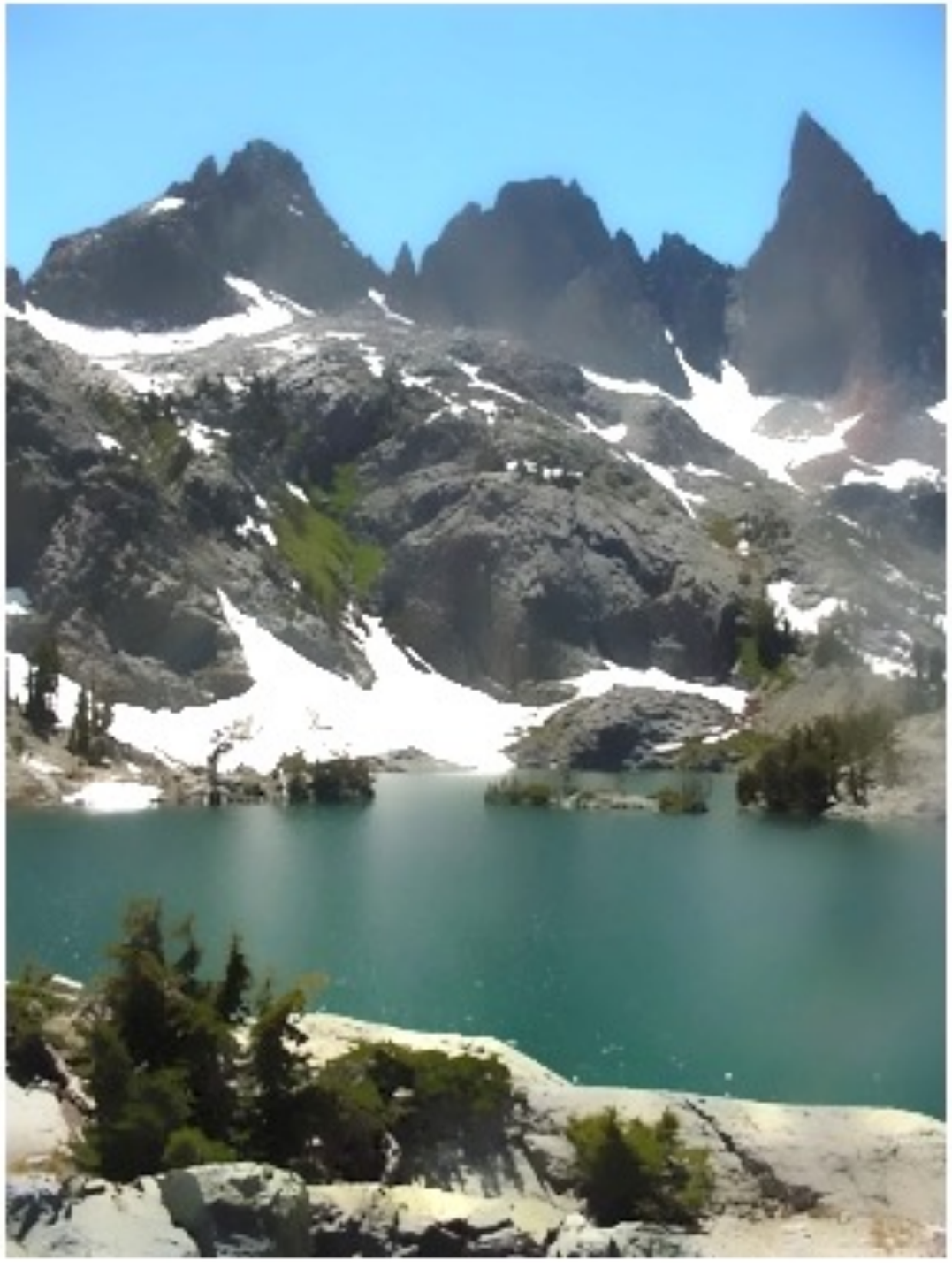
Bilateral Filter

- GC = Gauss. Conv.

The original image intensity values span $[0, 1]$.



Bilateral

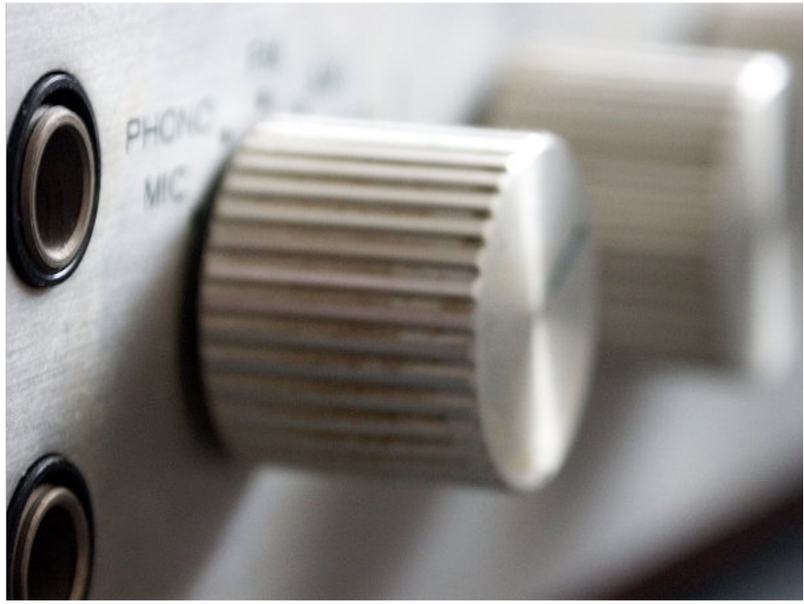


Bilateral Filter

$$BF[I]_{\mathbf{p}} = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)\, I_{\mathbf{q}}$$

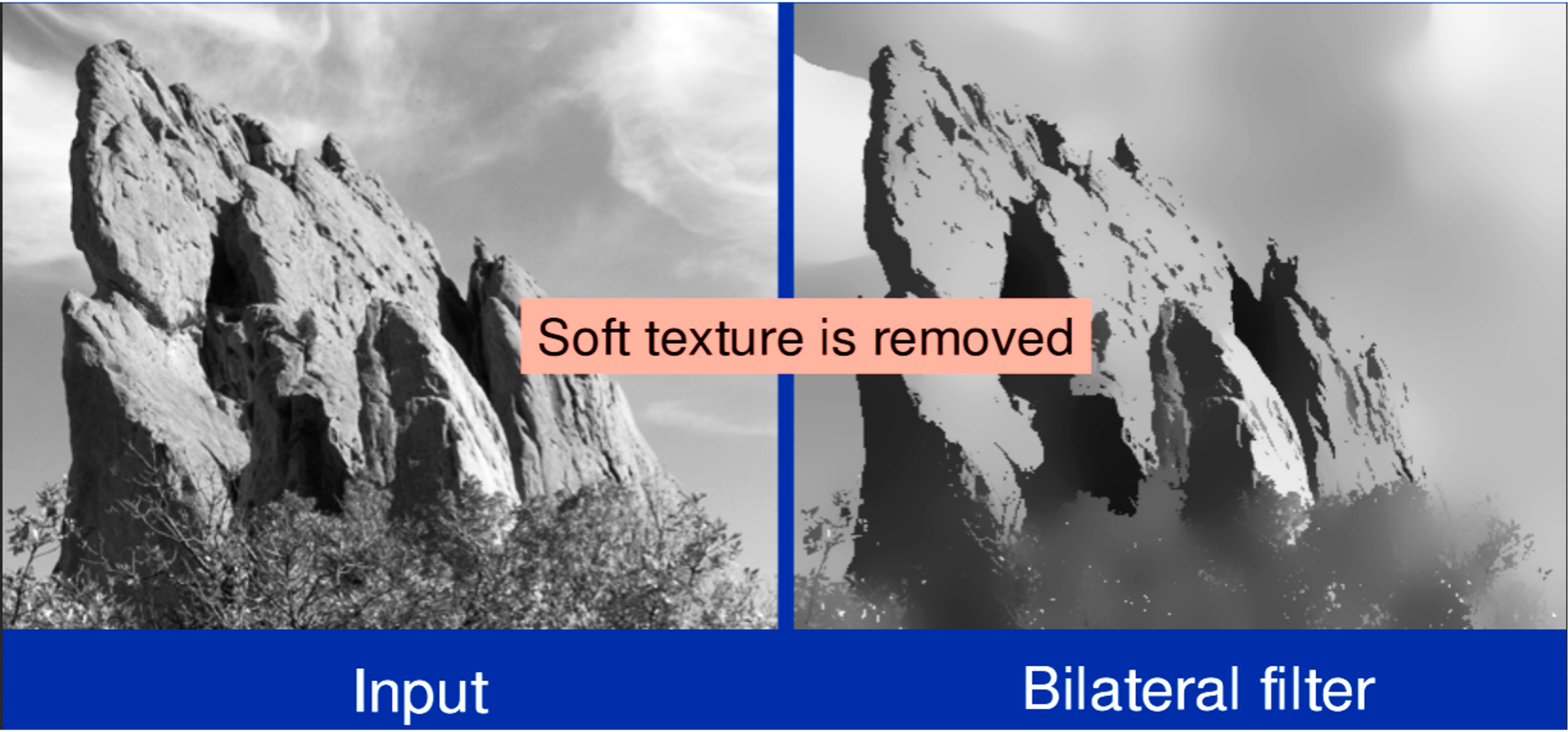
$$W_{\mathbf{p}} = \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)$$

- Parameters
- What happens when:
- … for Gaussian f(.) on intensities, variance → ∞ ?
  - Bilateral filter → Gaussian smoothing
- … for Gaussian f(.) on intensities, variance → 0 ?
  - Filter doesn't modify image
- … for Gaussian g(.) on space, variance → 0 ?
  - Filter doesn't modify image
- … for Gaussian g(.) on space, variance → ∞ ?
  - Filter averages similar intensities across entire image



Bilateral Filter

- Texture “softer” than intensity-kernel’s std. dev. is smoothed out



Bilateral Filter, Anisotropic Diffusion

IEEE Transactions on Pattern Analysis and Machine Intelligence

A Fundamental Relationship between Bilateral Filtering, Adaptive Smoothing, and the Nonlinear Diffusion Equation

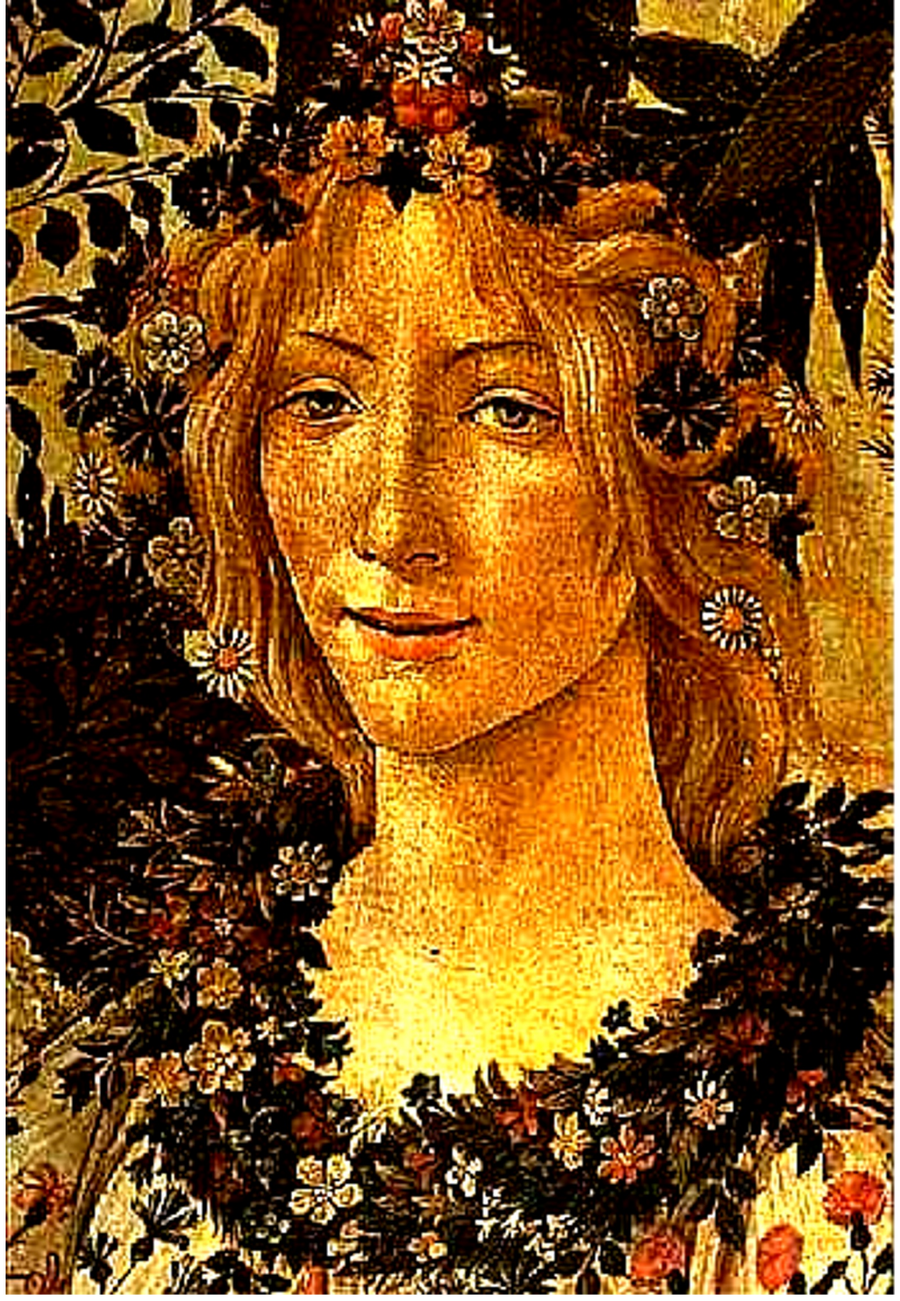
Jun. 2002, pp. 844-847, vol. 24

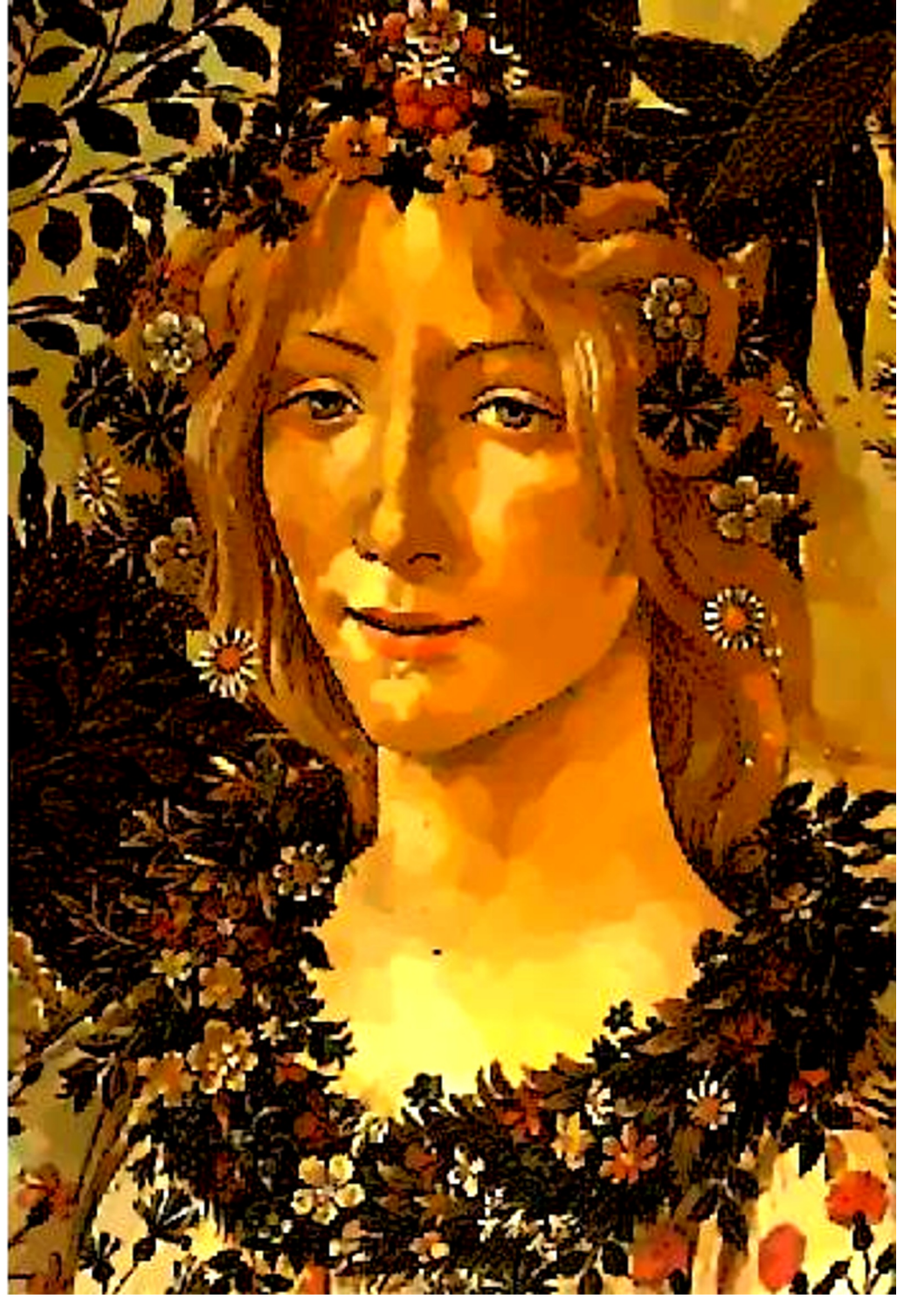
Fig. 2. Edge-preserving smoothing: anisotropic diffusion with 20 time-steps of $\tau = 1.0$ (left) and Gaussian bilateral filtering with a 15×15 window size, $\sigma_D = 5.0$ and $\sigma_R = 30.0$ (right). σ_D and σ_R are bilateral filtering parameters, see [9] for details.

Bilateral Filter

- Limitations

1) Staircase effect: intensity plateaus make images appear cartoonish
2) Introduction of false edges in image
   - Similar to median filter
3) Computing cost
   - O(NumOfPixels * Size2DNeighborhood)





Bilateral Filter

- Converting limitations into strength
- Making cartoon versions of real scenes

Input

Output

people.csail.mit.edu/sparis/bf_course/slides/09_limitations.pdf

Bilateral Filter

- Converting limitations into strength

Ghibli Style

Studio Ghibli

Fast Bilateral Filter

- European Conference on Computer Vision (ECCV) 2006

A Fast Approximation of the Bilateral Filter using a Signal Processing Approach

Sylvain Paris and Frédo Durand

Massachusetts Institute of Technology
Computer Science and Artificial Intelligence Laboratory

Fast Bilateral Filter

- Two key equations for processing pixel at location p

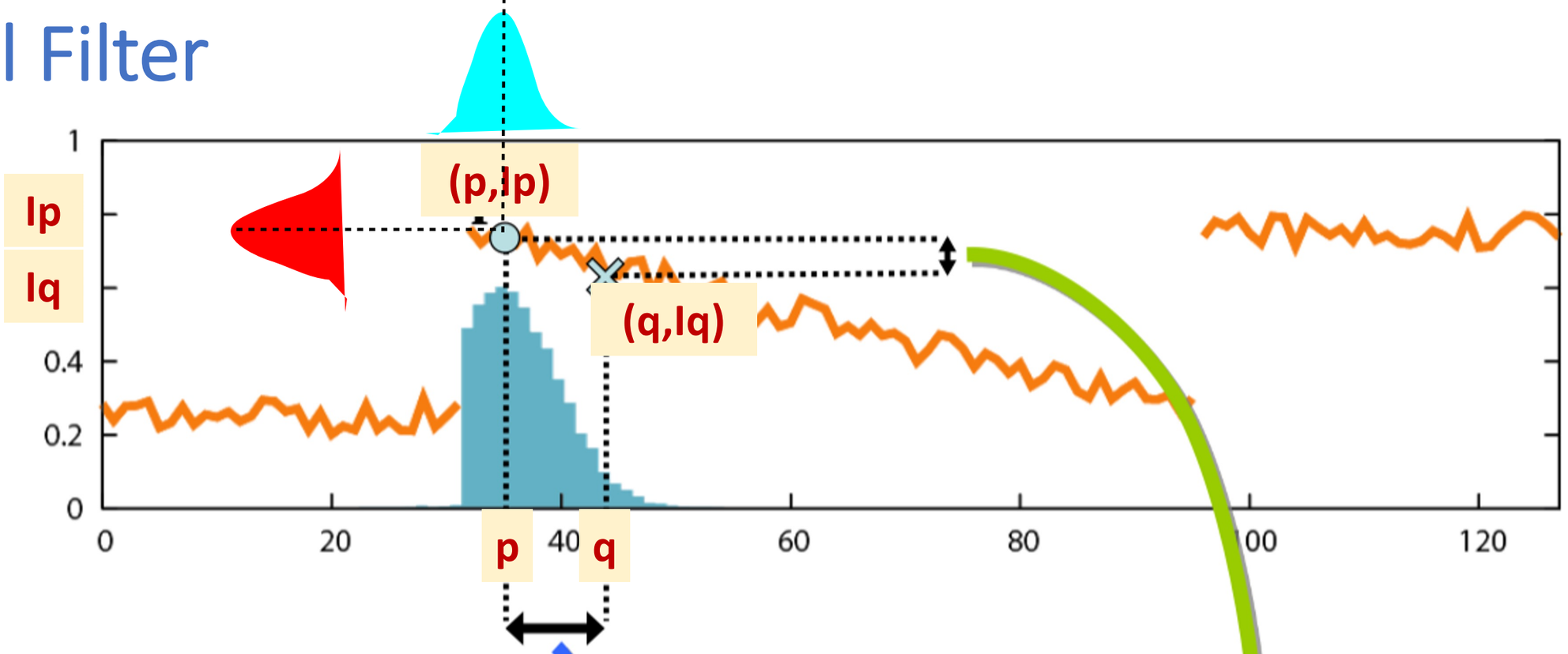
$$I_{\mathbf{p}}^{\mathrm{bf}} = \frac{1}{W_{\mathbf{p}}^{\mathrm{bf}}} \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)\, I_{\mathbf{q}}$$

$$W_{\mathbf{p}}^{\mathrm{bf}} = \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)$$

- In 1st equation, multiply both sides by normalization factor

$$\begin{pmatrix} W_{\mathbf{p}}^{\mathrm{bf}} I_{\mathbf{p}}^{\mathrm{bf}} \\ W_{\mathbf{p}}^{\mathrm{bf}} \end{pmatrix} = \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|) \begin{pmatrix} I_{\mathbf{q}} \\ 1 \end{pmatrix}$$

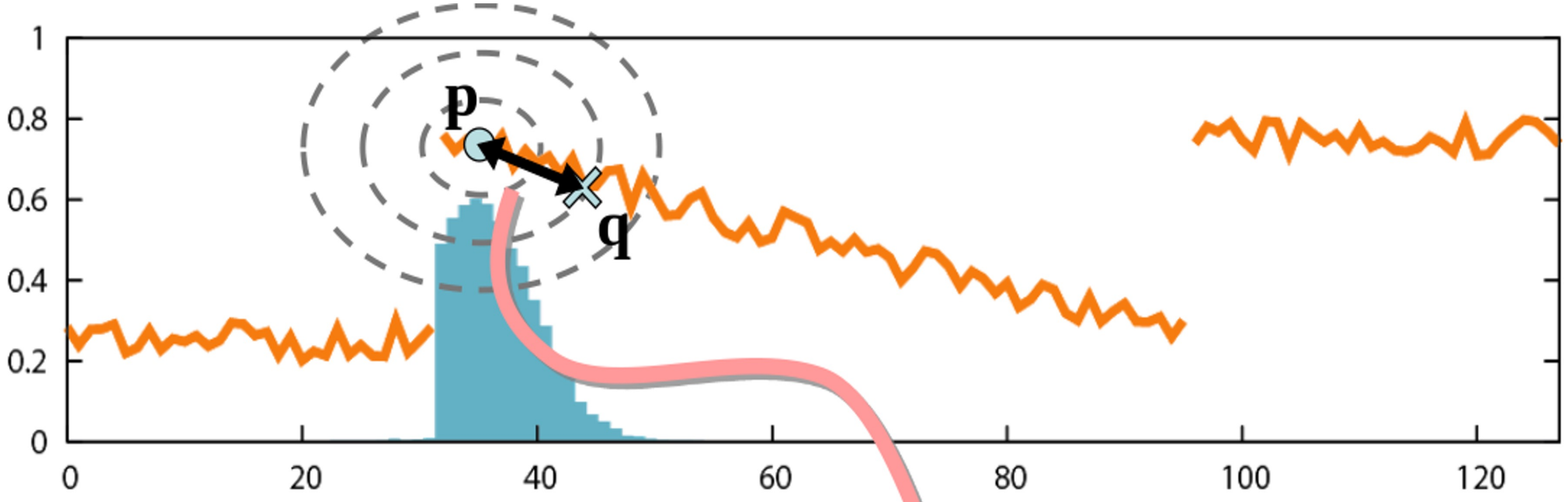
Fast Bilateral Filter



$$\begin{pmatrix} W_{\mathbf{p}}^{\mathrm{bf}} I_{\mathbf{p}}^{\mathrm{bf}} \\ W_{\mathbf{p}}^{\mathrm{bf}} \end{pmatrix} = \sum_{\mathbf{q}\in\mathcal{S}} \underbrace{G_{\sigma_s}(\|\mathbf{p}-\mathbf{q}\|)}_{\text{space}} \underbrace{G_{\sigma_r}(|I_{\mathbf{p}}-I_{\mathbf{q}}|)}_{\text{range}} \begin{pmatrix} I_{\mathbf{q}} \\ 1 \end{pmatrix}$$

2D Gaussian

Fast Bilateral Filter



$$\begin{pmatrix} W_{\mathbf{p}}^{\mathrm{bf}} I_{\mathbf{p}}^{\mathrm{bf}} \\ W_{\mathbf{p}}^{\mathrm{bf}} \end{pmatrix} = \sum_{\mathbf{q}\in\mathcal{S}} G_{\sigma_s}(\|\mathbf{p}-\mathbf{q}\|)\, G_{\sigma_r}(|I_{\mathbf{p}}-I_{\mathbf{q}}|) \begin{pmatrix} I_{\mathbf{q}} \\ 1 \end{pmatrix}$$

space x range

Fast Bilateral Filter

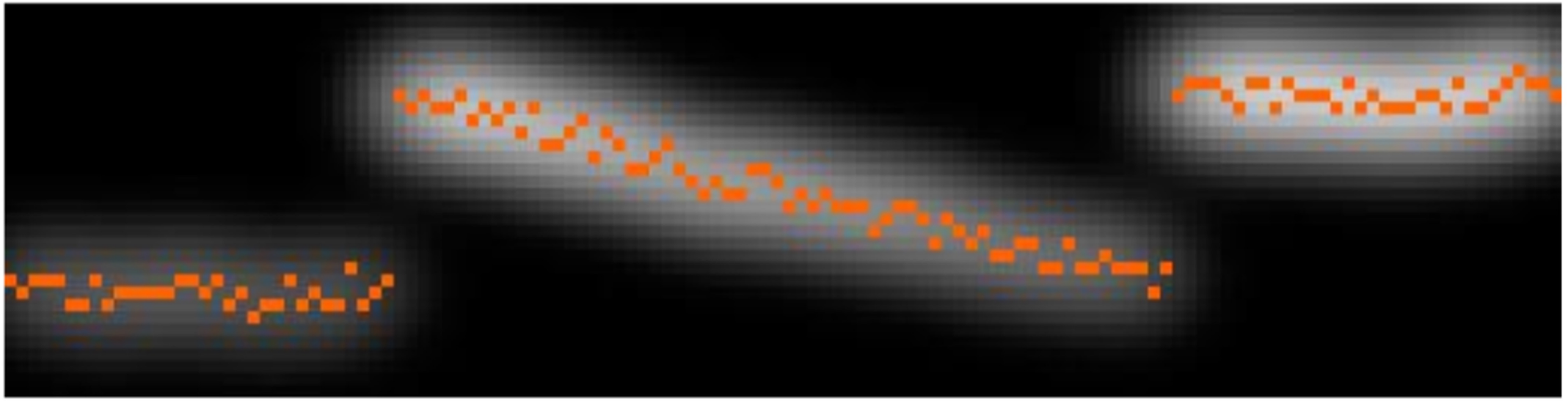
- **Re-interpret**
- Convolution in a higher-dim. space that is designed from ⟨location, intensity⟩
- 1D image in previous slide becomes 2D image

sum all values

To compute **numerator**.
Construct a **higher-dimensional space**.
1) At "location" (p,I_p), assign intensity I_p
2) Assign 0 elsewhere

$$\begin{pmatrix} W_{\mathbf{p}}^{\mathrm{bf}} & I_{\mathbf{p}}^{\mathrm{bf}} \\ W_{\mathbf{p}}^{\mathrm{bf}} & \end{pmatrix} = \sum\sum_{(\mathbf{q},\zeta)\in\mathcal{S}\times\mathcal{R}} \text{space-range Gaussian} \begin{pmatrix} I_{\mathbf{q}} \\ 1 \end{pmatrix}$$

Fast Bilateral Filter



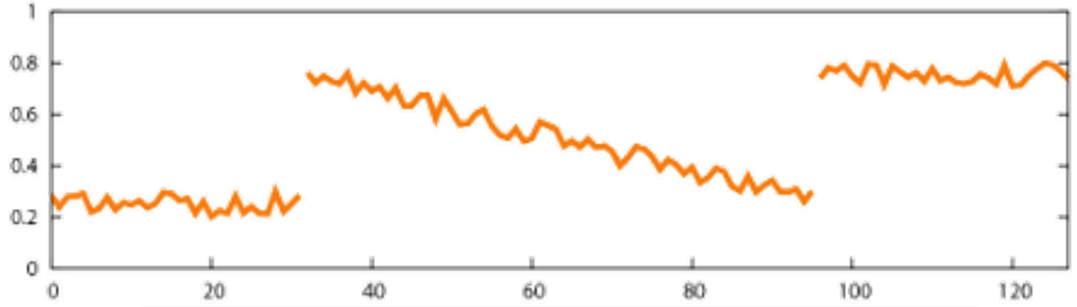
result of the convolution

Get numerator values by picking them from higher-dimensional "locations" (q,I_q) in convolved image

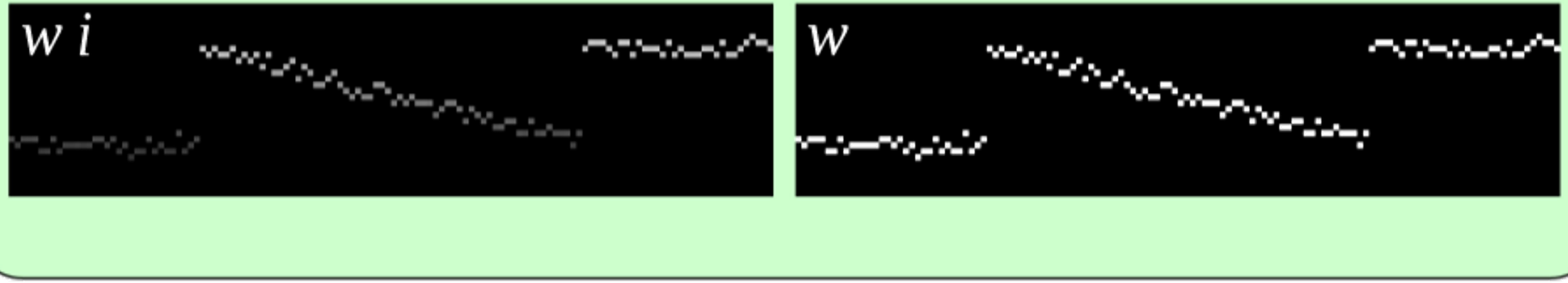
$$\begin{pmatrix} W_{\mathbf{p}}^{\mathrm{bf}} & I_{\mathbf{p}}^{\mathrm{bf}} \\ W_{\mathbf{p}}^{\mathrm{bf}} & \end{pmatrix} = \sum\sum_{(\mathbf{q},\zeta)\in\mathcal{S}\times\mathcal{R}} \text{space-range Gaussian} \begin{pmatrix} I_{\mathbf{q}} \\ 1 \end{pmatrix}$$

Fast Bilateral Filter

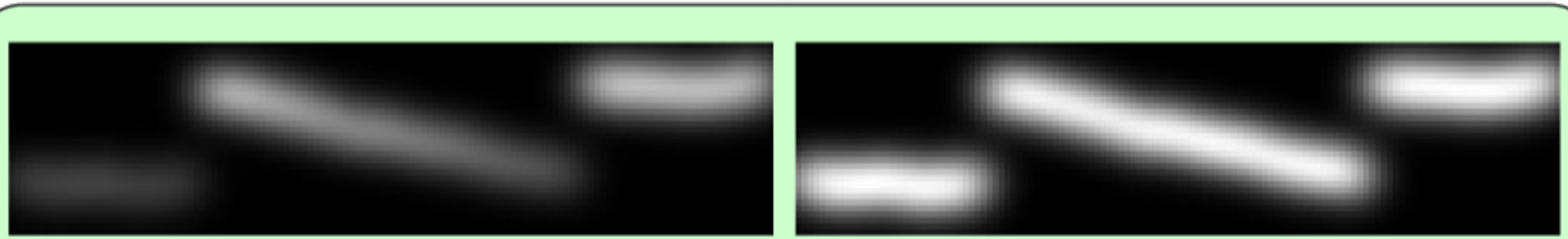
- How to compute values in denominator ?



higher dimensional functions

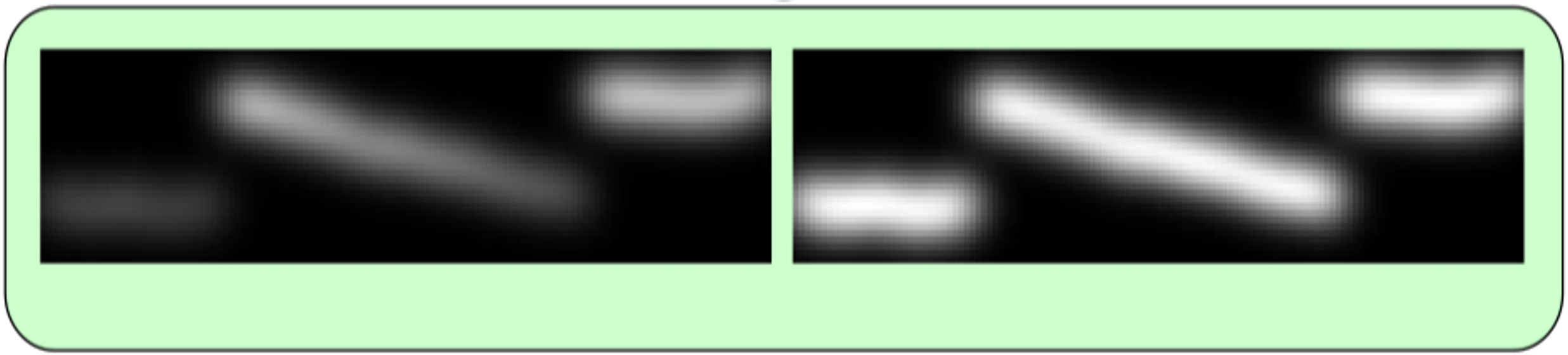


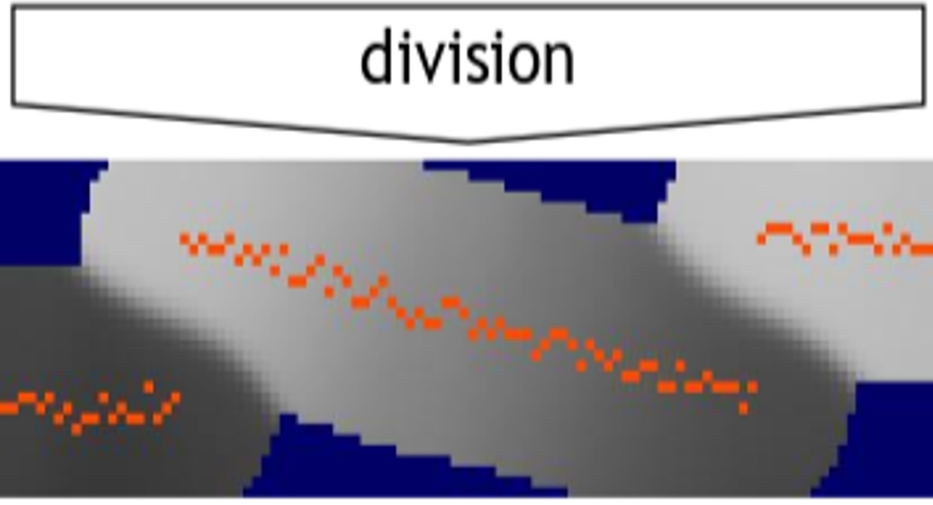
Gaussian convolution



Fast Bilateral Filter

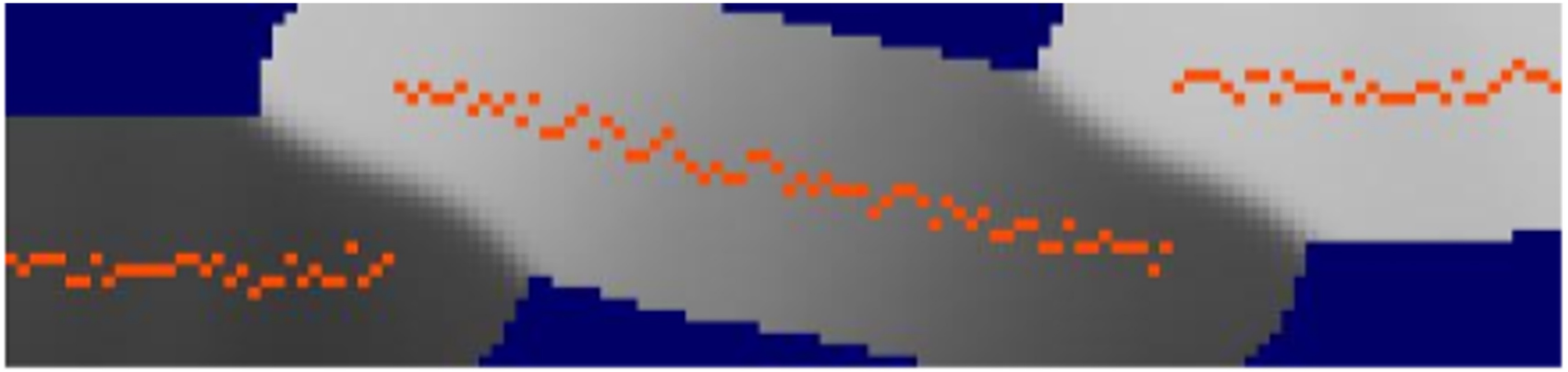
- Divide numerator values by denominator values
  [actually, we only need values at locations (p, I_p) in two images]



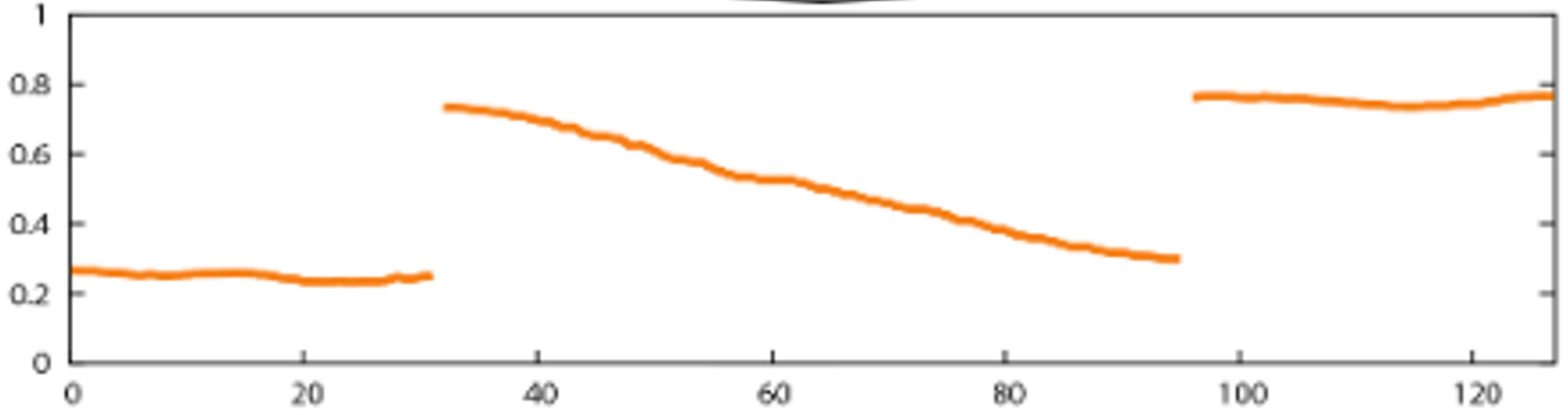


Fast Bilateral Filter

- Update original/1D image to get filtered result
- At each pixel location p, assign filtered values = ratios of values from specific locations (p, I_p)
- They call this "slicing"



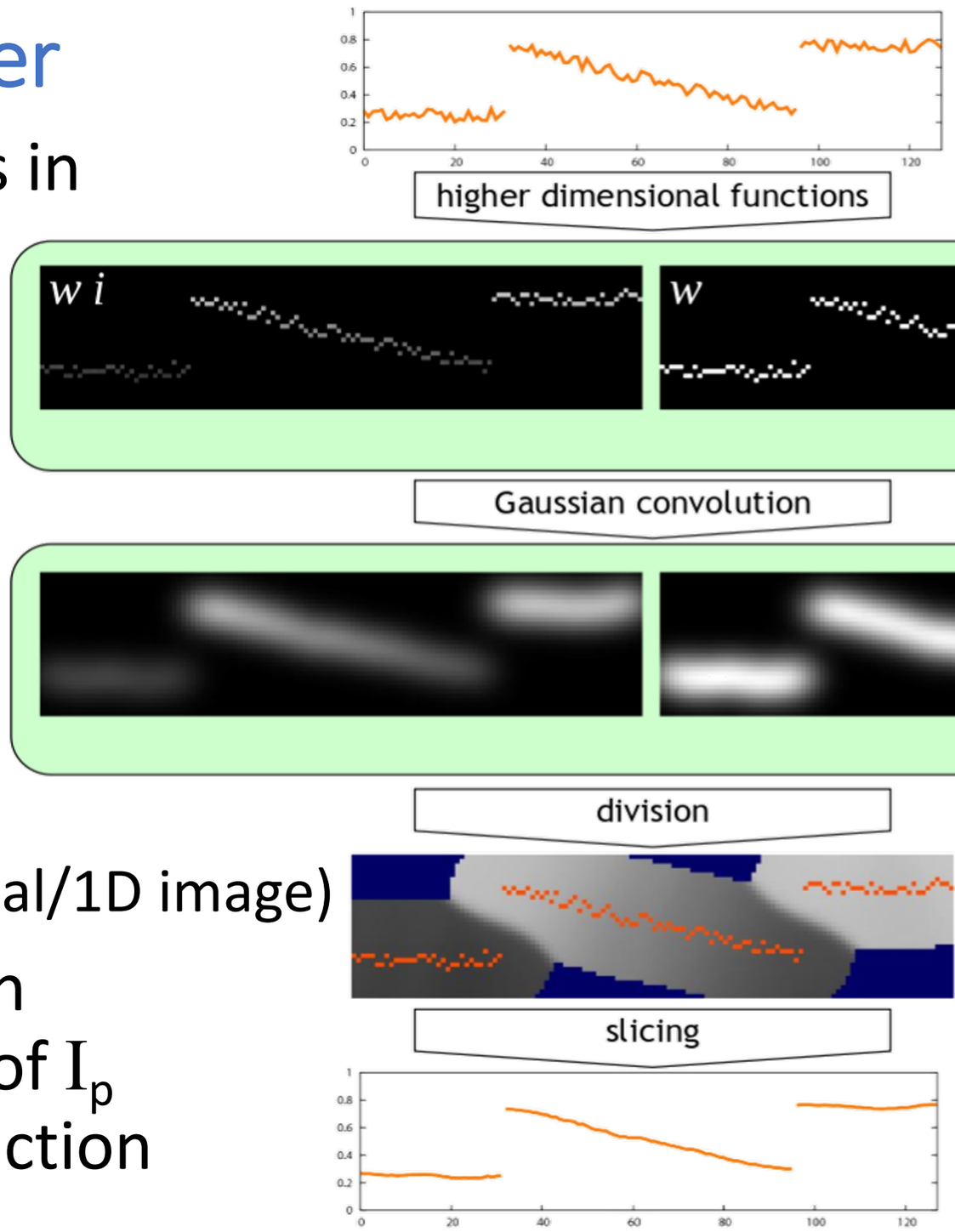
slicing



Fast Bilateral Filter

1) Two convolutions in higher-dim space $\langle p, I_p \rangle$
   - Both separable
   - Time complexity O(matrix size)
2) Pixel-by-pixel division between
   - Time complexity O(# pixels in original/1D image)

- Only approximation is in discretization of I_p during grid construction

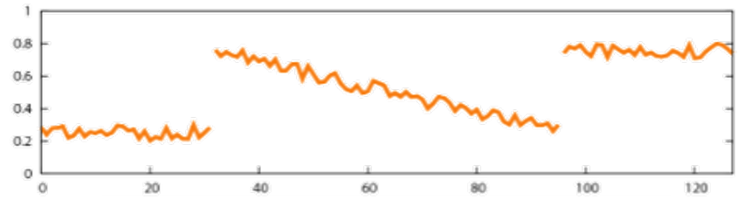


Recap:
- simple operations
- complex space

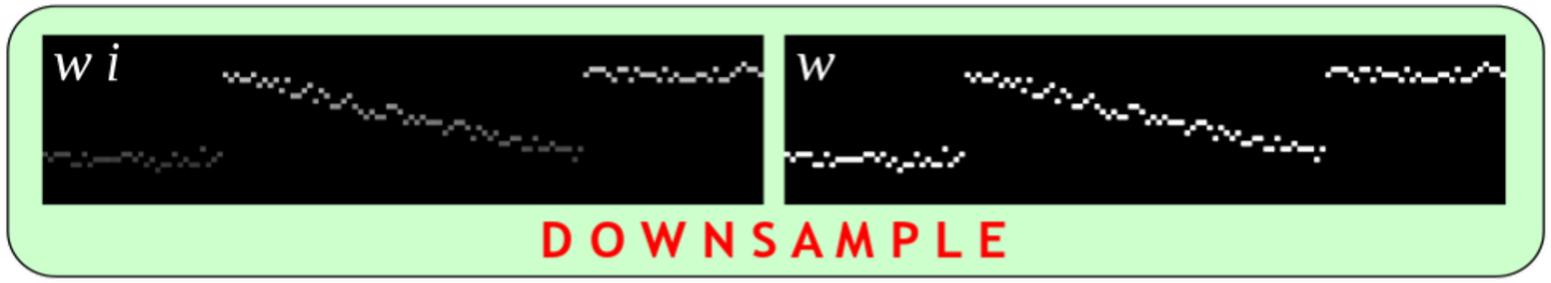
Fast Bilateral Filter

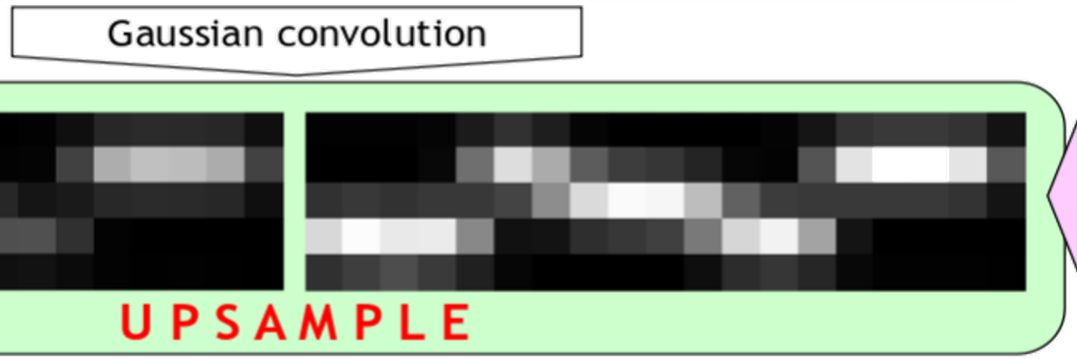
- Make convolutions even faster in higher-dim. space
- Gaussian convolution will blur anyway

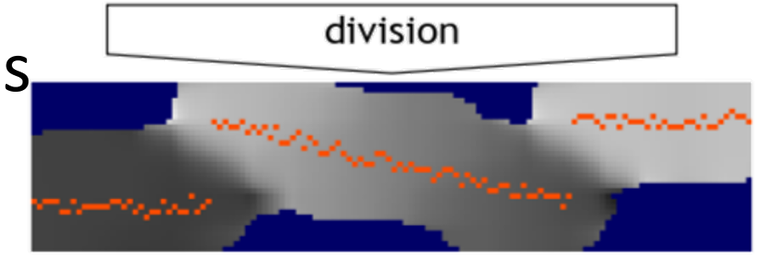
1) Average + downsample (before convolution)
   - Average non-overlapping 2x2 neighborhoods via box filter (separable)
   - Time complexity O(matrix size)
2) Upsample (after convolution; before slicing)

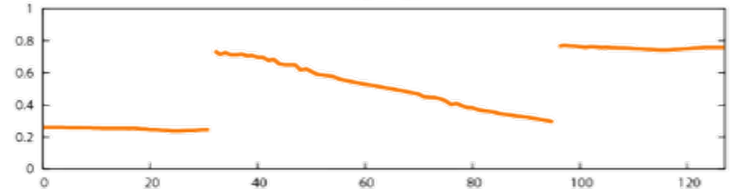


Strategy: downsampled convolution







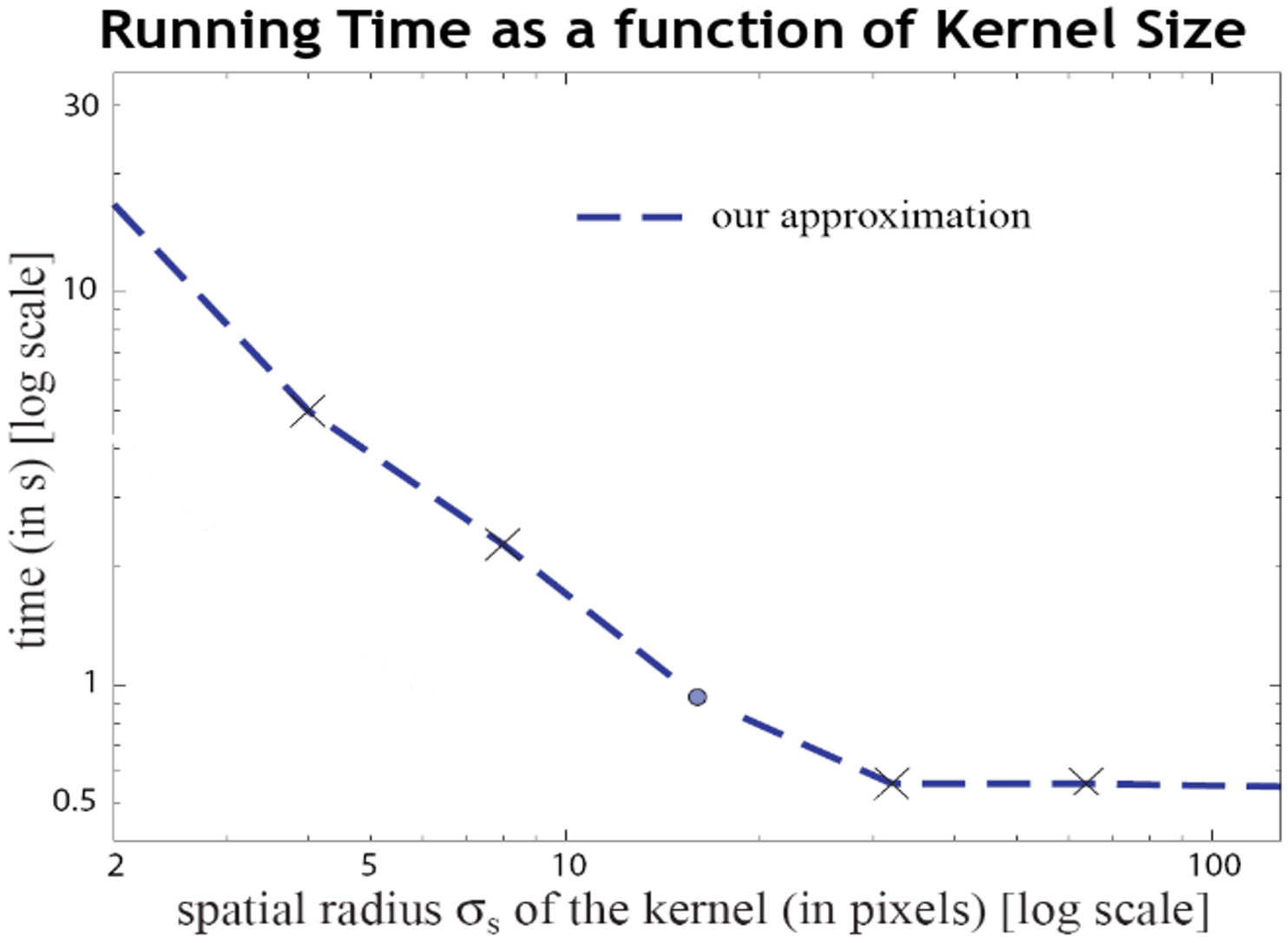


Fast Bilateral Filter

- Effects of downsampling

1) Faster algorithm ← less data to process
2) Some error ← approximate
3) Larger kernel →
   we can afford
   more downsampling
   - So faster algorithm !!

- Typical trade off:
  speed versus accuracy
- Evaluate (upcoming slides)



Fast Bilateral Filter

- 1200 × 600 image with reasonable parameter settings
- Original algorithm → Many minutes; some large neighborhood size
- New algorithm → Few seconds; same neighborhood size
- Evaluate running time: M × N image, P × Q neighborhood
- Original algorithm: MN PQ 4 (find exp weights; multiply twice; add)
- New algorithm:
  - In 3D, average + downsample image (MNI → M/a N/b I/c): MNI
    In 3D, average + downsample neighborhood/kernel (PQJ→ P/a Q/b J/c): PQJ
  - Two Gauss convolutions (separable) in 3D: 2 (M/a N/b I/c) (P/a + Q/b + J/c)
    - If a = b = c, then operations = 2 (M N I) (P + Q + J) / a^4

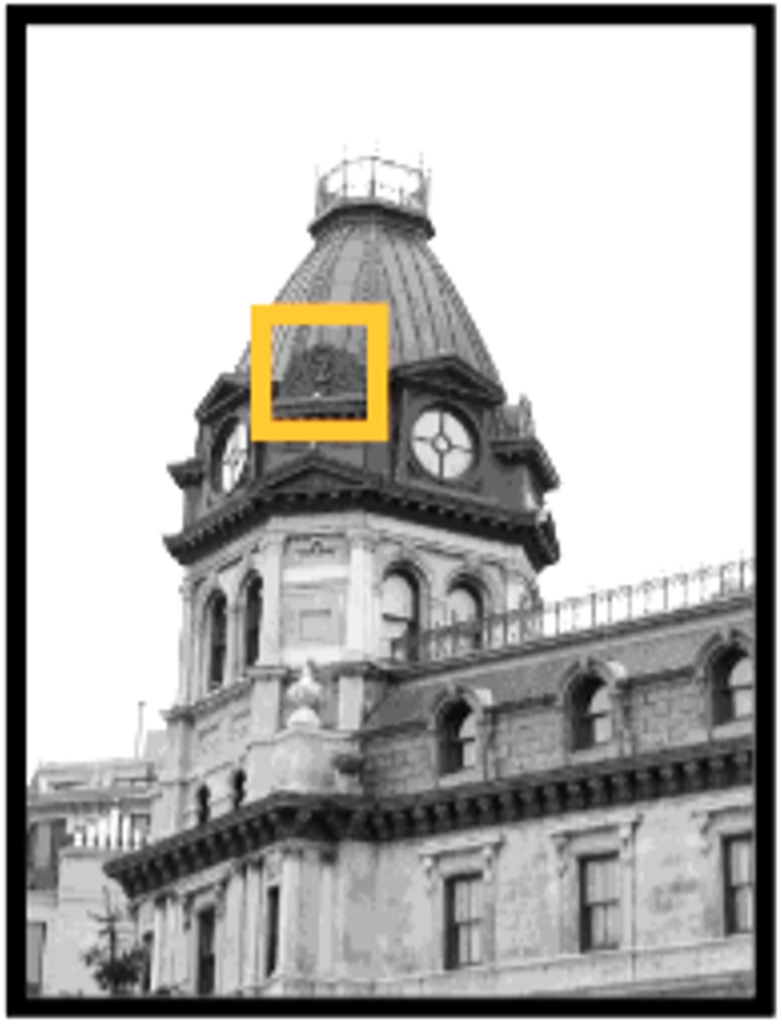
1) 3D upsampling: MN 8 (tri-linear interpolation at (p, Ip) locations only)
2) Division and update: MN

Fast Bilateral Filter

input

exact BF

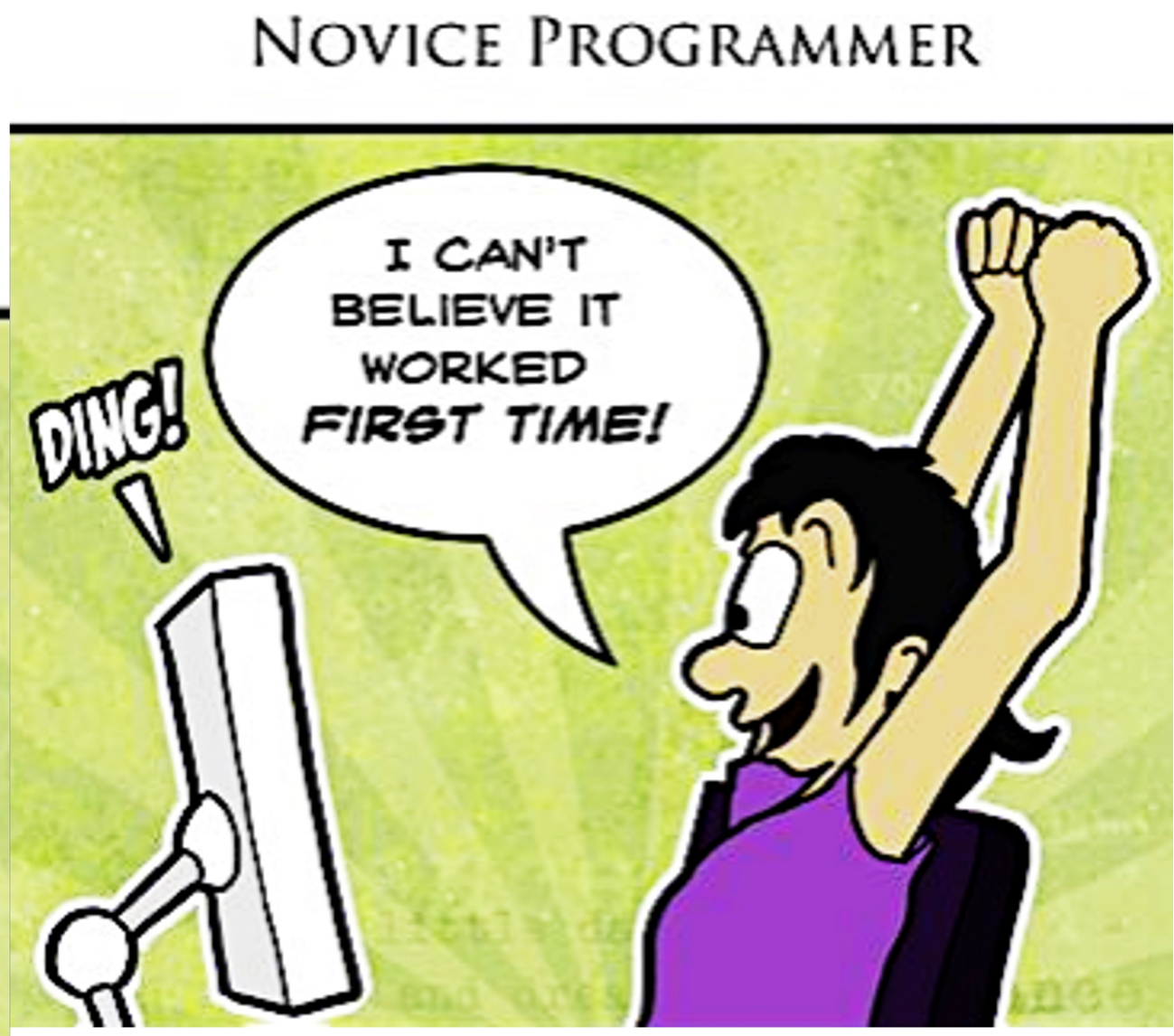
our result



1200 × 1600

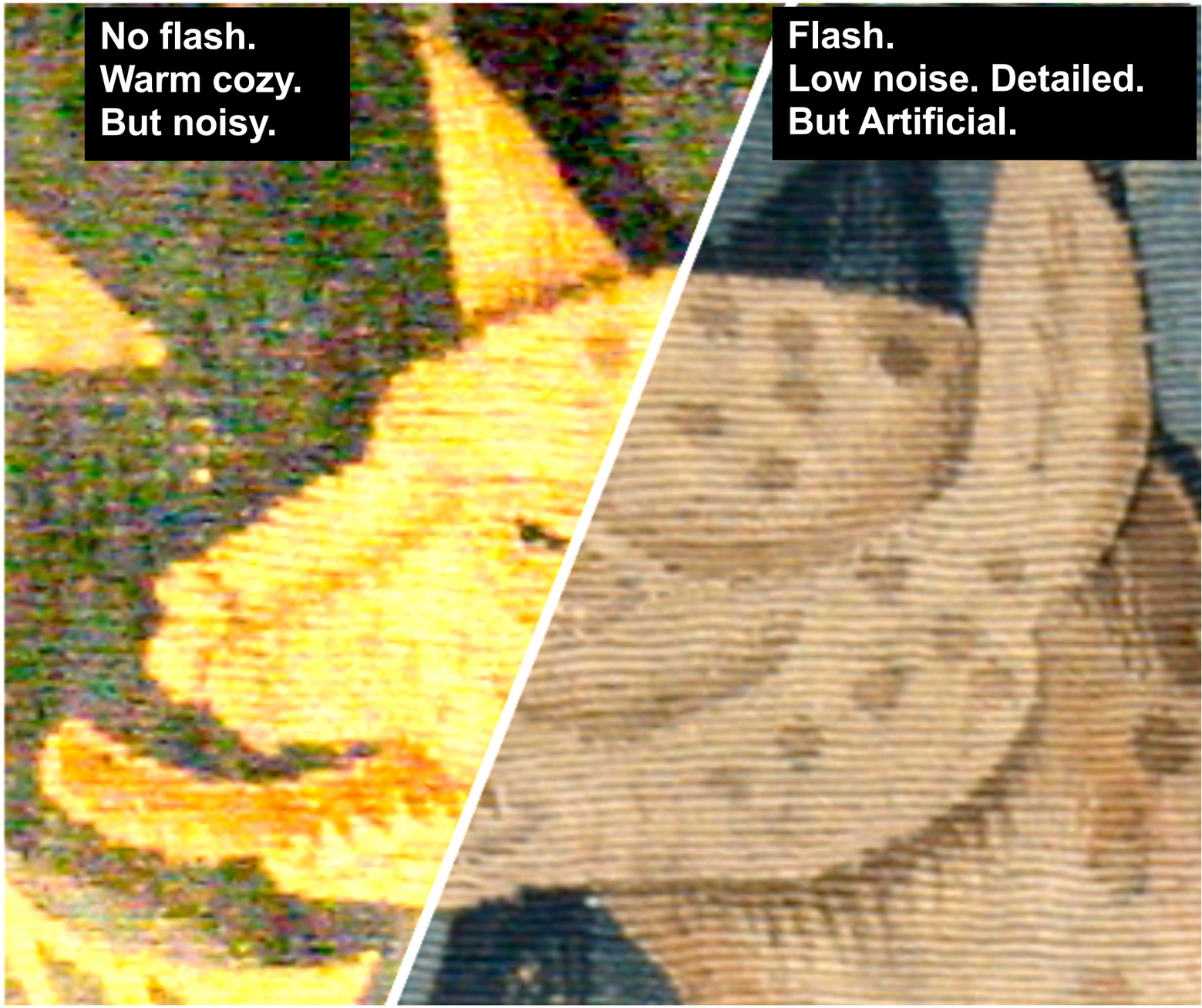
difference
with exact
computation
(intensities in [0:1])

Fast Bilateral Filter



Bilateral Filter Applications: Cross/Joint Bilateral Filter

- Application:
  Merging camera images with & without flash
- Images of an artistic carpet



Bilateral Filter Applications: Cross/Joint Bilateral Filter

(a) flash picture

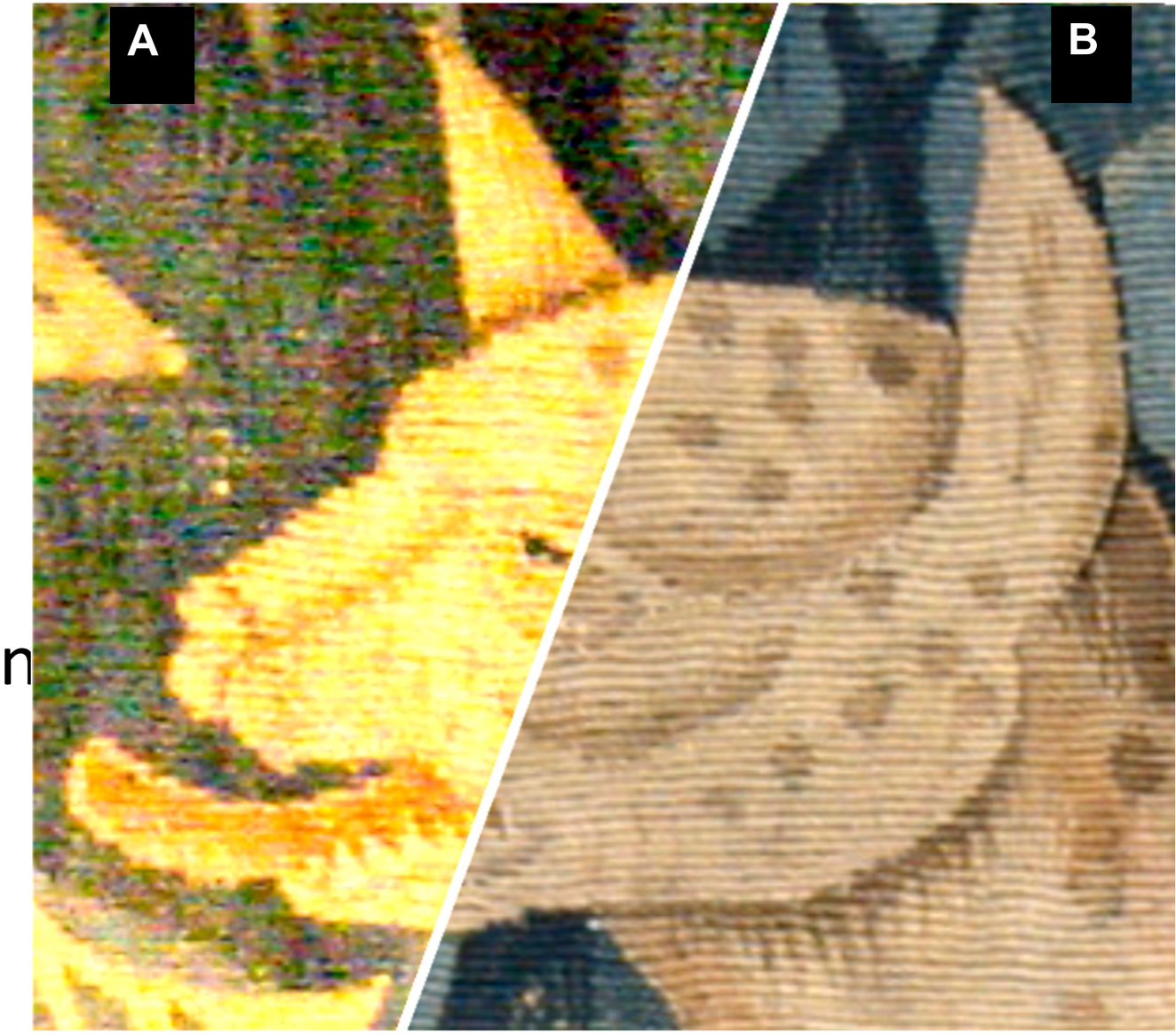
(b) no-flash picture

(c) output of [Petschnigg *et al.*, 2004]

Bilateral Filter Applications: Cross/Joint Bilateral Filter

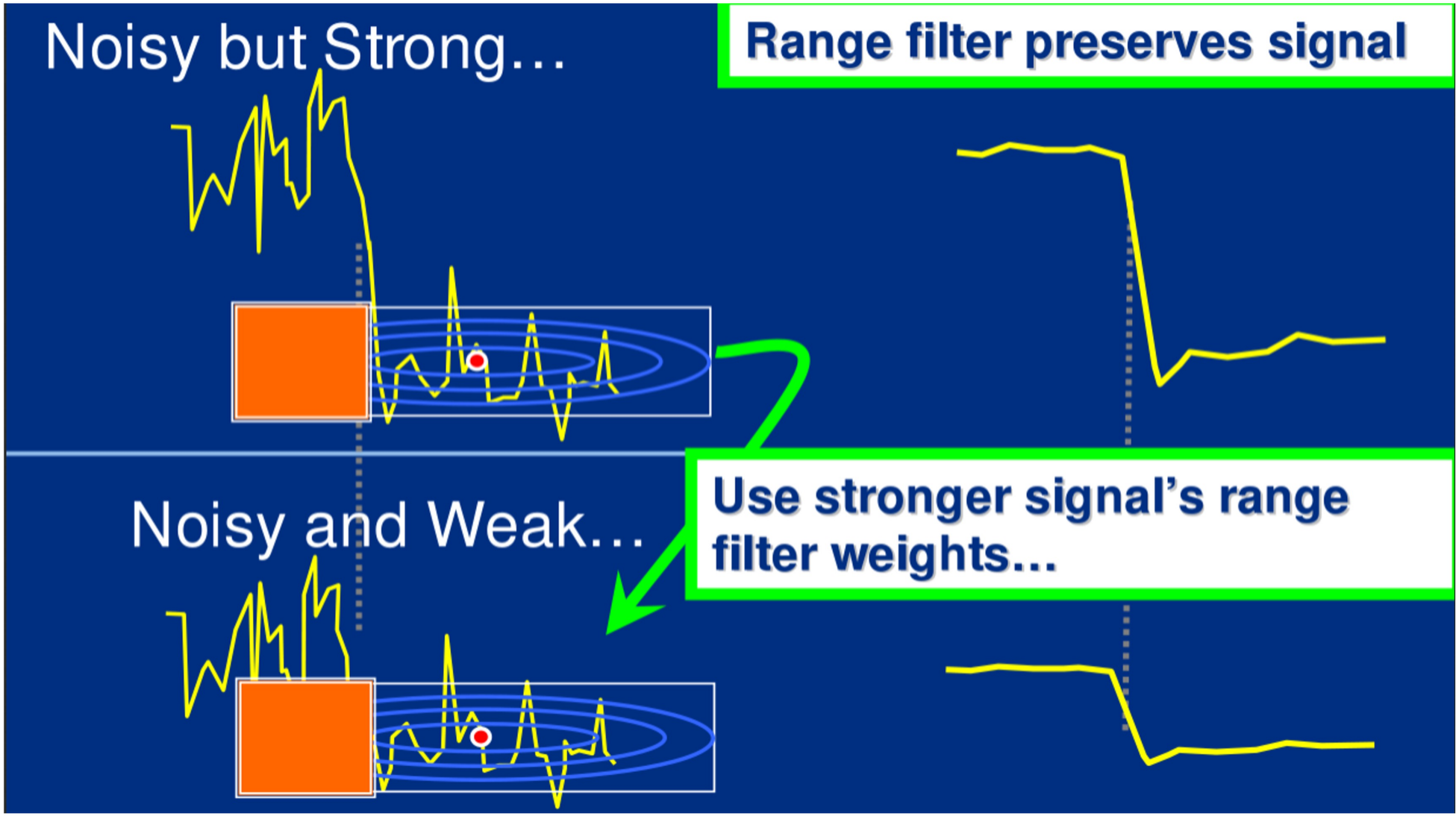
- 2 kinds of weights
- Spatial Gaussian and Intensity Gaussian
- A = image without flash
- B = image with flash
- Smooth image A with:

1) Weights based on spatial location
   → From A or B (doesn't matter)
2) Weights based on intensity
   → From intensities in B



Bilateral Filter Applications: Cross/Joint Bilateral Filter

- Enhances ability to find weak details in noisy image A (without flash)
- Intensities in image B (with flash) inform which intensities to average in image A



people.csail.mit.edu/sparis/bf_course/slides/08_applications_advanced.pdf

Bilateral Filter Applications: C

- (d) = | (b) – (c) |
- Difference larger near edges
- Edges better retained in (c) compared to (b)

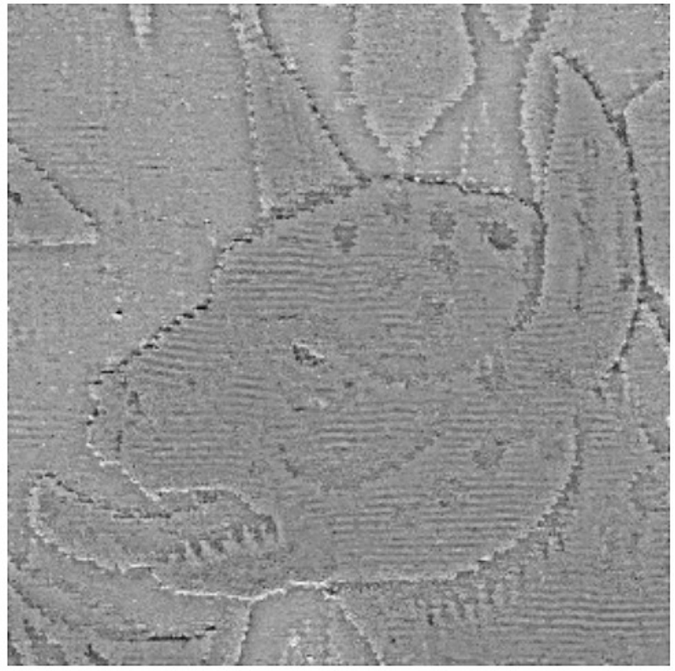
- Joint bilateral filter is denoising
- Can we also some kind of "detail enhancement" in this process ?
  - See "detail transfer" next …

research.microsoft.com/en-us/um/people/hoppe/flash.pdf

(a) No-Flash Flash

(b) Denoised via Bilateral Filter

(c) Denoised via Joint Bilateral Filter

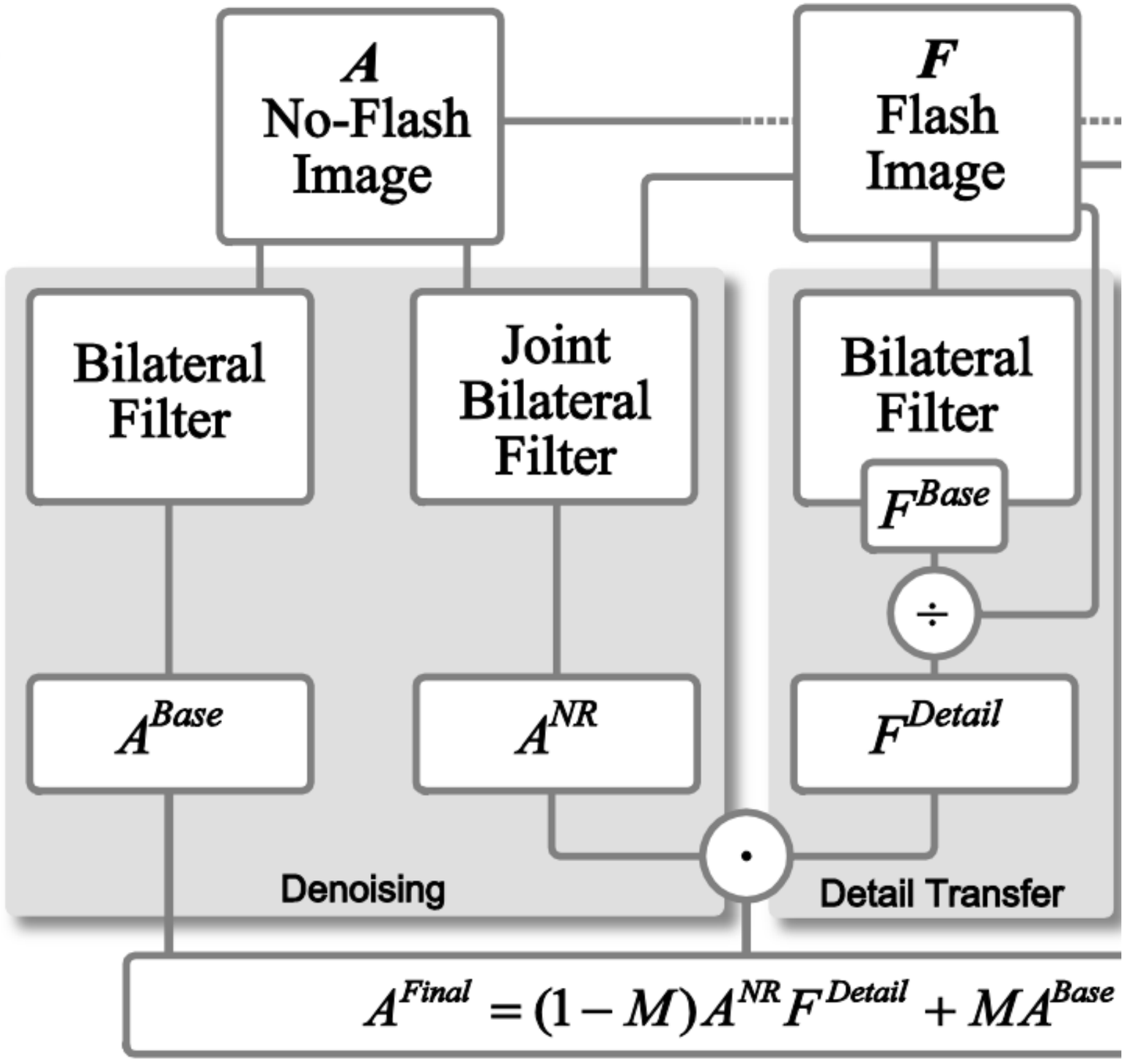


(d) Difference

Bilateral Filter Applicatio

- "Detail transfer"
- Consider M=0, for now

1) Use image-with-flash F to compute "detail"-factor map
   $F^{detail} = F \,./\, F^{base}$



Bilateral Filter Applications: Cross/Joint Bilateral Filter

- "Detail transfer": Detail-factor map

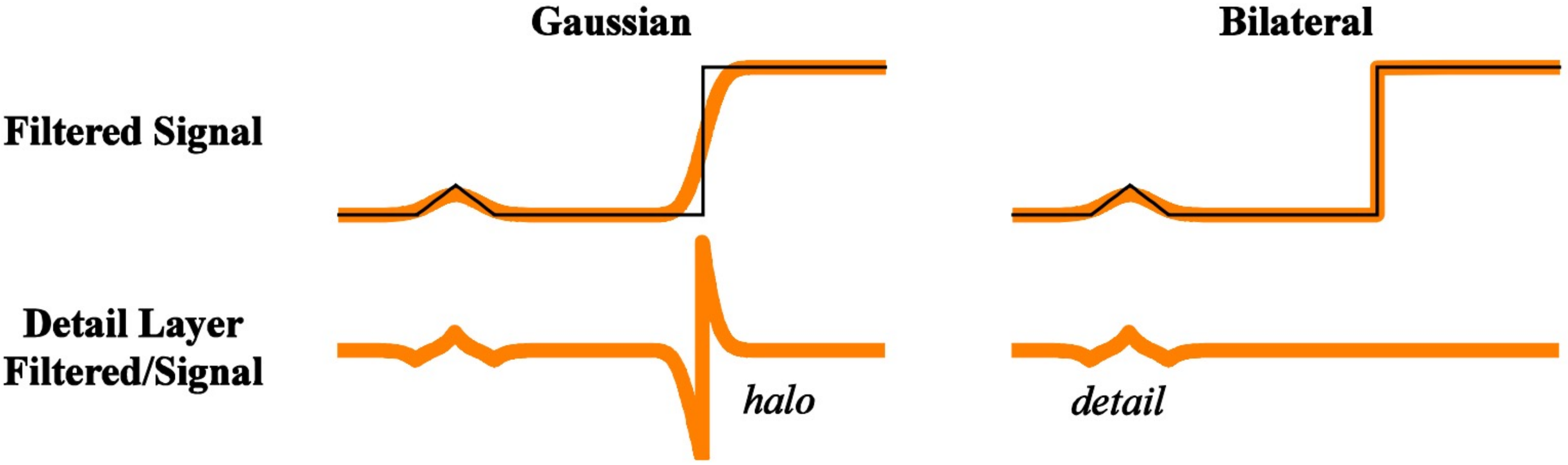


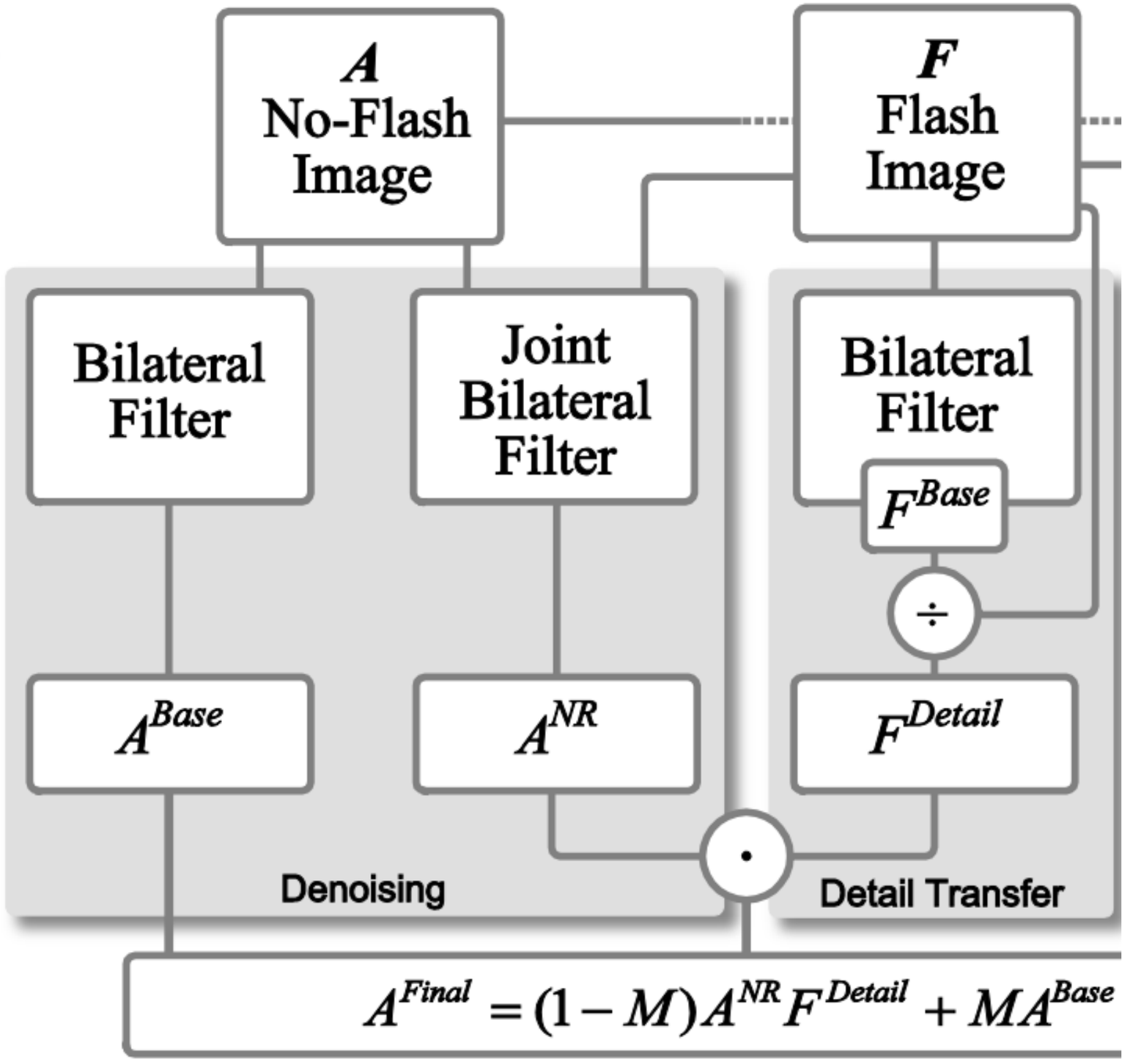
Figure 5: (left) A Gaussian low-pass filter blurs across all edges and will therefore create strong peaks and valleys in the detail image that cause halos. (right) The bilateral filter does not smooth across strong edges and thereby reduces halos, while still capturing detail.

Bilateral Filter Applicatio

- "Detail transfer"
- Consider M=0, for now

1) Use image-with-flash F to compute "detail"-factor map
   $F^{detail} := F \,./\, F^{base}$

2) Apply ratios to cross-bilateral-filtered img
   $(F \,./\, F^{base}) \,.*\, A^{NoiseReduced}$



Bilateral Filter Applications: Cross/Joint Bilateral Filter

- Before detail transfer
- A^NoiseReduced

(c) Denoised via Joint Bilateral Filter

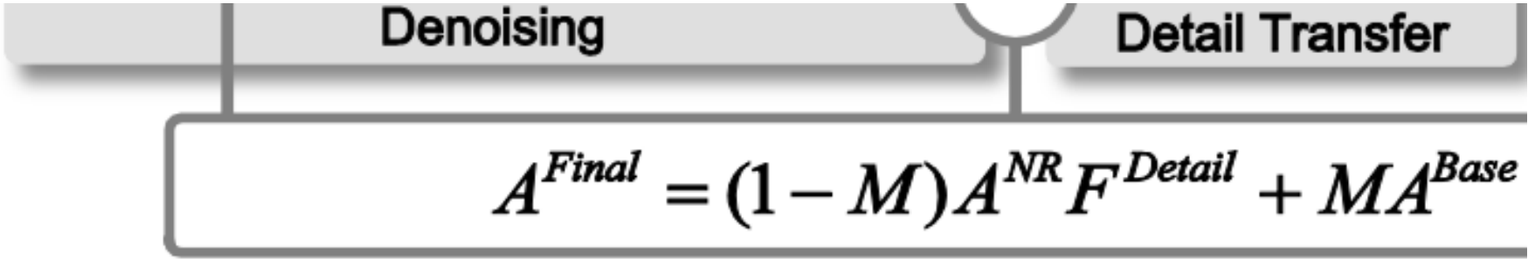
Bilateral Filter Applications: Cross/Joint Bilateral Filter

(e) Detail Layer

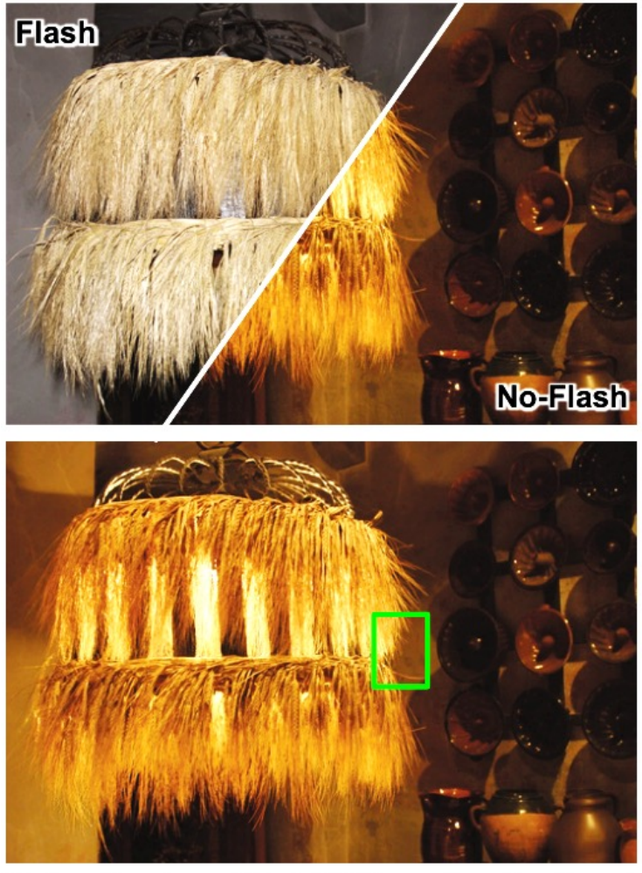
(f) Detail Transfer

Bilateral Filter Applications: Cross/Joint Bilateral Filter

- "Detail transfer": Role of "M"
- Detail estimate F^{Detail} is poor in regions of shadows and specular reflections caused by flash, because colors tend to black (de-saturation) or white (saturation)
  - M = binary mask indicating such regions
- Apply detail transfer ONLY in remaining regions
- For regions of desaturation & saturation, don't use result from cross-bilateral-filter at all, but use result of simple bilateral filter applied to no-flash image = A^{base}



Bilateral Filter Applications: Cross/Joint Bilateral Filter



Orig. (top) Detail Transfer (bottom)

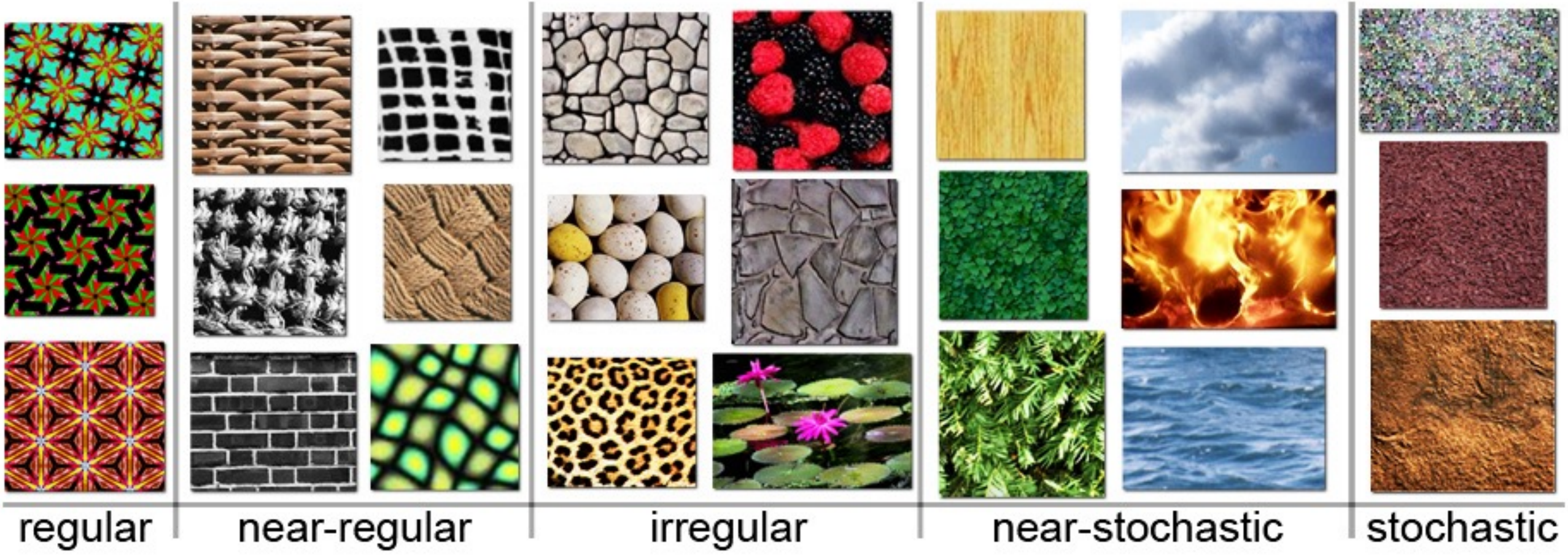
Flash

No-Flash

Detail Transfer with Denoising

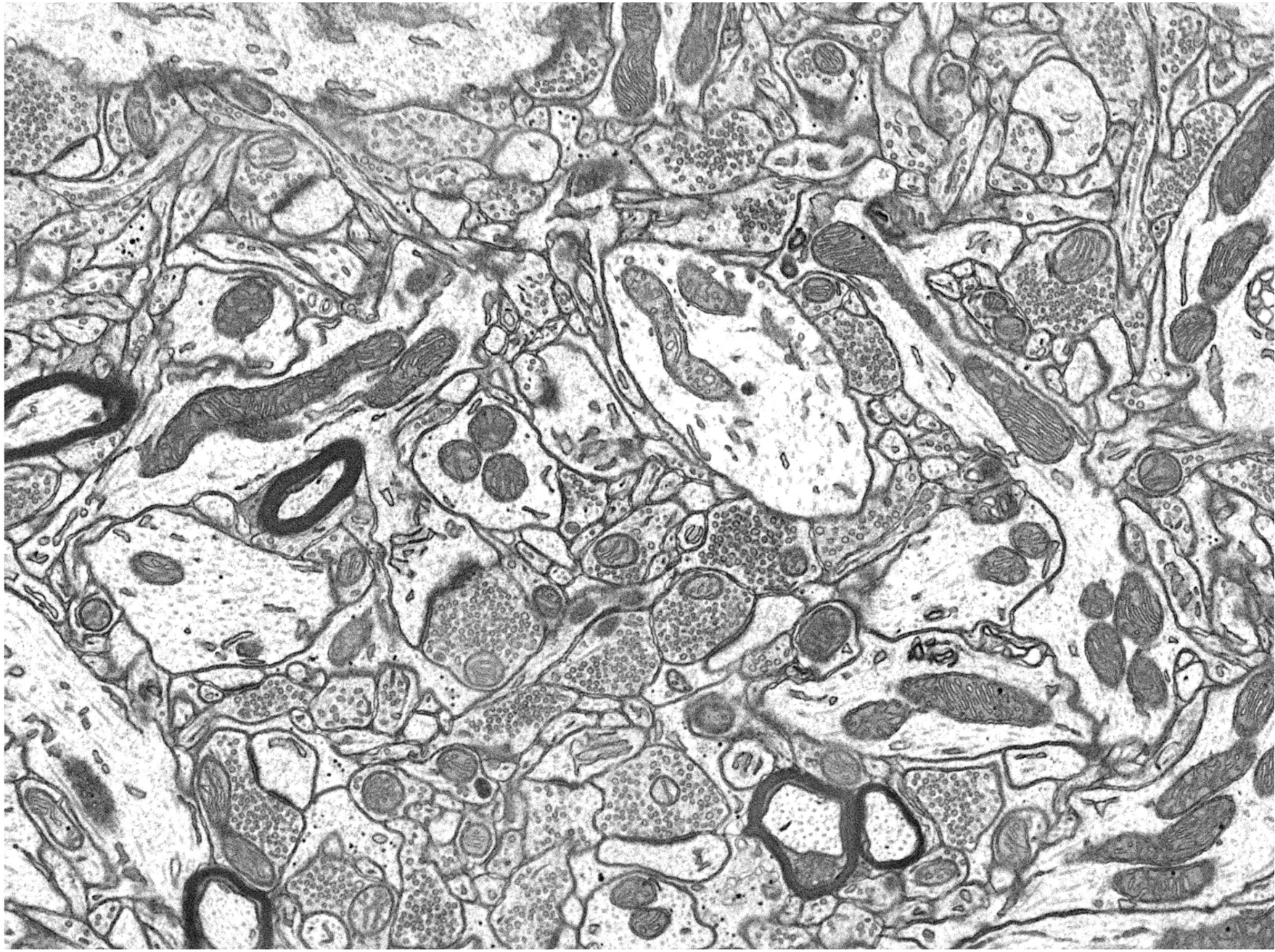
Images With Texture

- Image texture = repetitive pattern/structure perceived in an image, based on spatial arrangement of colors or intensities



Images With Texture

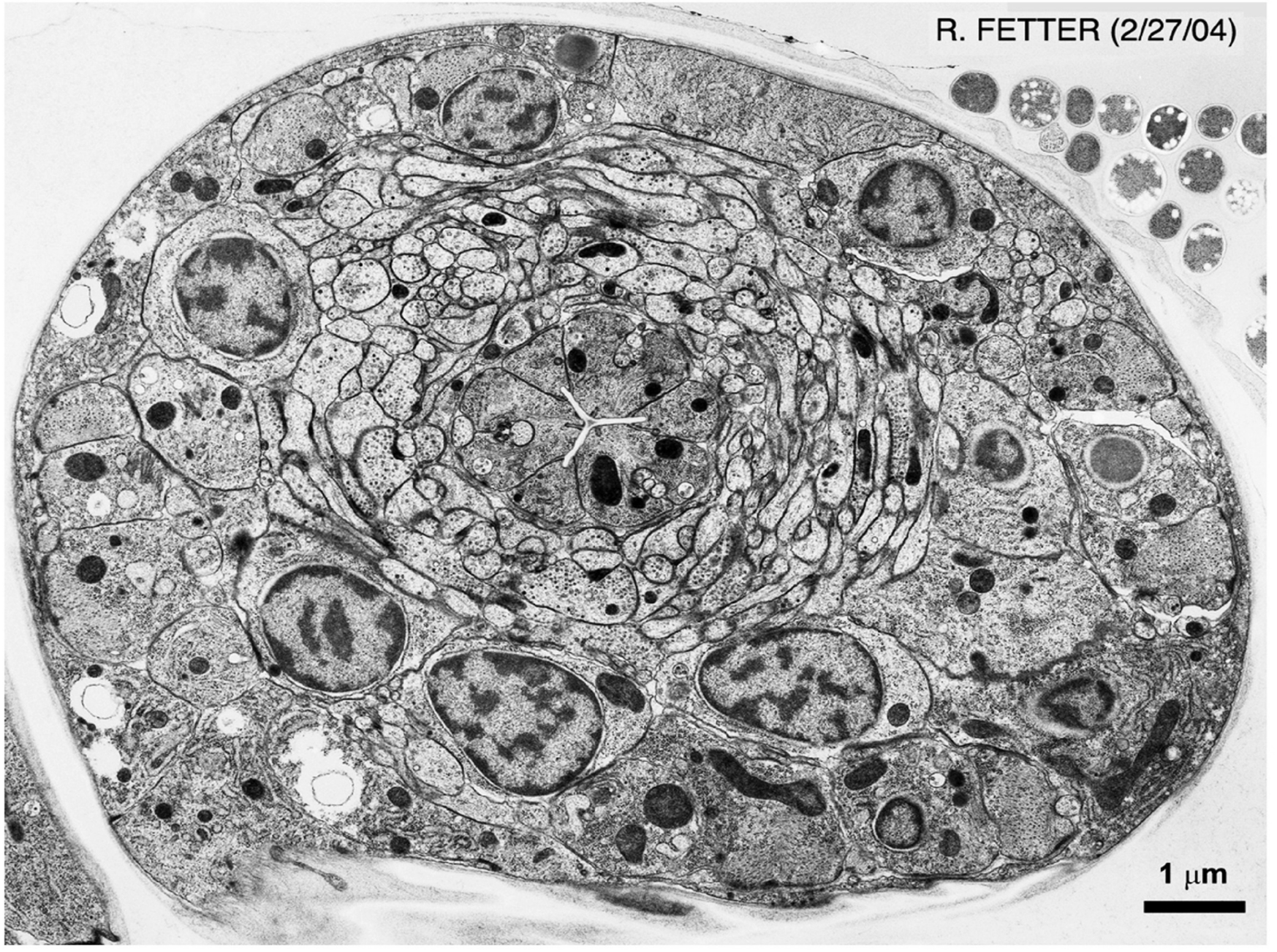
- Neuronal fibers in electron microscopy



cmp.felk.cvut.cz/~kybic/research.html

Images With Texture

- TEM section through an embryo fixed using HPF (Rick Fetter and Cori Bargmann)



Images With Texture

- Retinal Veins

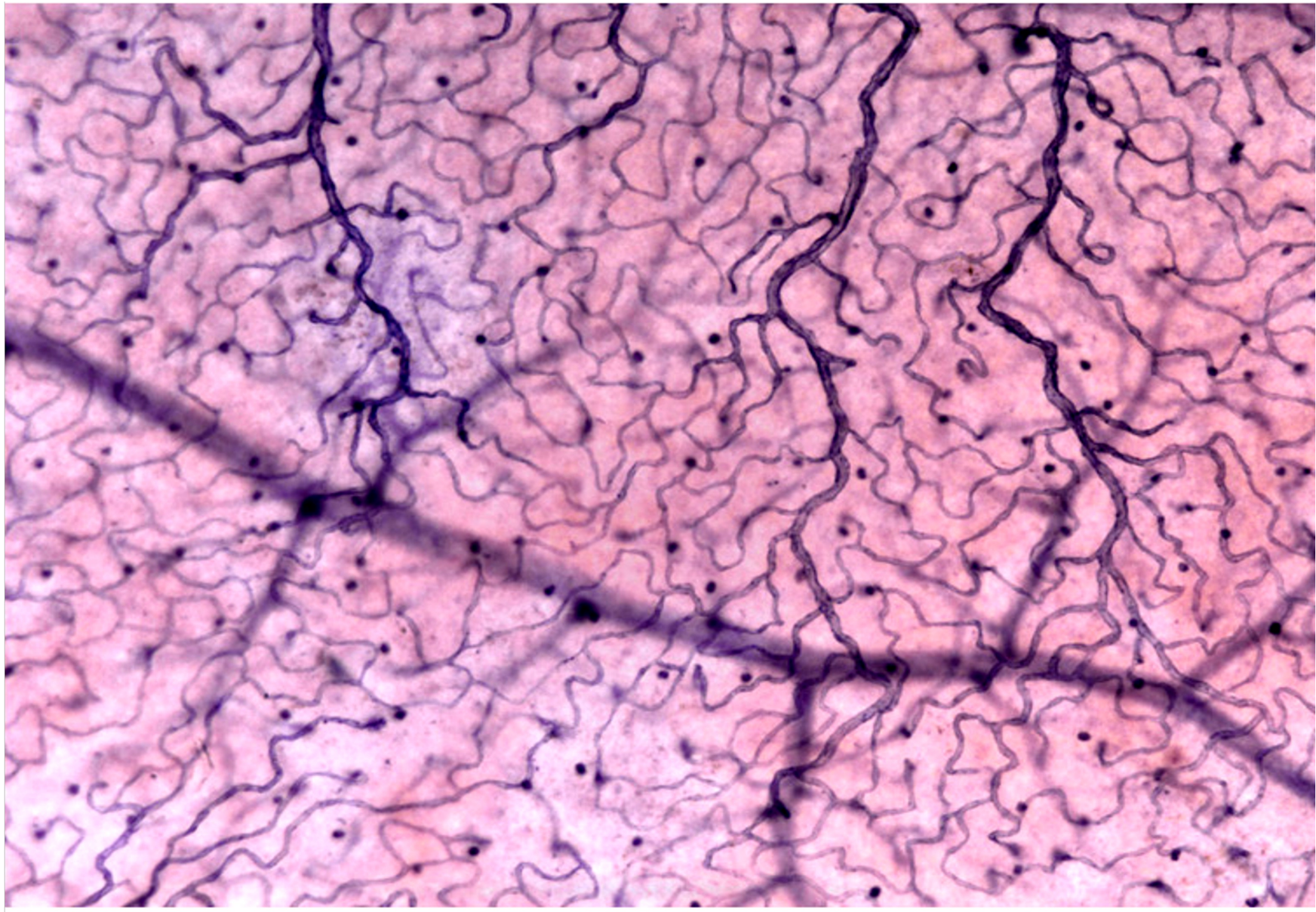
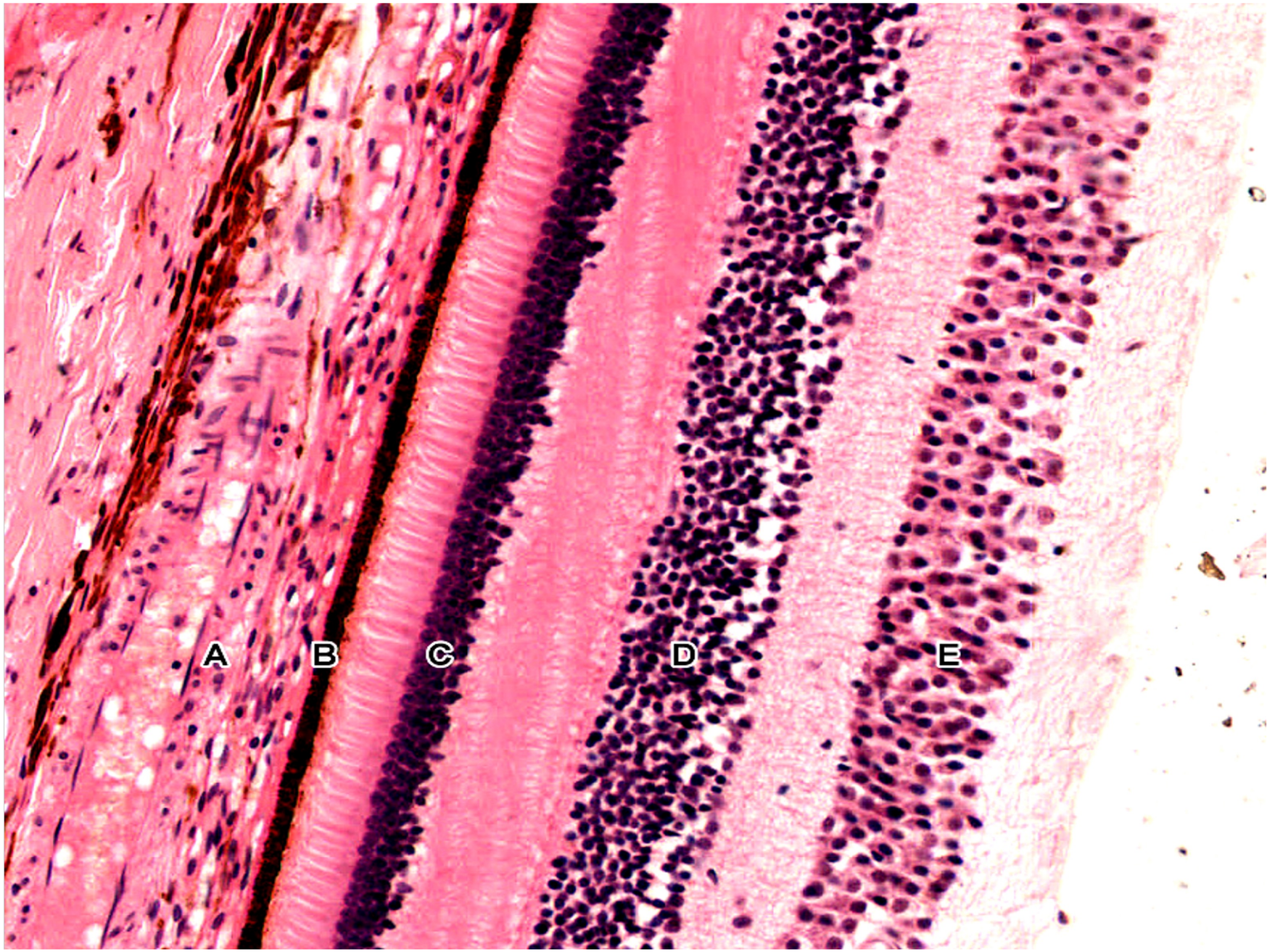


Fig. 18b. Flatmount view of a rat retina stained with NADPH-diaphorase at the level of focus of a major vein and venules. (Courtesy of Toby Holmes, Moran Eye Center).

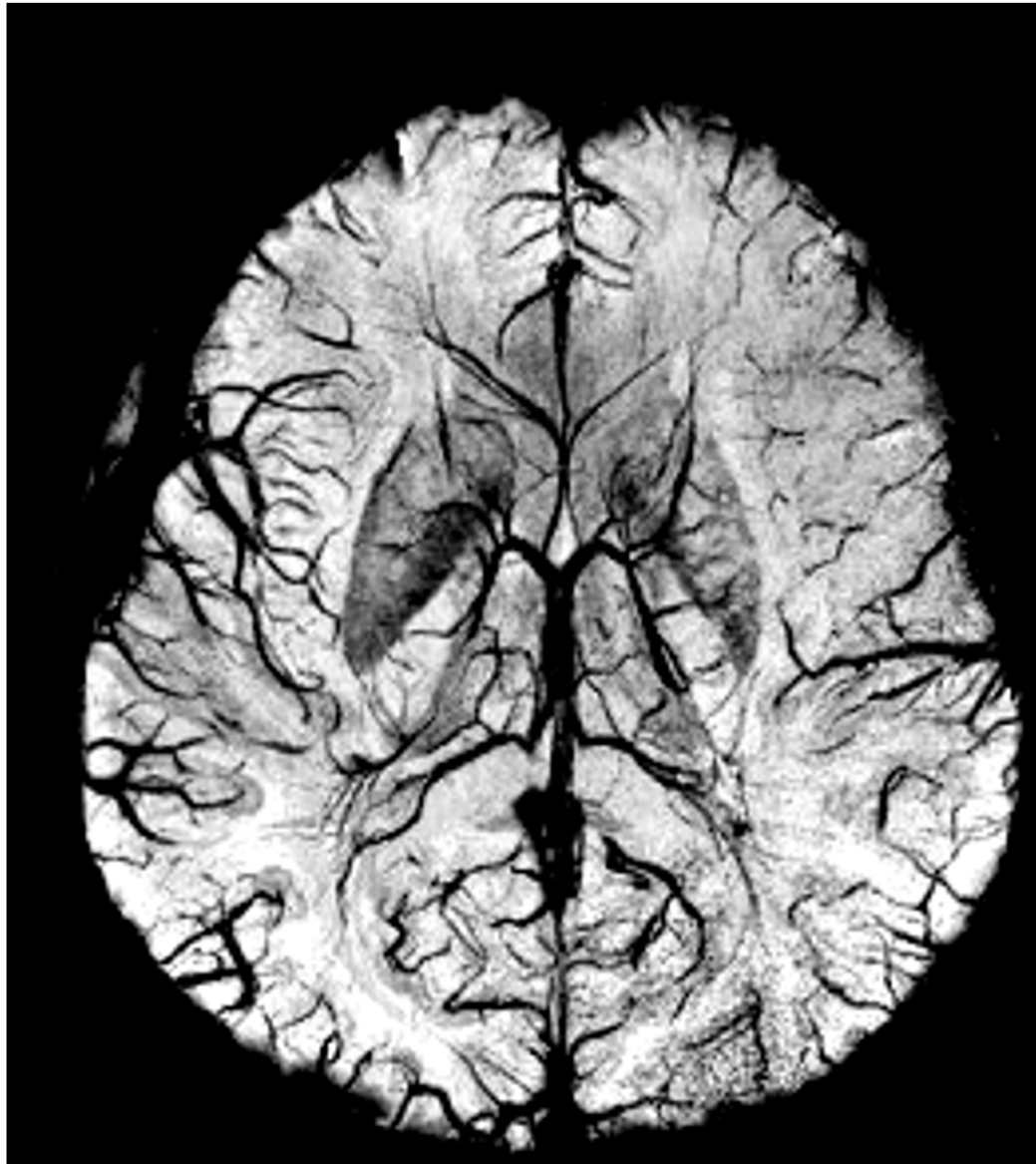
Images With Texture

- Retinal cells



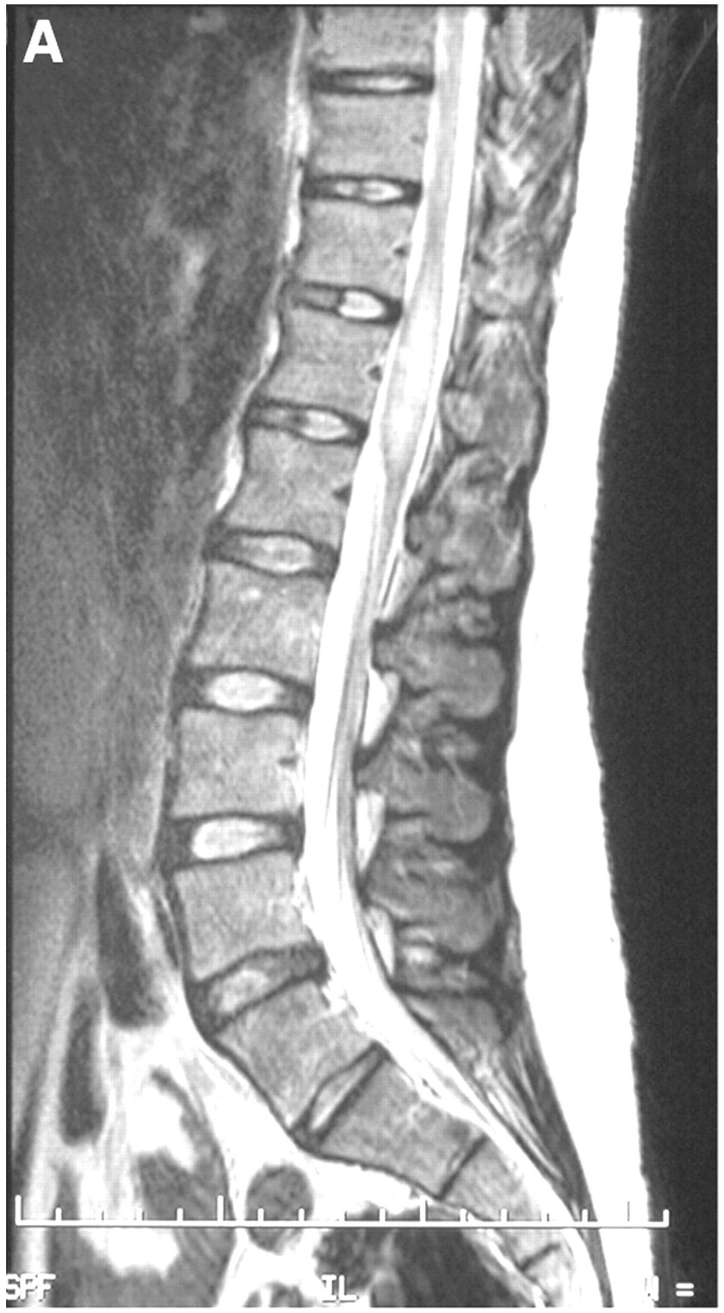
Images With Texture

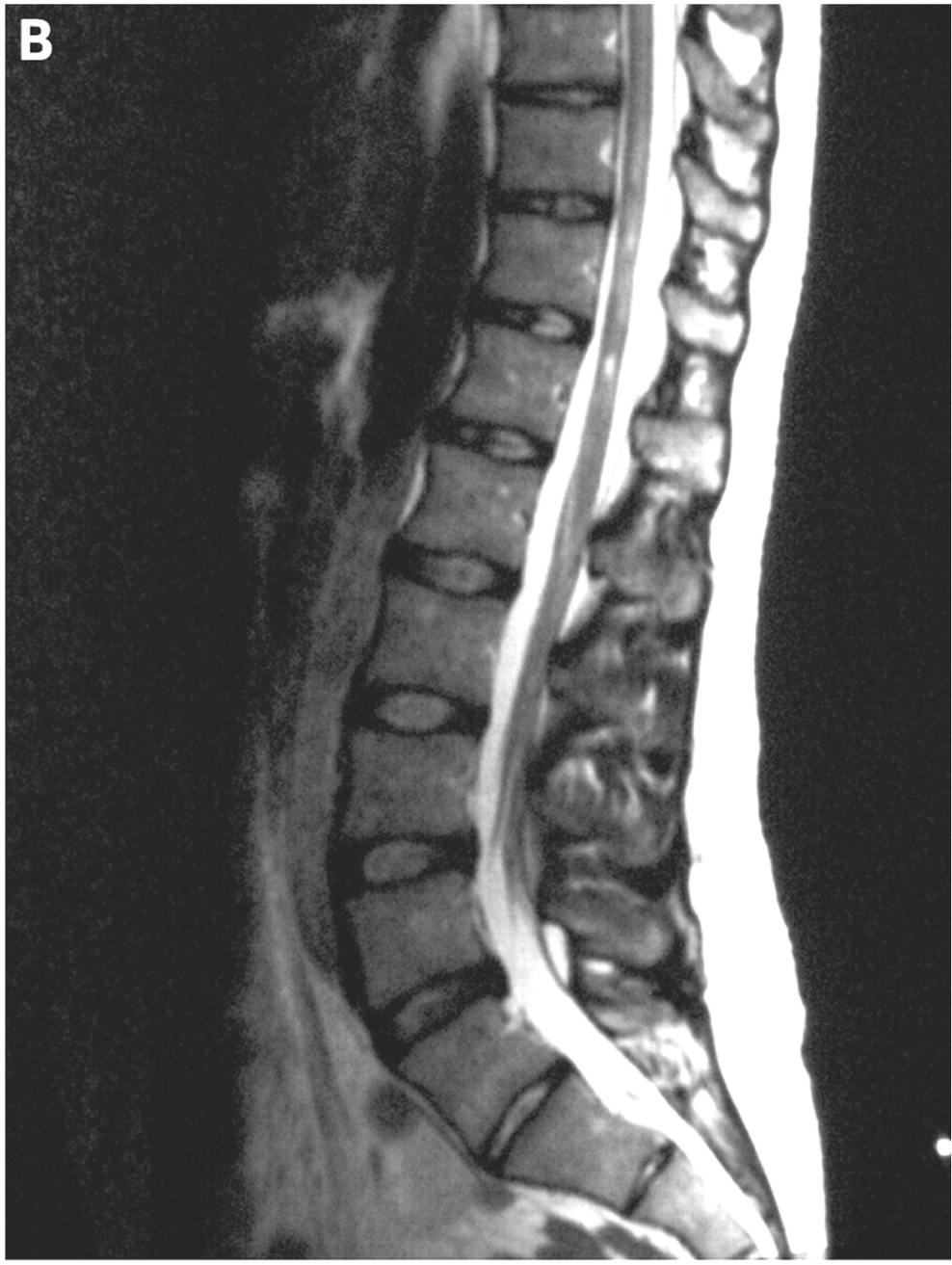
- Head MRI



Images With Texture

- Spine





Texture Synthesis

- “Grows” texture, pixel by pixel, outwards from an initial seed texture patch

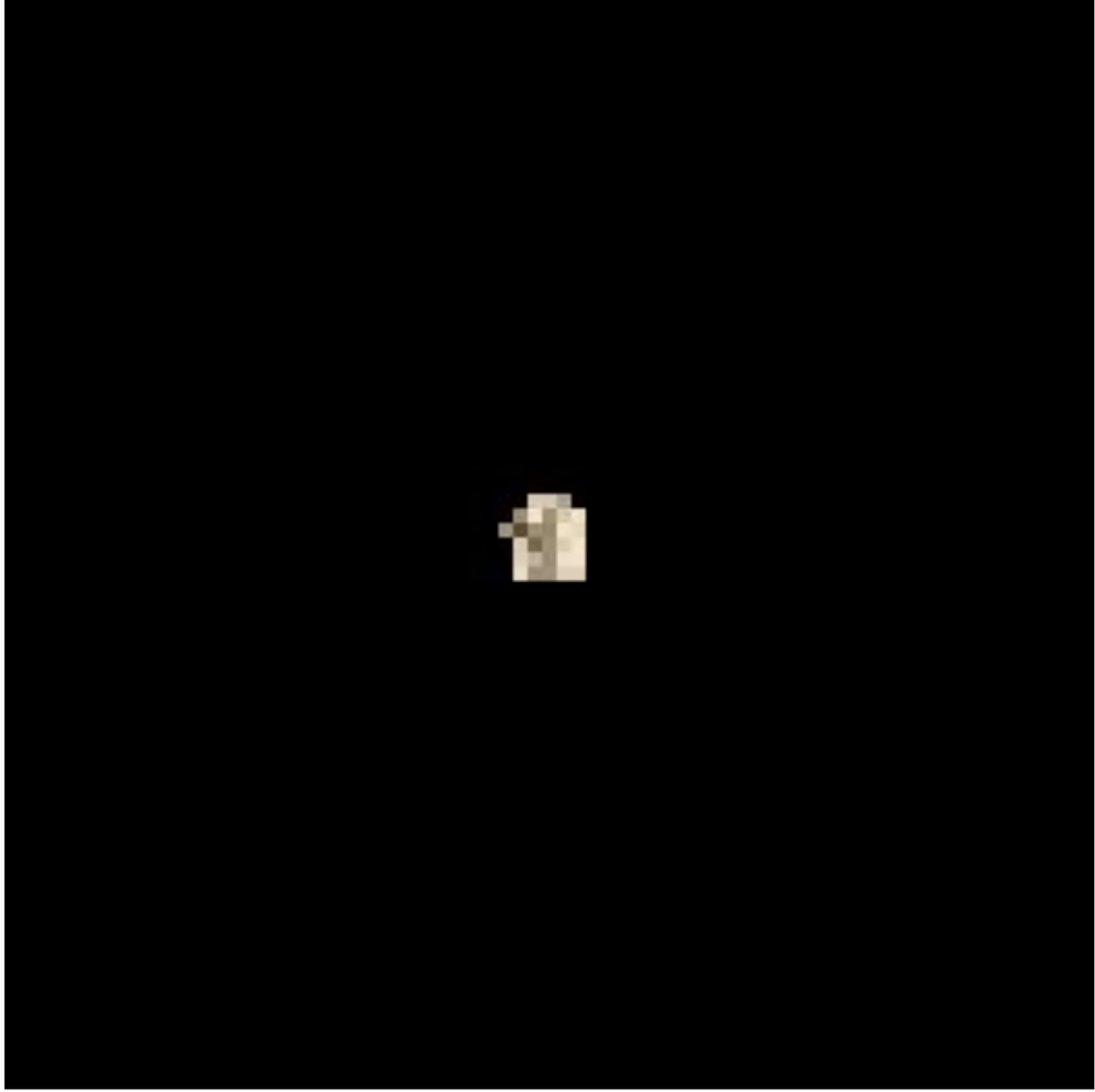
IEEE International Conference on Computer Vision, Corfu, Greece, September 1999

Texture Synthesis by Non-parametric Sampling

Alexei A. Efros and Thomas K. Leung
Computer Science Division
University of California, Berkeley
Berkeley, CA 94720-1776, U.S.A.
{efros,leungt}@cs.berkeley.edu

Texture Synthesis

- Given:
  sample texture patch
- Goal:
  grow it into a larger image



Texture Synthesis

- Similar to modeling sentences in a language
- Shannon [A Mathematical Theory of Communication, 1948] describes a way to generate English-sounding written text using N-grams
- N-gram := sequence of N consecutive letters
- Learn probability of N+1th letter by constructing histograms from training data of a large corpus of text
- Generate a new sentence by:

1) Starting from a seed sequence of a few letters
2) Generating next letter by drawing randomly from **conditional distribution** of letter given previous N–1 letters
3) Repeat

Texture Synthesis

- To find the conditional distribution, use a nearest-neighbor strategy

1) Choose a distance metric *d(.,.)* between neighborhoods *w*
   - e.g., Gaussian-weighted sum-of-squared differences
2) Around pixel *p*, for a given (incomplete) neighborhood *w(p)*, find nearest such neighborhood among all neighborhoods *w*

$$\omega_{best} = argmin_{\omega}\, d(\omega(p), \omega)$$

3) Find all such neighborhoods *w* with distance within, say, 1.1*w_{best}

$$d_(\omega(p), \omega) < (1+\epsilon) d(\omega(p), \omega_{best})$$

4) For all such neighborhoods, get distribution of their center pixels
5) Sample a center-pixel value randomly from this distribution
   - Uniformly, or weighted by distances

Texture Synthes

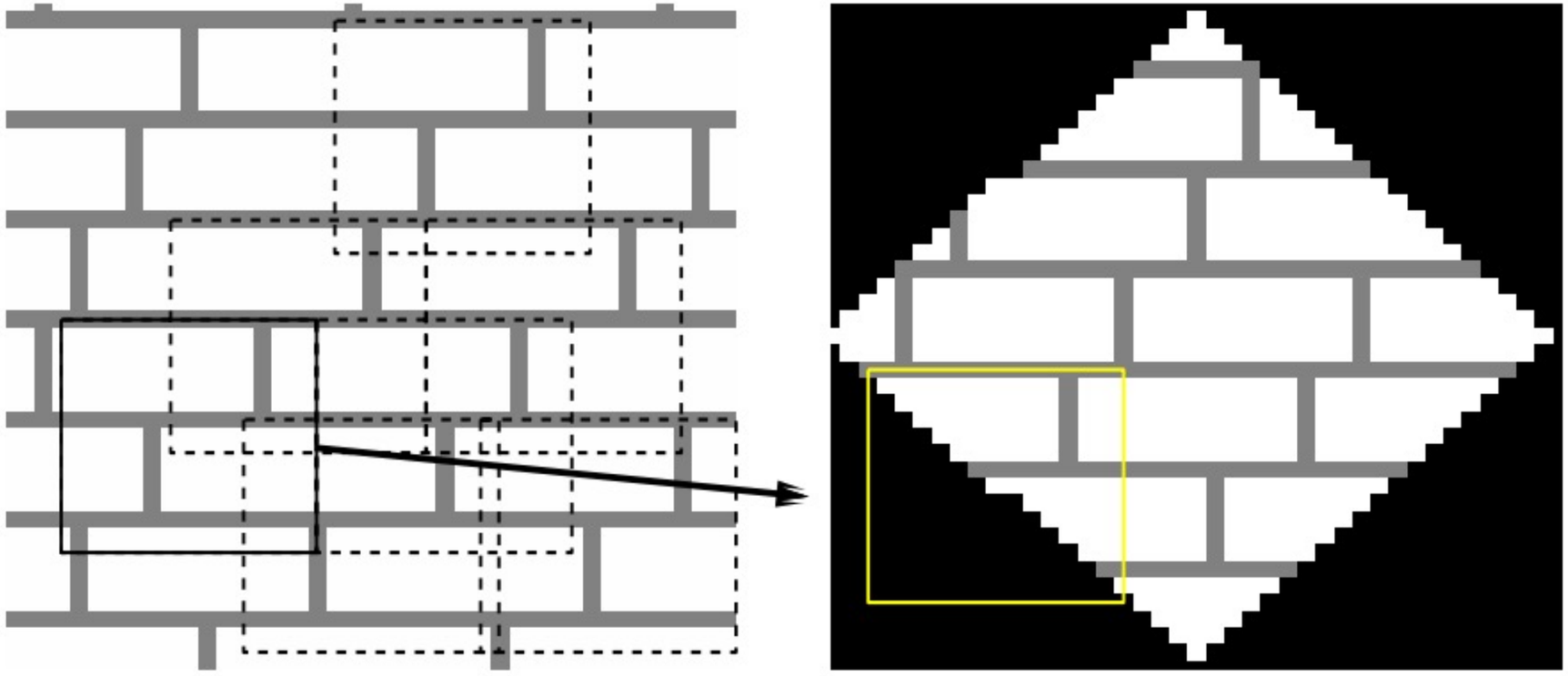


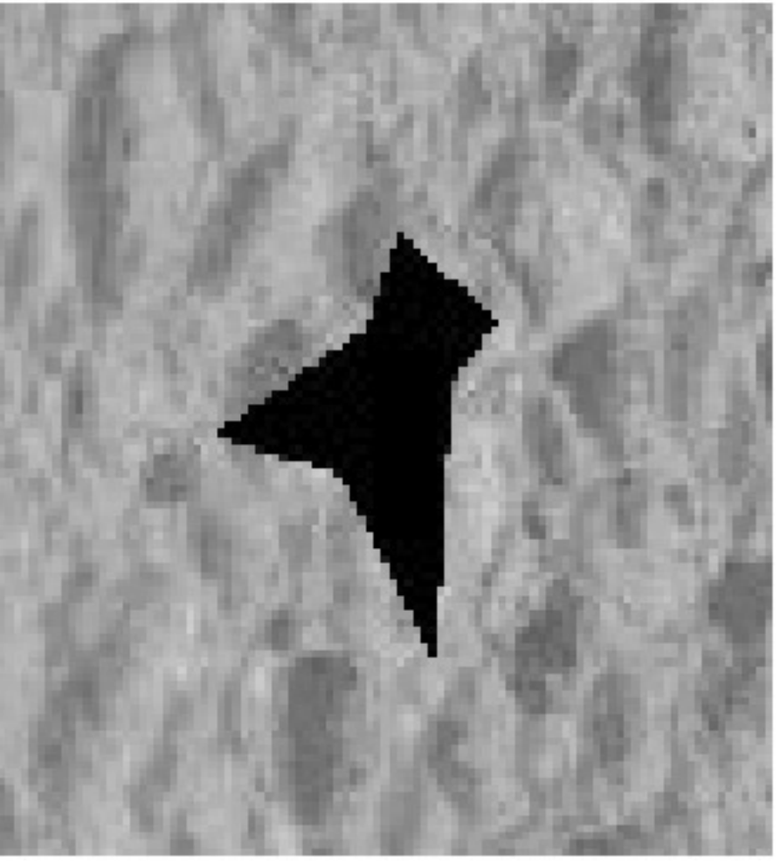
Figure 1. Algorithm Overview. Given a sample texture image (left), a new image is being synthesized one pixel at a time (right). To synthesize a pixel, the algorithm first finds all neighborhoods in the sample image (boxes on the left) that are similar to the pixel's neighborhood (box on the right) and then randomly chooses one neighborhood and takes its center to be the newly synthesized pixel.

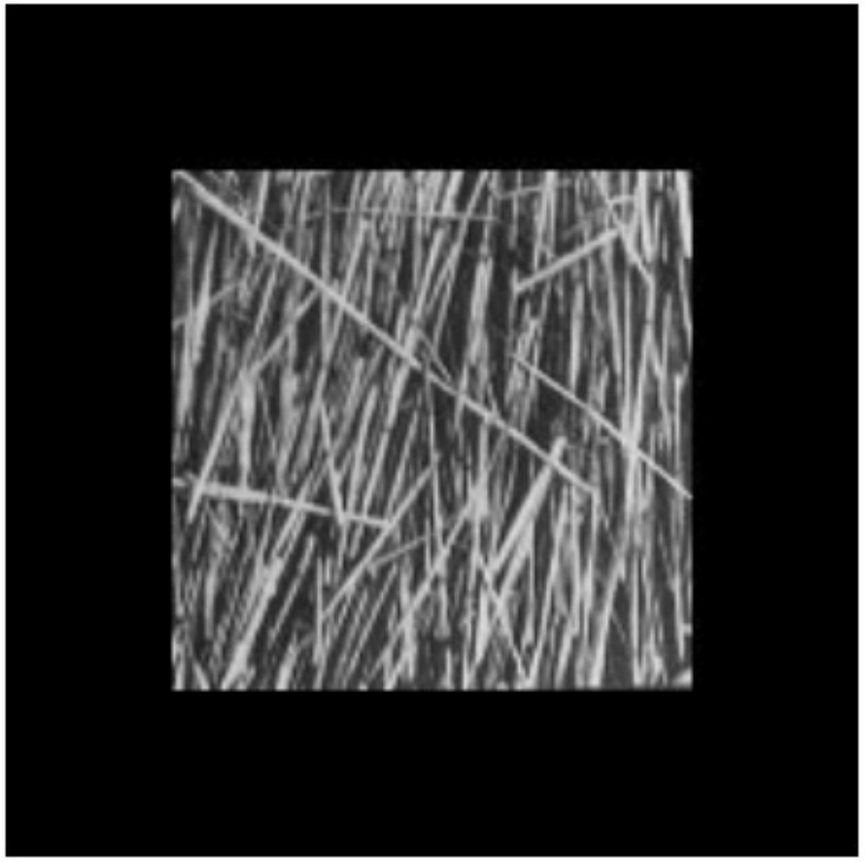
Texture Synthesis

- Can replace more-than-1-pixel at a time
- In action:
  3 examples

Texture Synthesis

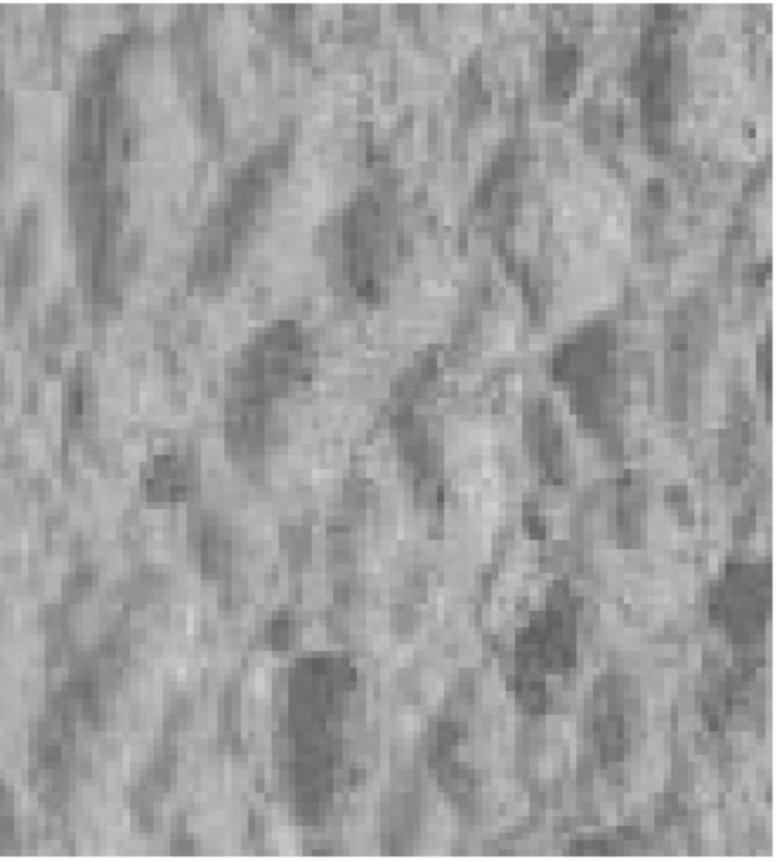
- More results: 3 input images below





Texture Synthesis

- More results: output images for input images on previous slide



Texture Synthesis

- Results on color images